

Lecture 15

Breadth-first Search, Dijkstra

Source: Introduction to Algorithms, CLRS and Kleinberg & Tardos

Reachability

Reachability

Reachability:

Reachability

Reachability:

Input: A directed graph G and its two vertices s and t .

Reachability

Reachability:

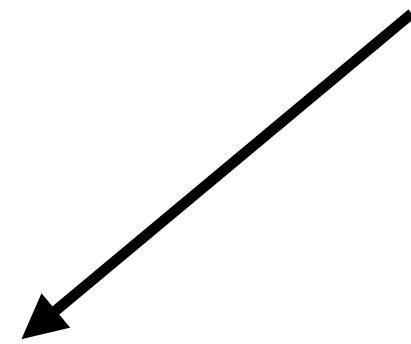
Input: A directed graph G and its two vertices s and t .

Output: Output "Yes" if there is a path from s to t , otherwise, output "No".

Reachability

Problem can be stated and solved for **undirected** graphs too

Reachability:



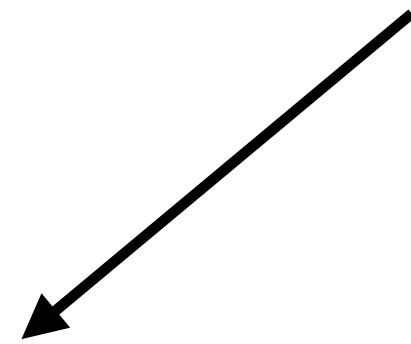
Input: A directed graph G and its two vertices s and t .

Output: Output "Yes" if there is a path from s to t , otherwise, output "No".

Reachability

Problem can be stated and solved for **undirected graphs** too

Reachability:



Input: A directed graph G and its two vertices s and t .

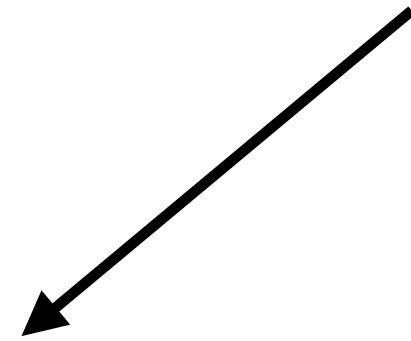
Output: Output "Yes" if there is a path from s to t , otherwise, output "No".

Two standard approaches to solve *Reachability*:

Reachability

Problem can be stated and solved for **undirected** graphs too

Reachability:



Input: A directed graph G and its two vertices s and t .

Output: Output "Yes" if there is a path from s to t , otherwise, output "No".

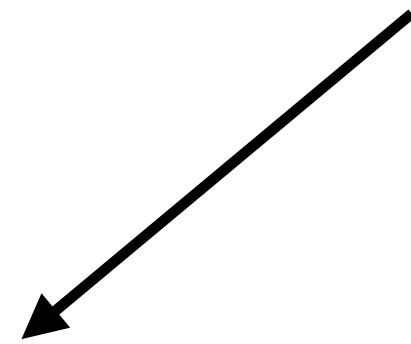
Two standard approaches to solve *Reachability*:

- Depth-first Search.

Reachability

Problem can be stated and solved for **undirected** graphs too

Reachability:



Input: A directed graph G and its two vertices s and t .

Output: Output "Yes" if there is a path from s to t , otherwise, output "No".

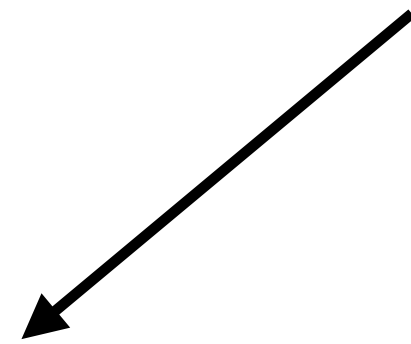
Two standard approaches to solve *Reachability*:

- Depth-first Search.
- Breadth-first Search.

Reachability

Problem can be stated and solved for **undirected graphs** too

Reachability:



Input: A directed graph G and its two vertices s and t .

Output: Output "Yes" if there is a path from s to t , otherwise, output "No".

Two standard approaches to solve *Reachability*:

- Depth-first Search.
- Breadth-first Search.

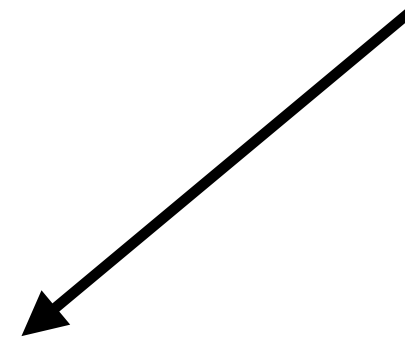


Can compute **topological order**, **strongly connected component**, etc.

Reachability

Problem can be stated and solved for **undirected graphs** too

Reachability:

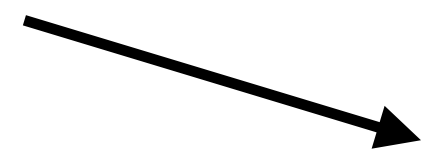


Input: A directed graph G and its two vertices s and t .

Output: Output "Yes" if there is a path from s to t , otherwise, output "No".

Two standard approaches to solve *Reachability*:

- Depth-first Search.
- Breadth-first Search.



Can find **shortest path** as well.

Breadth-First Search: Idea

Breadth-First Search: Idea

BFS visits vertices of a graph G ,

Breadth-First Search: Idea

BFS visits vertices of a graph G ,
starting from vertex s in this manner

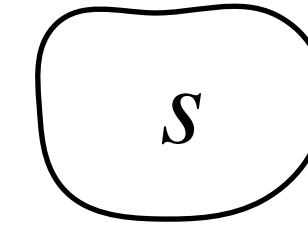
Breadth-First Search: Idea

BFS visits vertices of a graph G ,
starting from vertex s in this manner



Breadth-First Search: Idea

BFS visits vertices of a graph G ,
starting from vertex s in this manner



Breadth-First Search: Idea

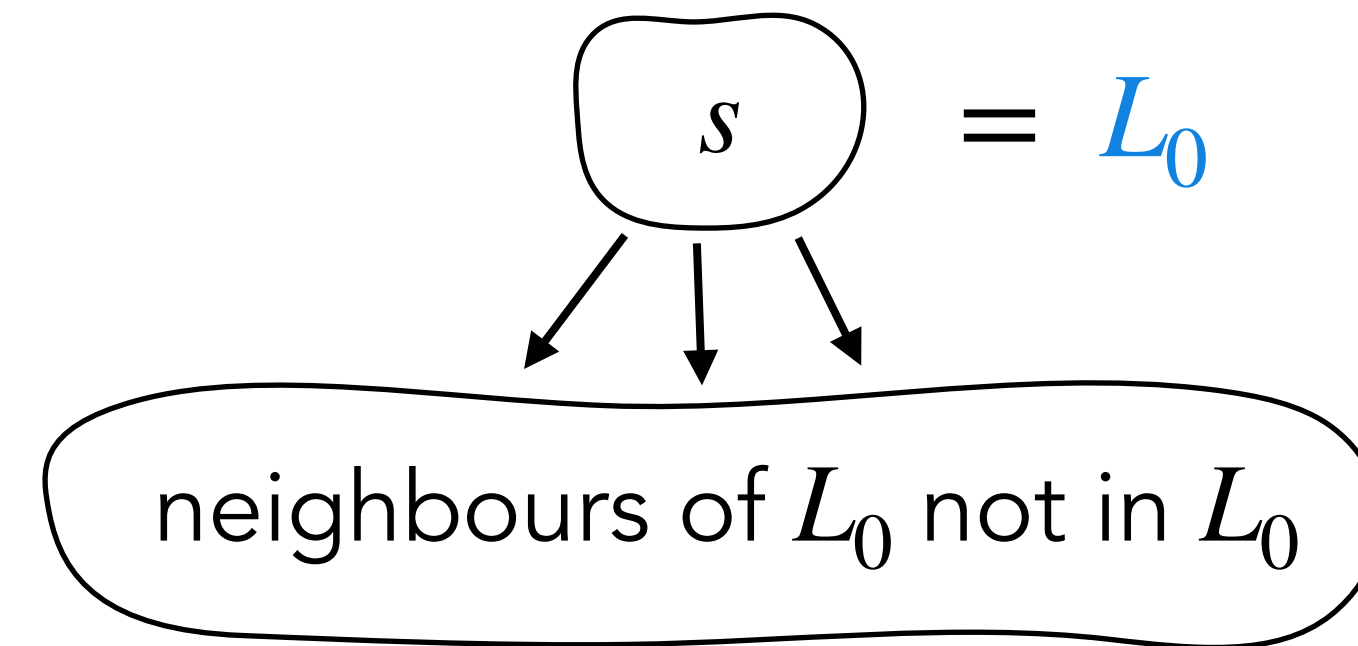
BFS visits vertices of a graph G ,
starting from vertex s in this manner



$$\boxed{s} = L_0$$

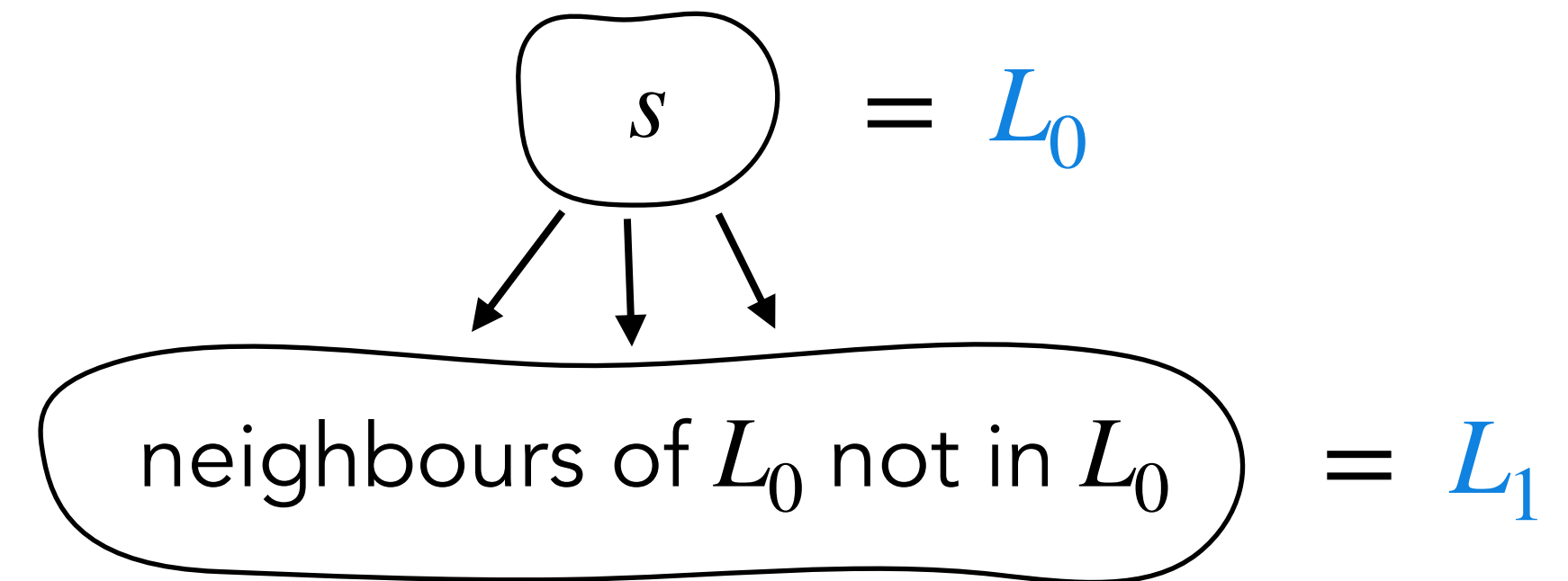
Breadth-First Search: Idea

BFS visits vertices of a graph G ,
starting from vertex s in this manner



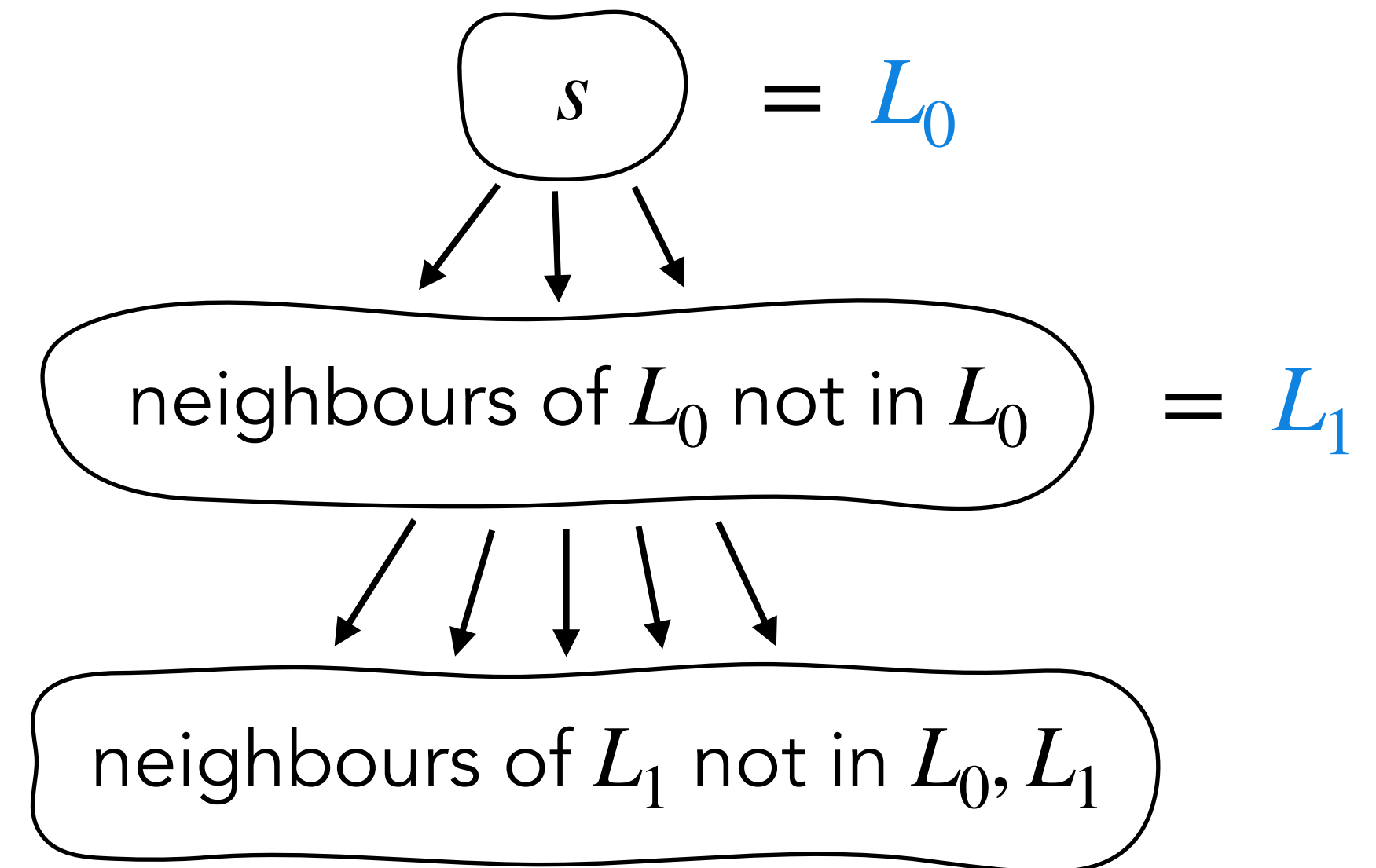
Breadth-First Search: Idea

BFS visits vertices of a graph G ,
starting from vertex s in this manner



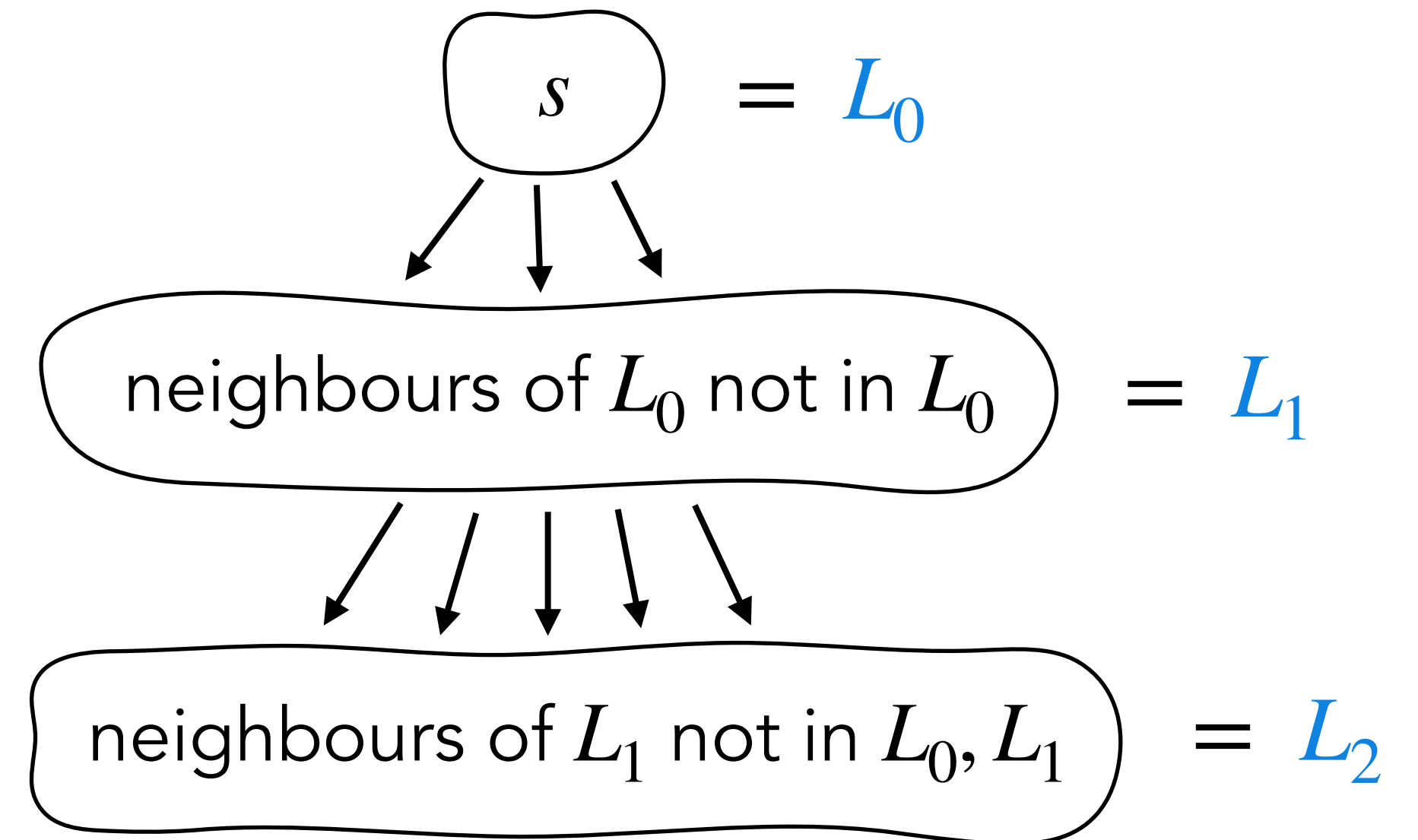
Breadth-First Search: Idea

BFS visits vertices of a graph G ,
starting from vertex s in this manner



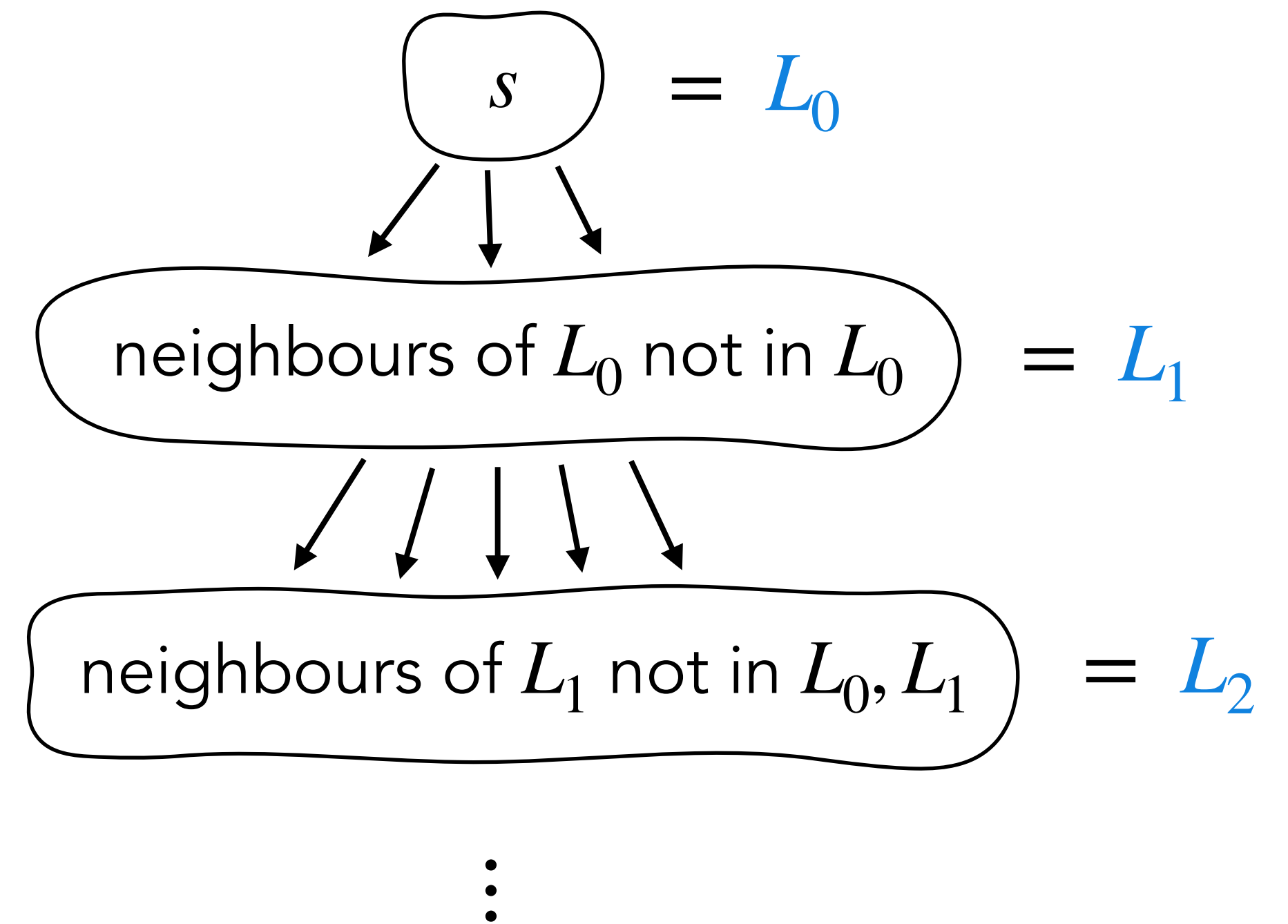
Breadth-First Search: Idea

BFS visits vertices of a graph G ,
starting from vertex s in this manner



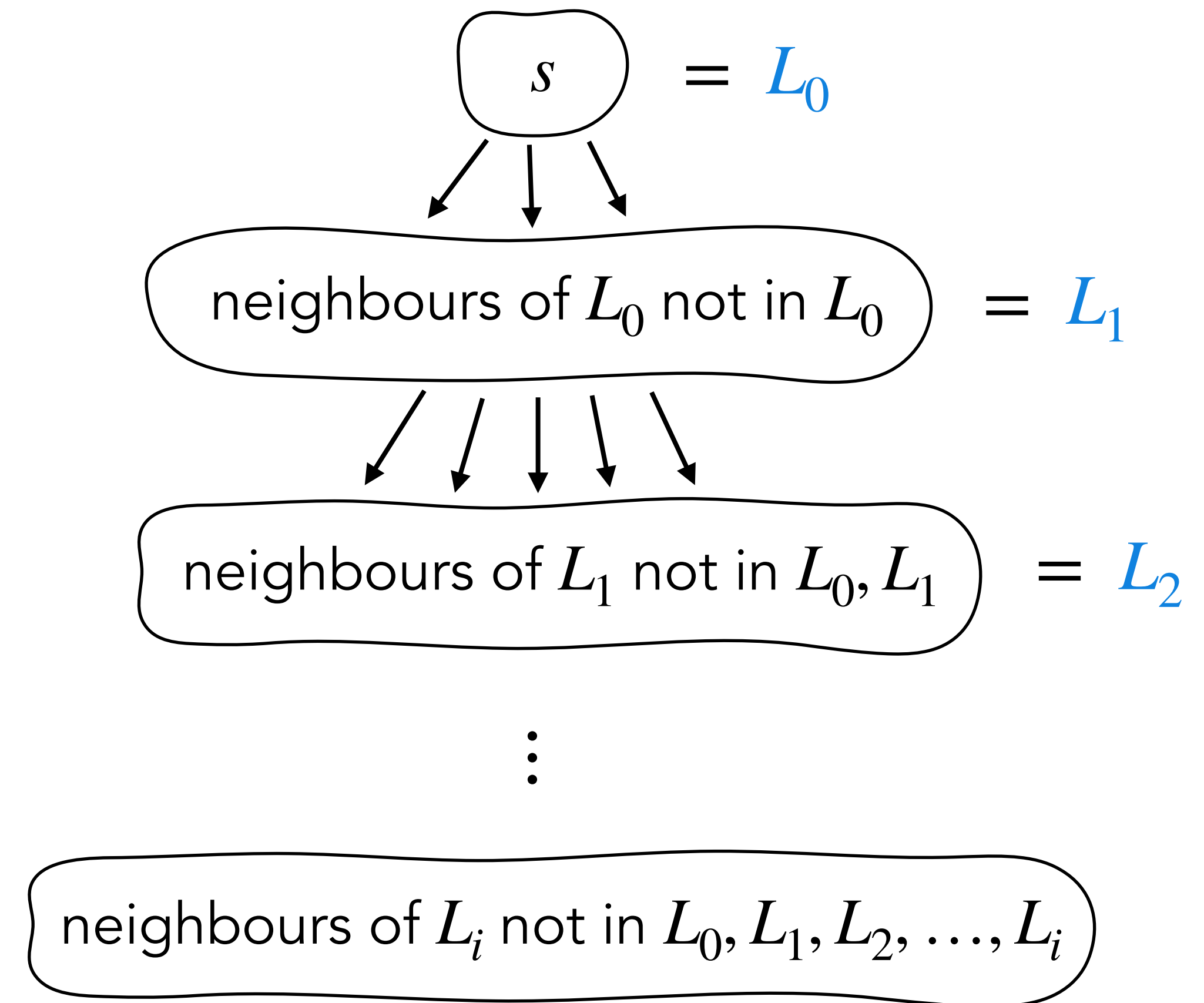
Breadth-First Search: Idea

BFS visits vertices of a graph G ,
starting from vertex s in this manner



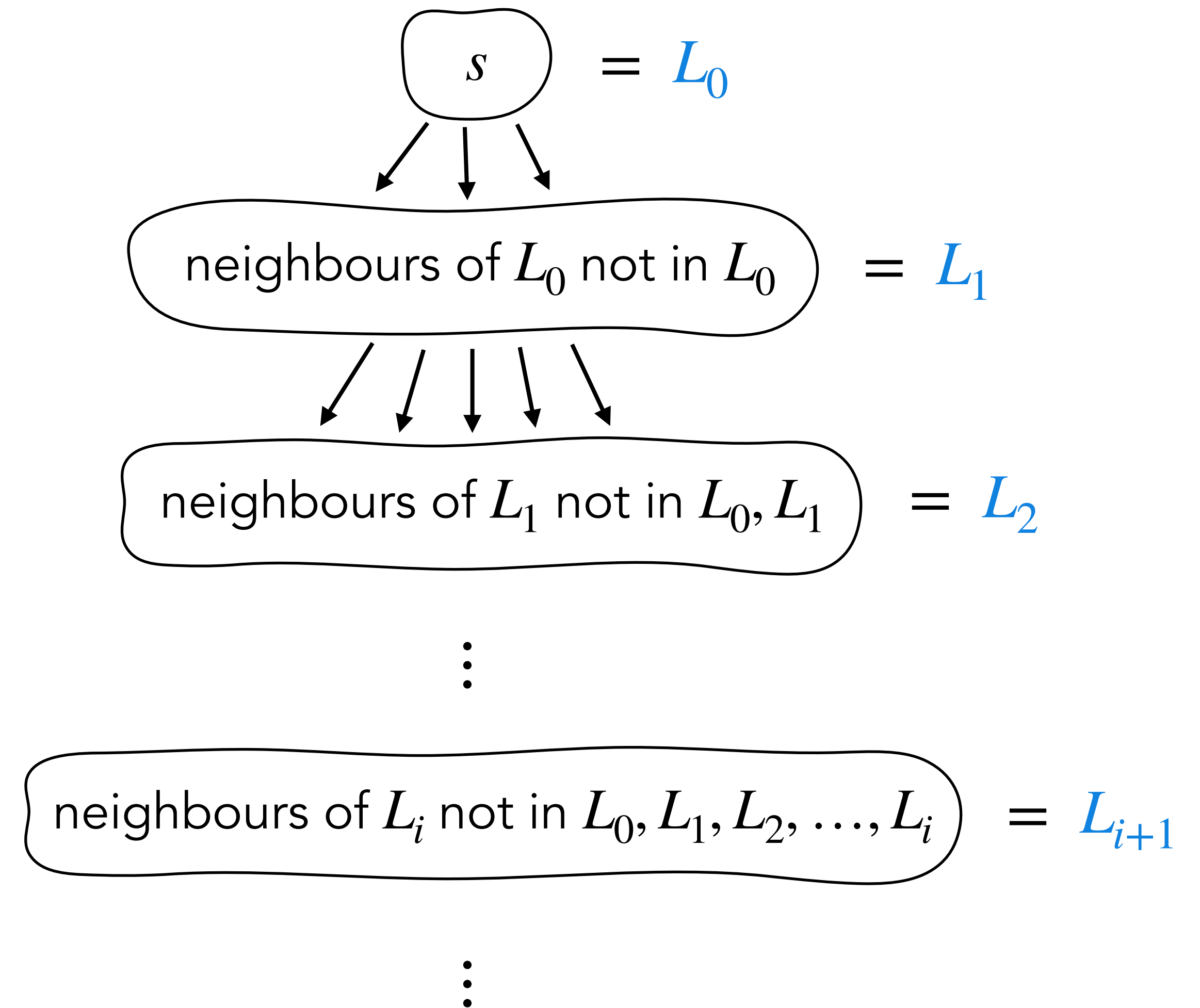
Breadth-First Search: Idea

BFS visits vertices of a graph G ,
starting from vertex s in this manner

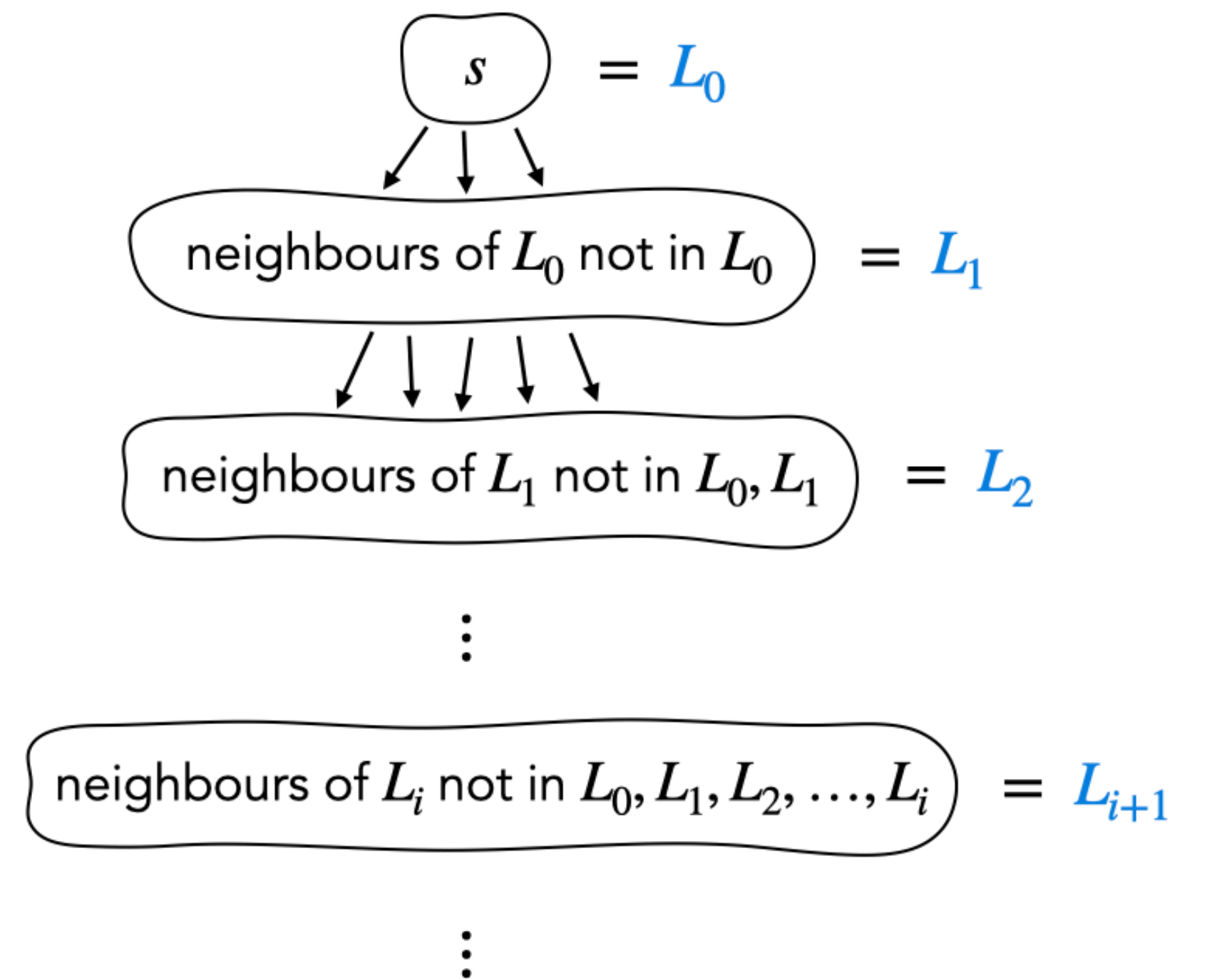


Breadth-First Search: Idea

BFS visits vertices of a graph G ,
starting from vertex s in this manner

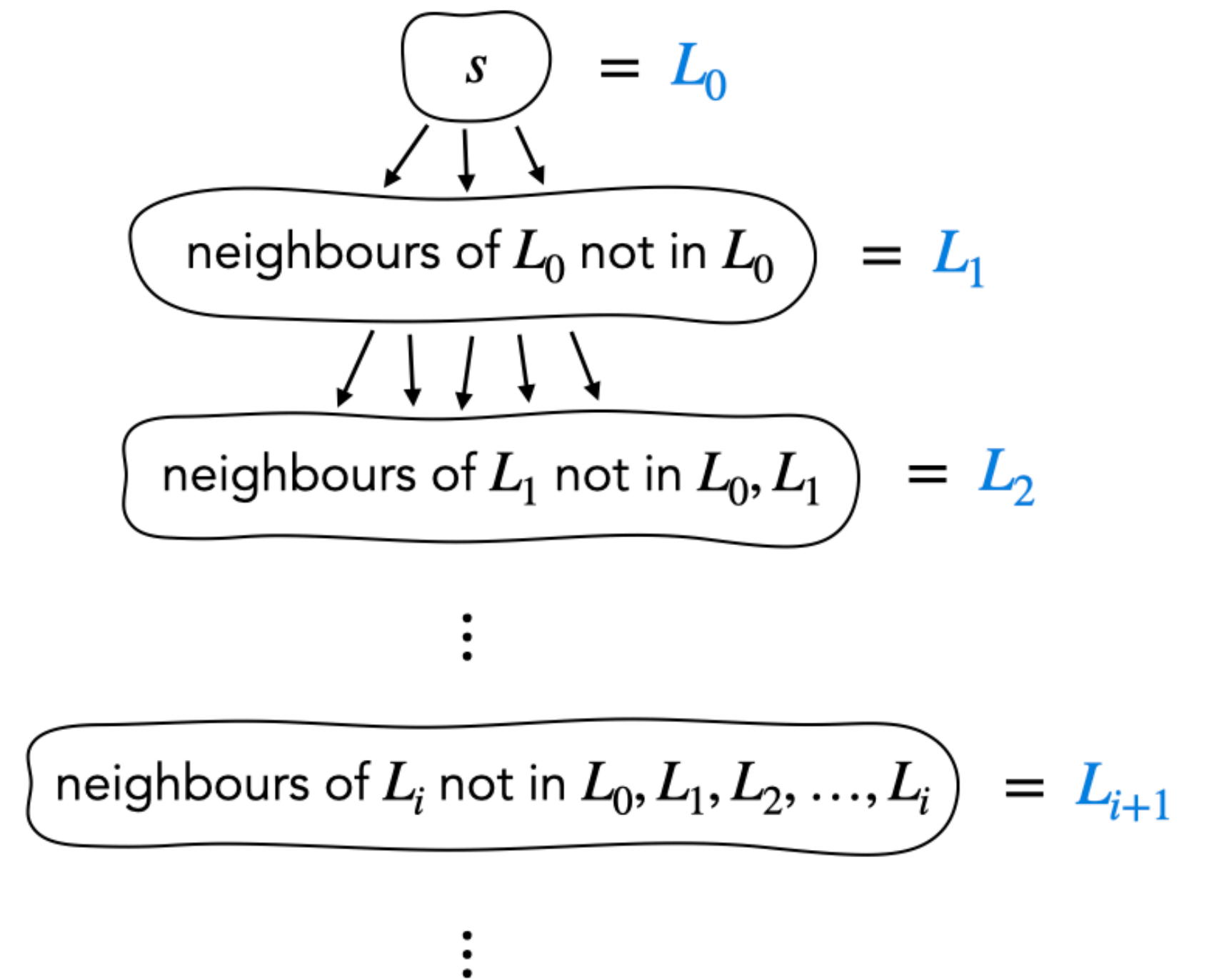


Breadth-First Search: Implementation



Breadth-First Search: Implementation

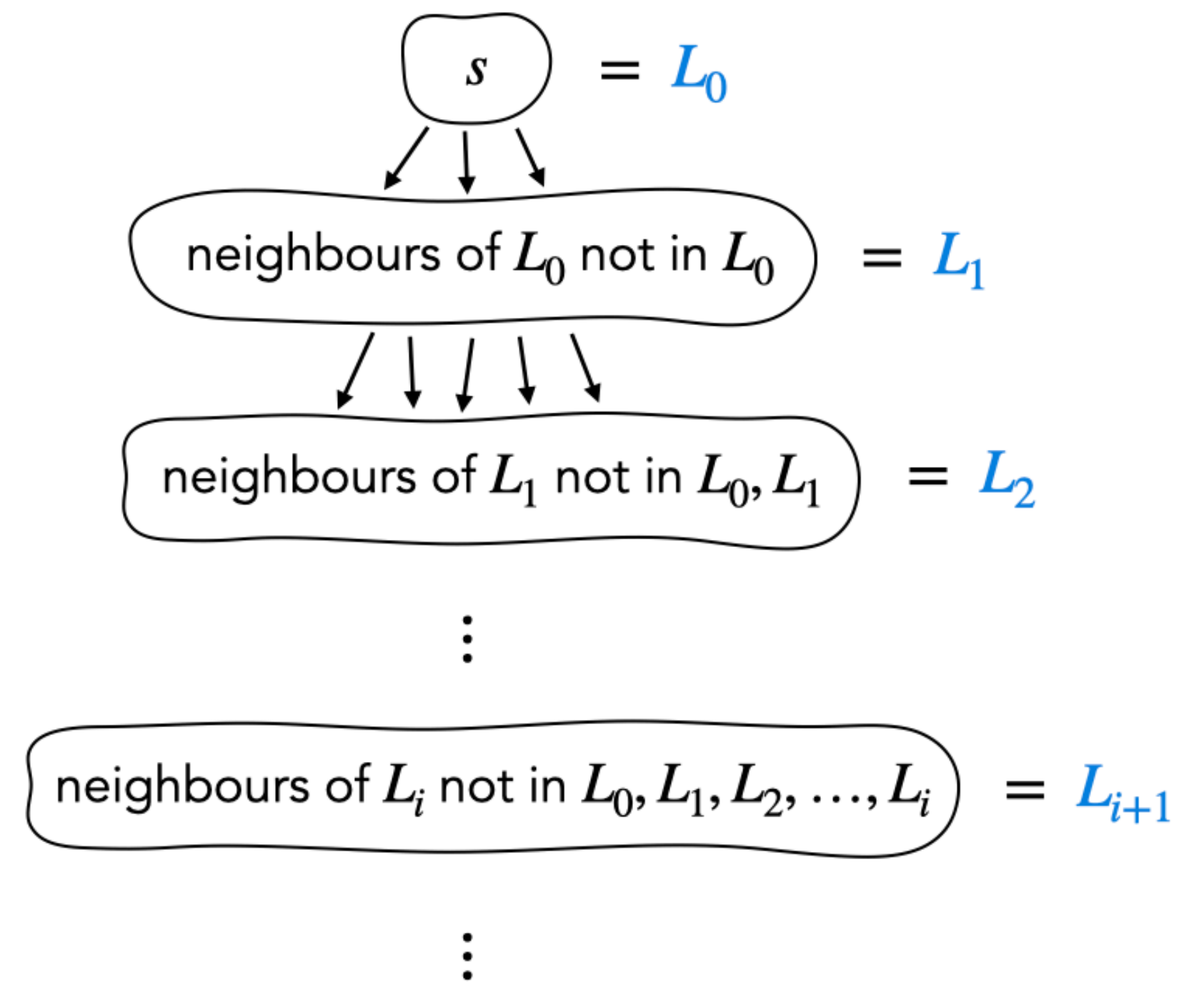
BFS visits the vertices in a level wise manner using a **queue** Q :



Breadth-First Search: Implementation

BFS visits the vertices in a level wise manner using a **queue Q** :

Step 1: Visits vertices of L_0 and put them in Q .

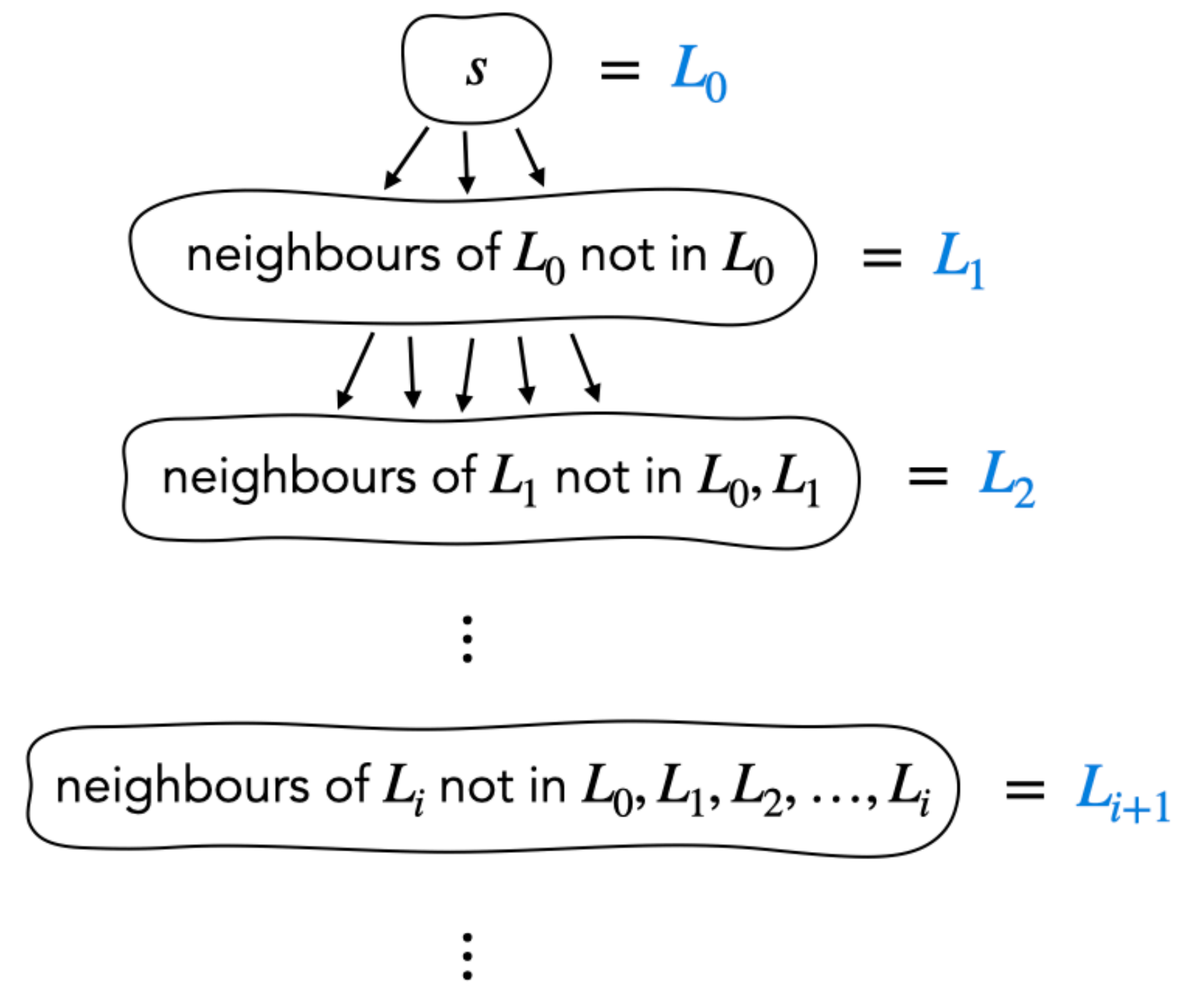


Breadth-First Search: Implementation

BFS visits the vertices in a level wise manner using a **queue** Q :

Step 1: Visits vertices of L_0 and put them in Q .

Step 2: Let $u = \text{Dequeue}(Q)$.



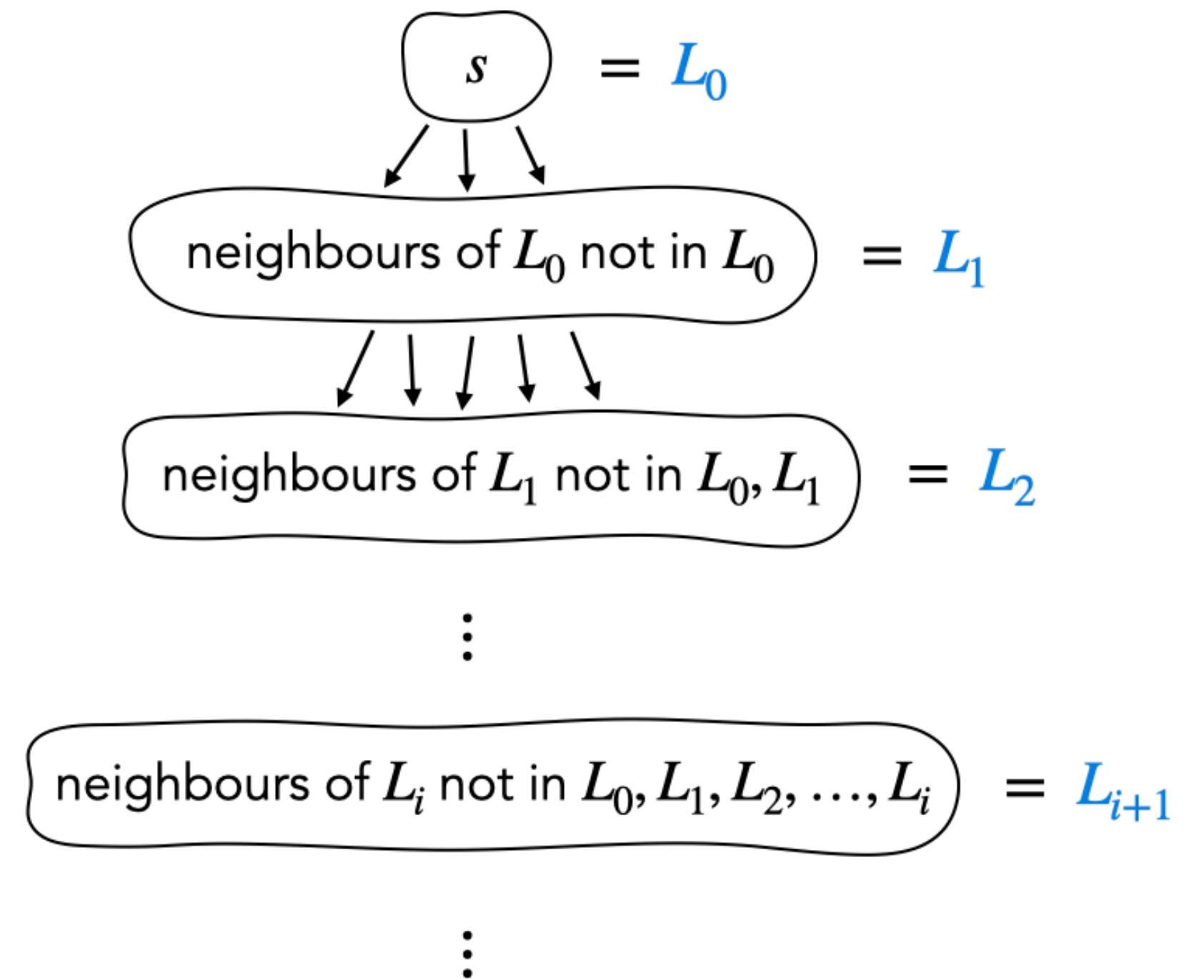
Breadth-First Search: Implementation

BFS visits the vertices in a level wise manner using a **queue** Q :

Step 1: Visits vertices of L_0 and put them in Q .

Step 2: Let $u = \text{Dequeue}(Q)$.

- Visit unvisited neighbours of u .



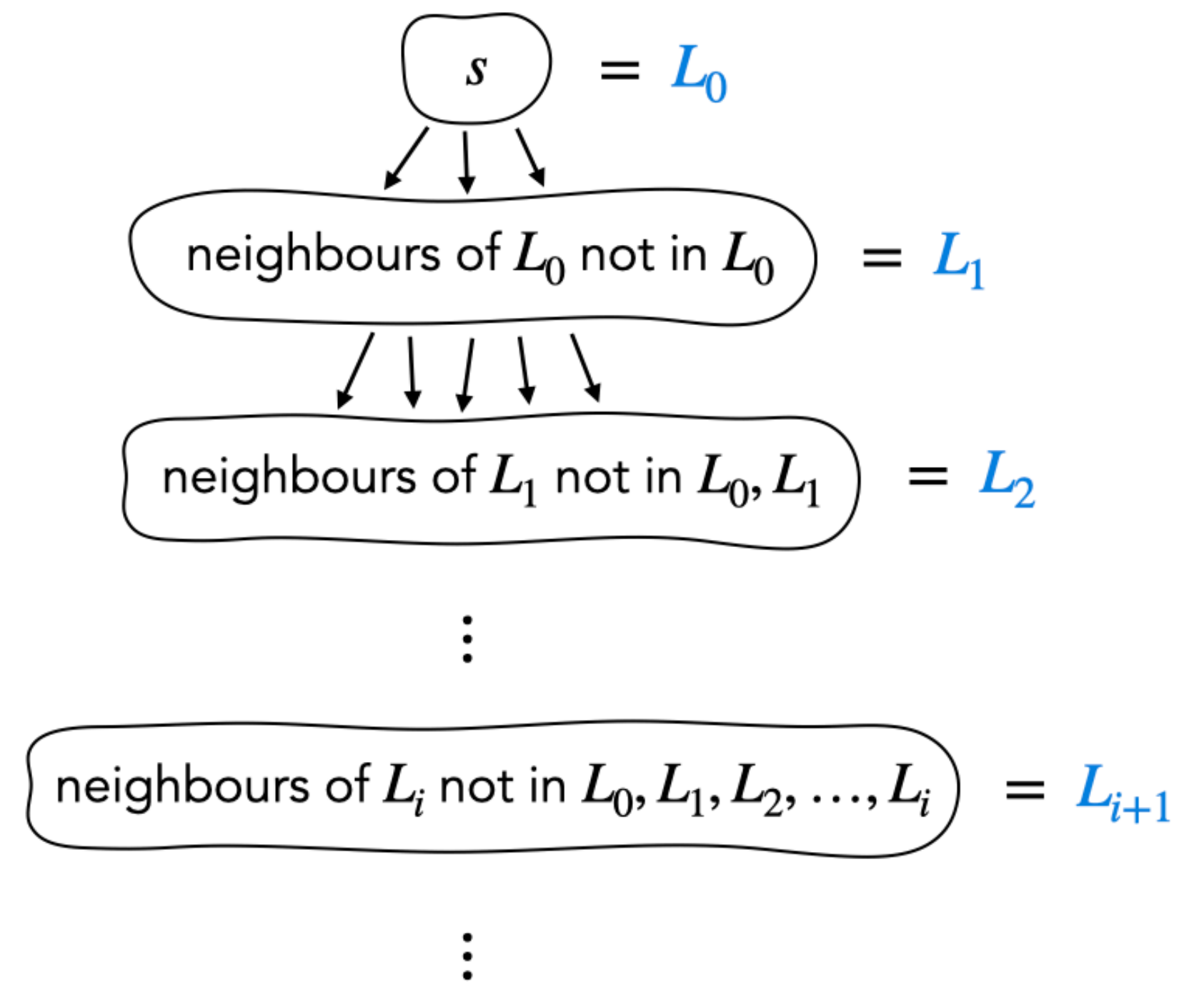
Breadth-First Search: Implementation

BFS visits the vertices in a level wise manner using a **queue** Q :

Step 1: Visits vertices of L_0 and put them in Q .

Step 2: Let $u = \text{Dequeue}(Q)$.

- Visit unvisited neighbours of u .
- Put them in Q .



Breadth-First Search: Implementation

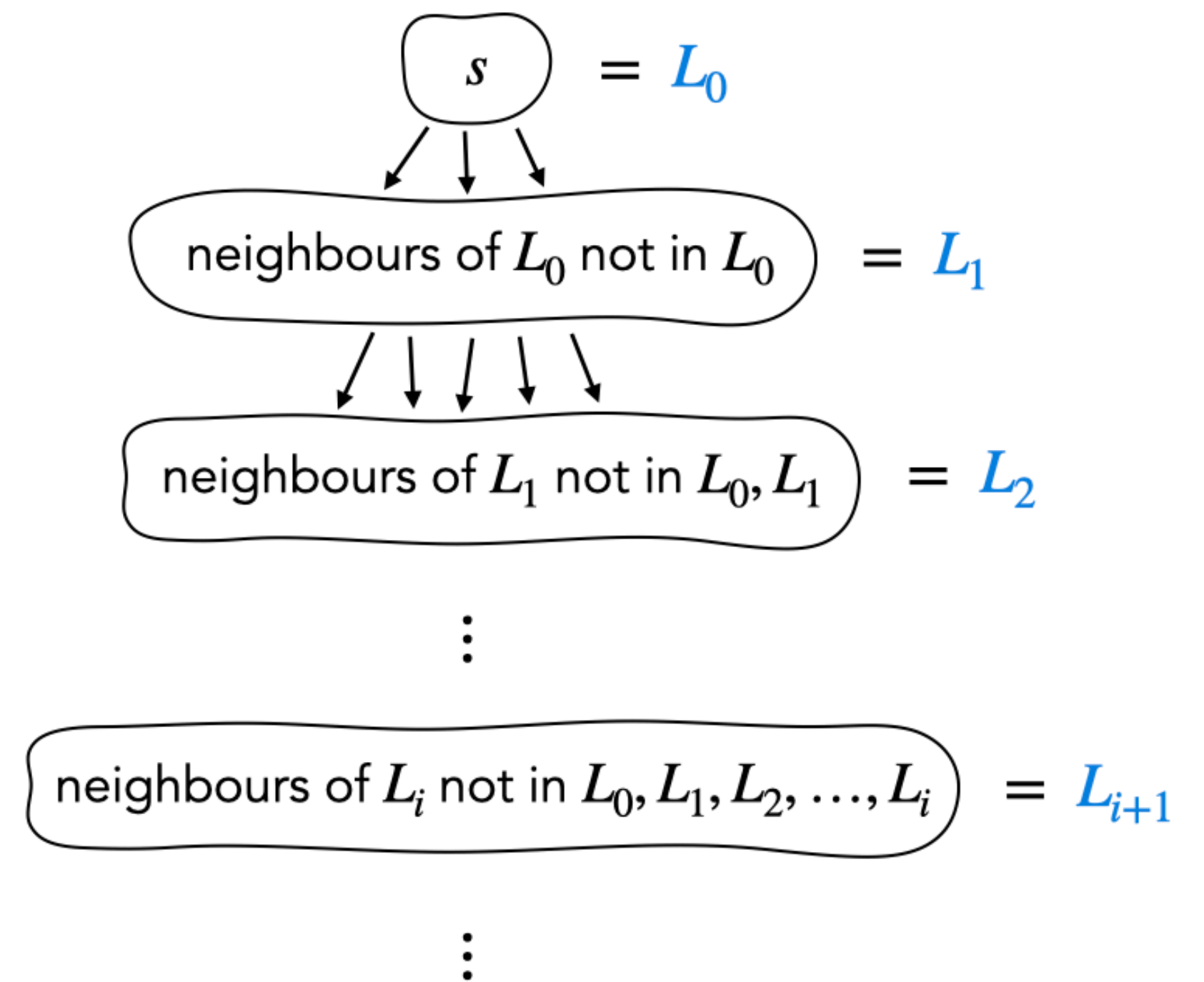
BFS visits the vertices in a level wise manner using a **queue** Q :

Step 1: Visits vertices of L_0 and put them in Q .

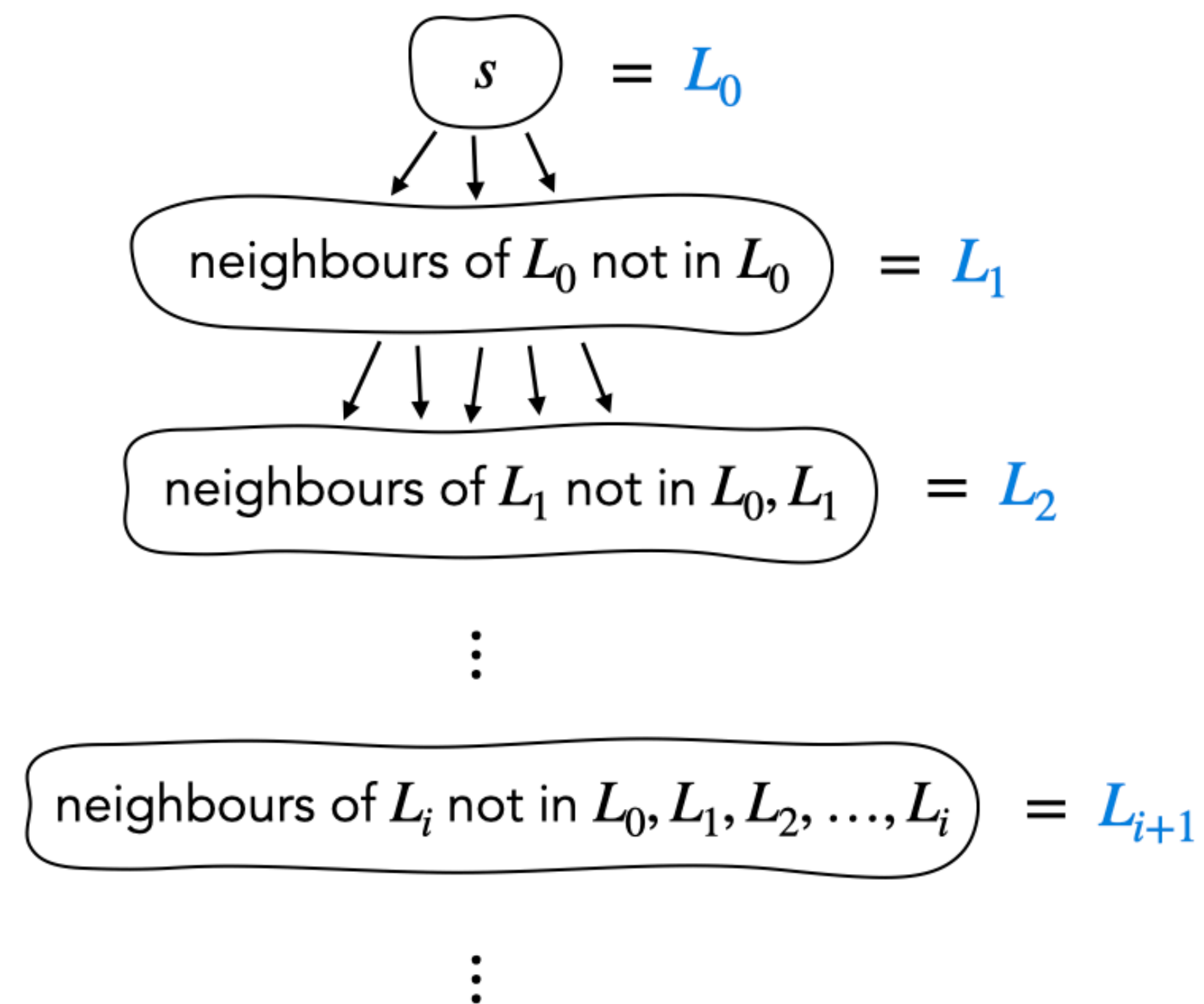
Step 2: Let $u = \text{Dequeue}(Q)$.

- Visit unvisited neighbours of u .
- Put them in Q .

Step 3: Go to **Step 2** if Q is not empty.

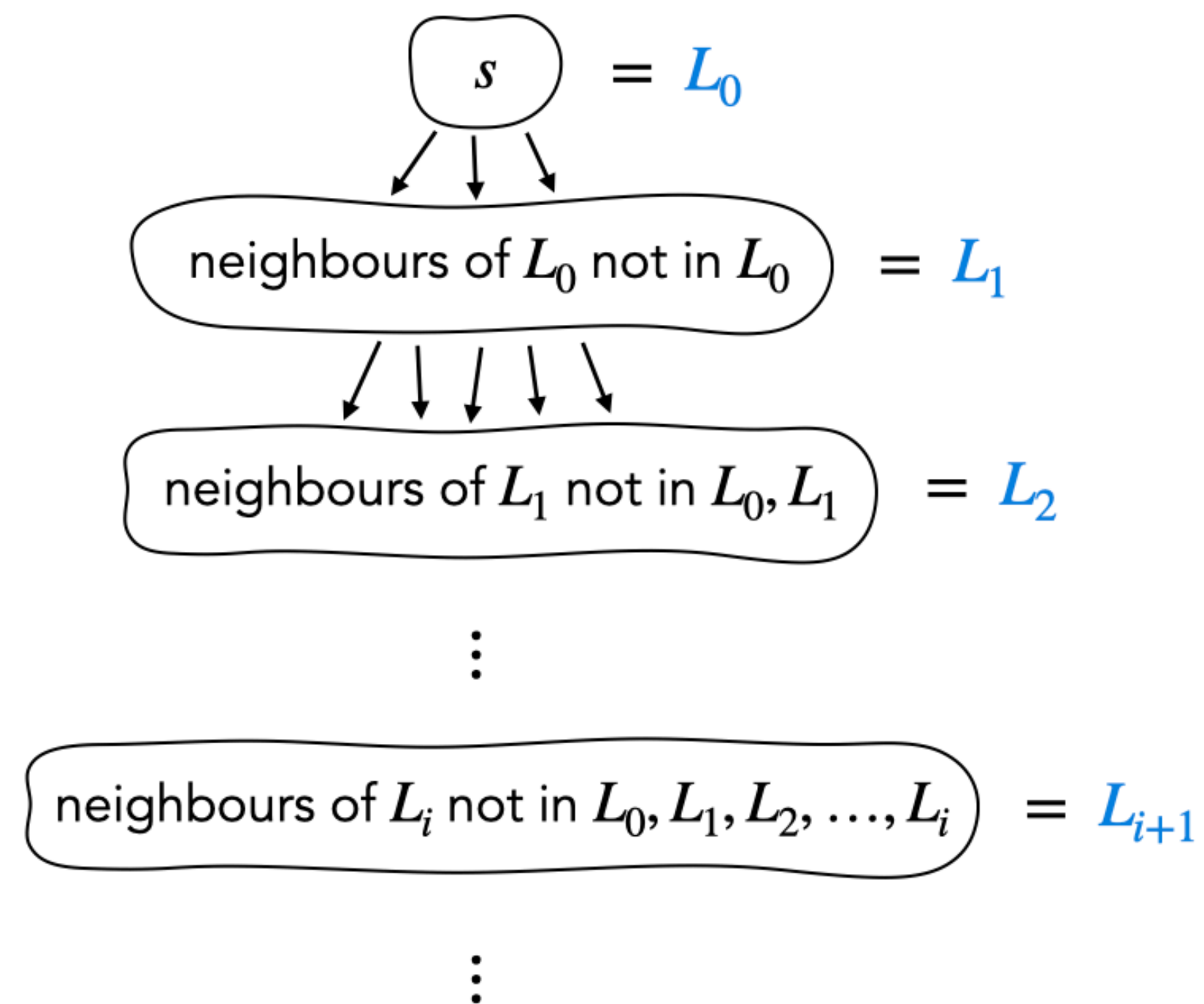


Breadth-First Search: Shortest Path



Breadth-First Search: Shortest Path

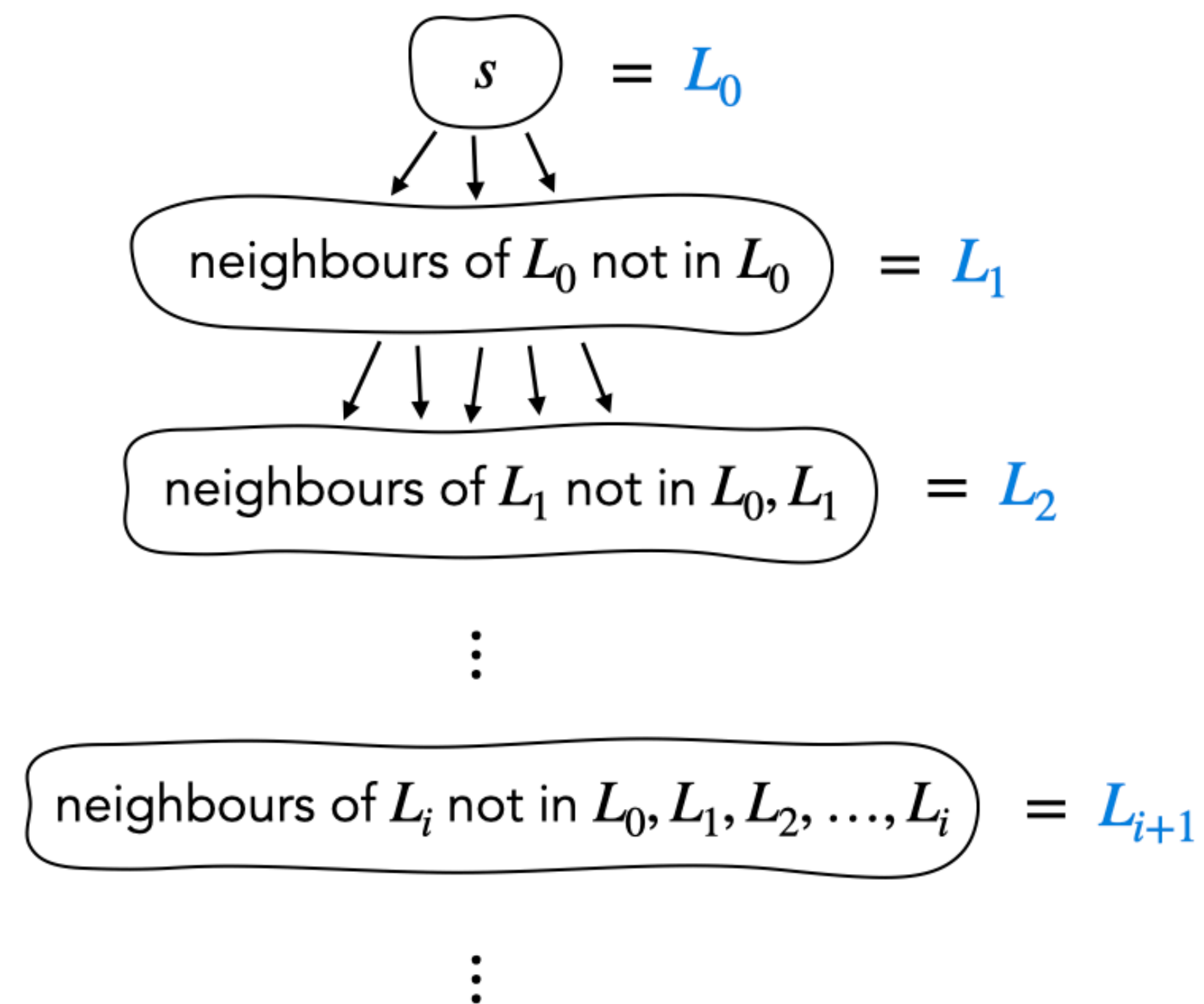
Theorem: A vertex $u \in V(G)$ has distance i from s if and only if $u \in L_i$.



Breadth-First Search: Shortest Path

Theorem: A vertex $u \in V(G)$ has distance i from s if and only if $u \in L_i$.

Proof: DIY.



Breadth-First Search: Pseudocode

Breadth-First Search: Pseudocode

BFS(G, s):

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$ *\\ array d stores distance of vertices*

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$ *\\ array d stores distance of vertices*
3. $Q = \emptyset$

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$ *\\ array d stores distance of vertices*
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$ *\\ setting distance before enqueueing*

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$ *\\ array d stores distance of vertices*
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$ *\\ setting distance before enqueueing*
5. Enqueue(Q, s)

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$ *\\ array d stores distance of vertices*
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$ *\\ setting distance before enqueueing*
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$ *\\ array d stores distance of vertices*
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$ *\\ setting distance before enqueueing*
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$ *\\ array d stores distance of vertices*
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$ *\\ setting distance before enqueueing*
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$ *\\ going over all the neighbours of u*

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$ *\\ array d stores distance of vertices*
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$ *\\ setting distance before enqueueing*
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$ *\\ going over all the neighbours of u*
9. if $visited[v] == false$ *\\ v is being discovered for the first time?*

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$ *\\ array d stores distance of vertices*
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$ *\\ setting distance before enqueueing*
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$ *\\ going over all the neighbours of u*
9. if $visited[v] == false$ *\\ v is being discovered for the first time?*
10. $d[v] = d[u] + 1, visited[v] = true$

Breadth-First Search: Pseudocode

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$ *\\ array d stores distance of vertices*
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$ *\\ setting distance before enqueueing*
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$ *\\ going over all the neighbours of u*
9. if $visited[v] == false$ *\\ v is being discovered for the first time?*
10. $d[v] = d[u] + 1, visited[v] = true$
11. Enqueue(Q, v)

Breadth-First Search: Demonstration

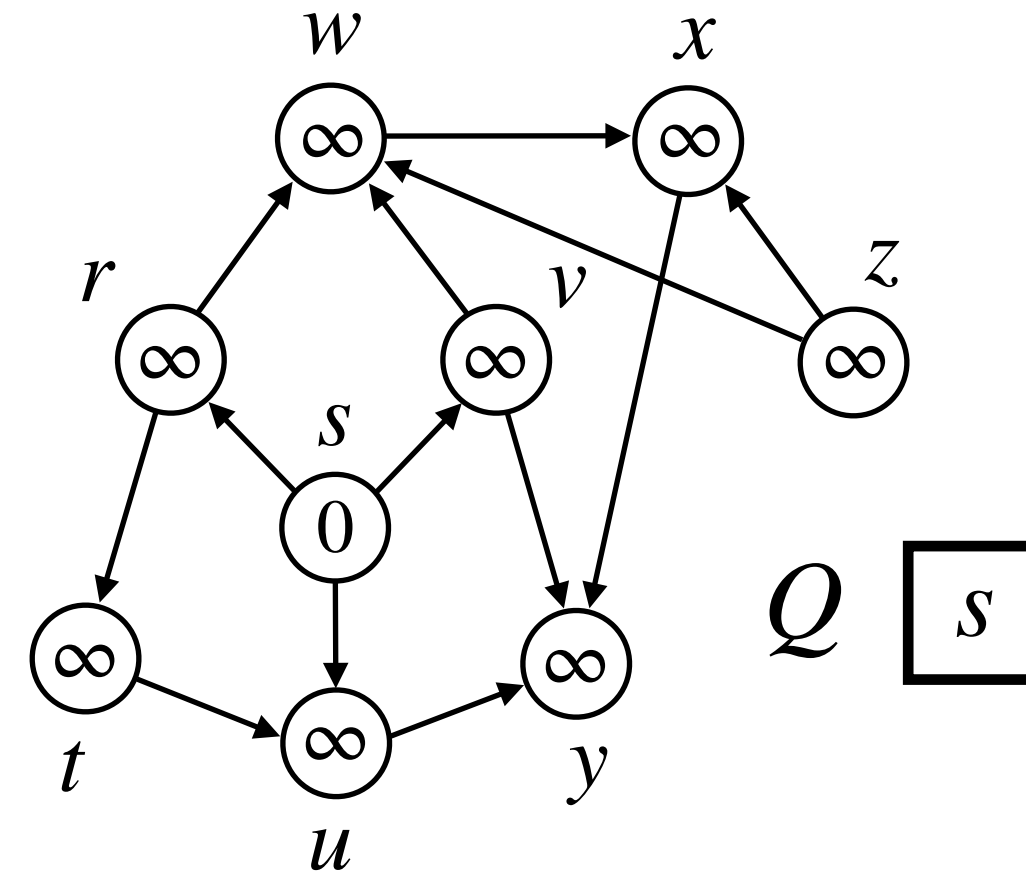
BFS(G, s):

1. **for every** $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$
5. **Enqueue**(Q, s)
6. **while** $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. **for every** v in $Adj[u]$
9. **if** $visited[v] == false$
10. $d[v] = d[u] + 1, visited[v] = true$
11. **Enqueue**(Q, v)

Breadth-First Search: Demonstration

BFS(G, s):

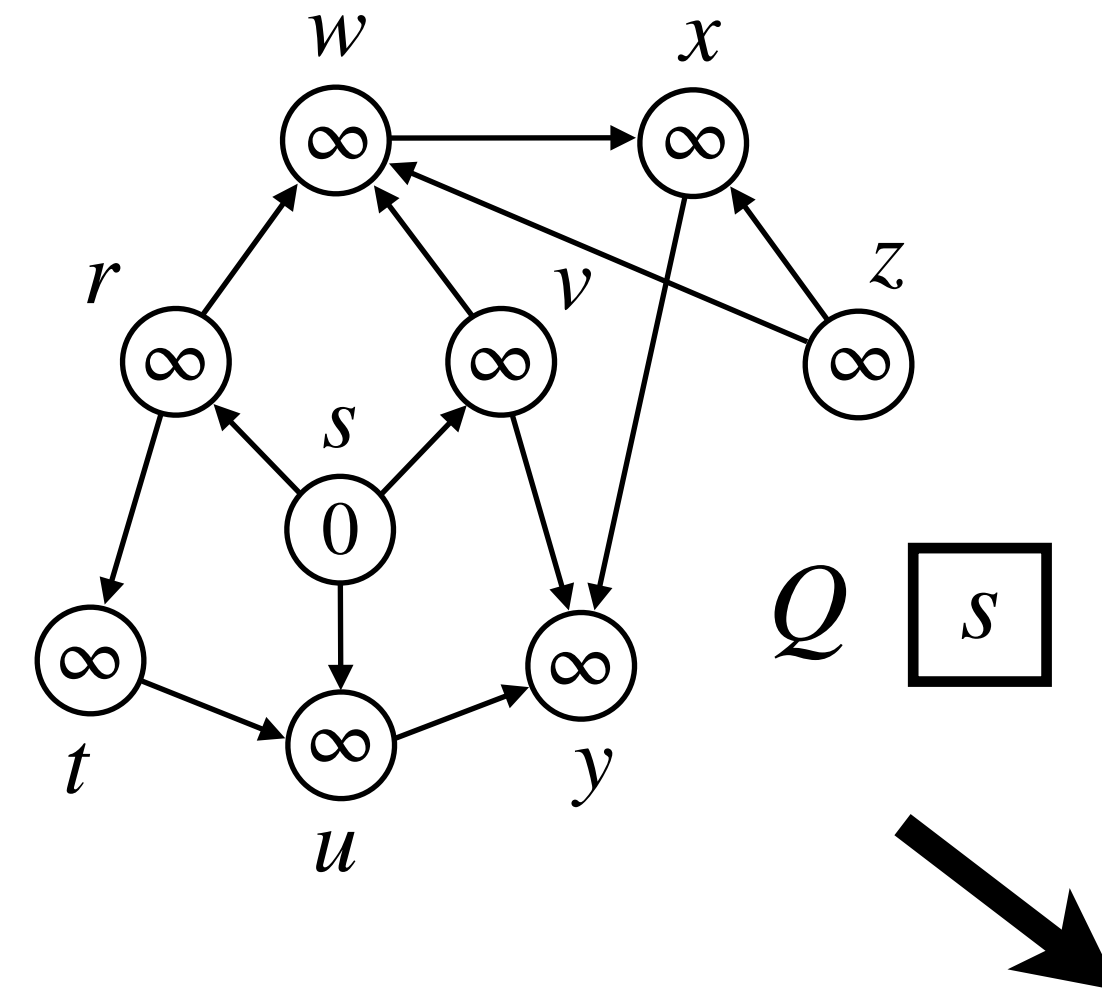
1. **for every** $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$
5. **Enqueue**(Q, s)
6. **while** $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. **for every** v in $Adj[u]$
9. **if** $visited[v] == false$
10. $d[v] = d[u] + 1, visited[v] = true$
11. **Enqueue**(Q, v)



Breadth-First Search: Demonstration

BFS(G, s):

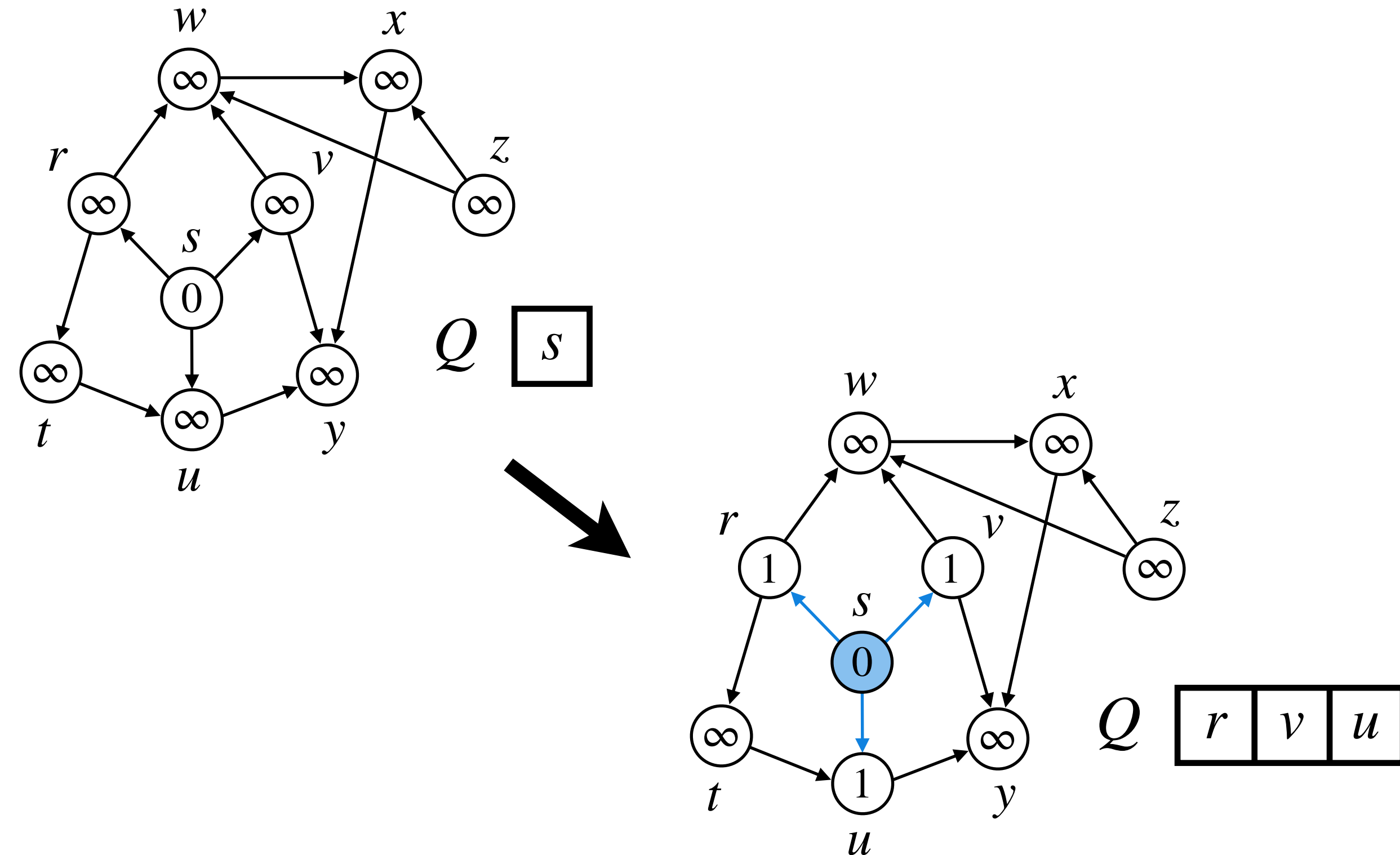
1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty$, $visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0$, $visited[s] = true$
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$
9. if $visited[v] == false$
10. $d[v] = d[u] + 1$, $visited[v] = true$
11. Enqueue(Q, v)



Breadth-First Search: Demonstration

BFS(G, s):

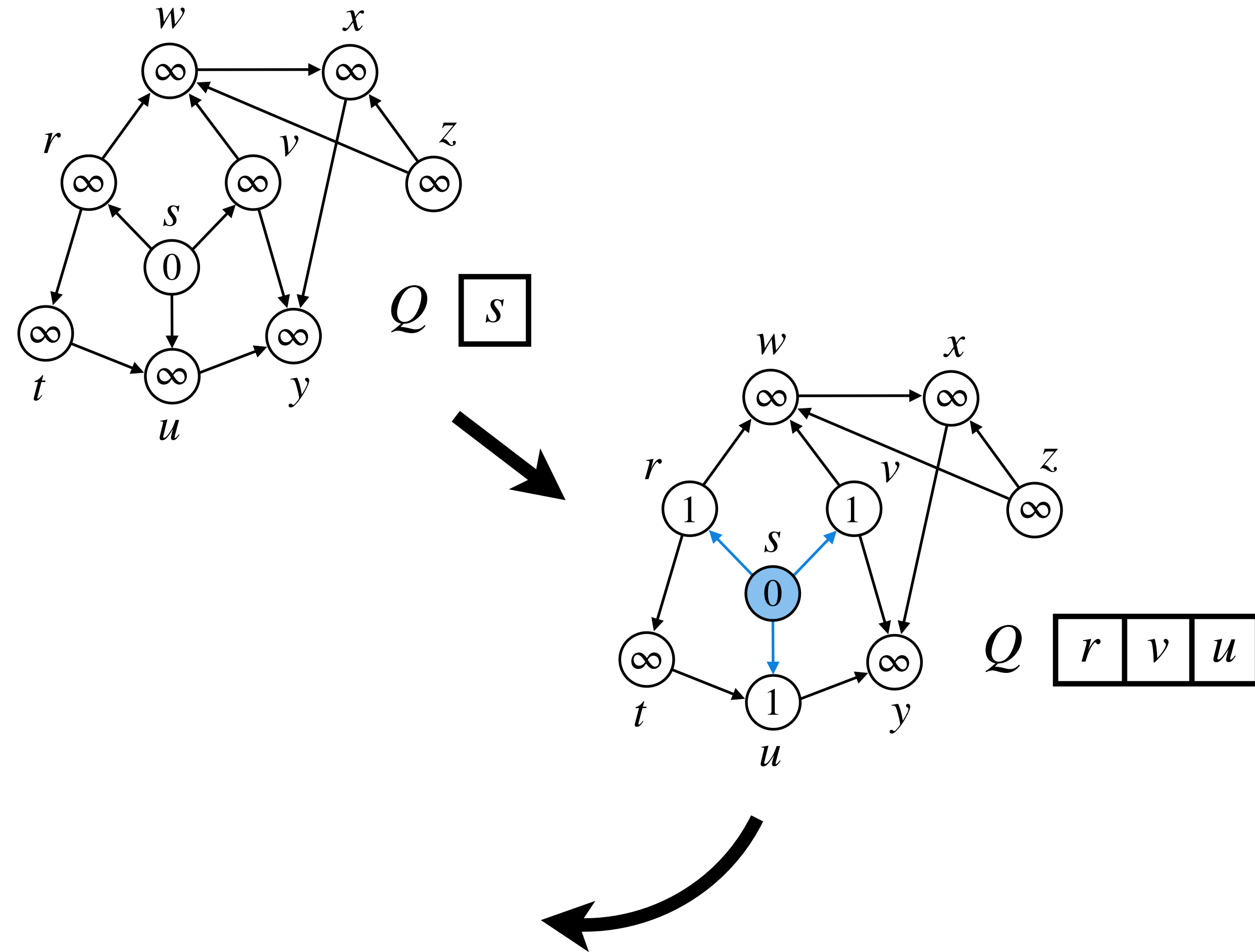
1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty$, $visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0$, $visited[s] = true$
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$
9. if $visited[v] == false$
10. $d[v] = d[u] + 1$, $visited[v] = true$
11. Enqueue(Q, v)



Breadth-First Search: Demonstration

BFS(G, s):

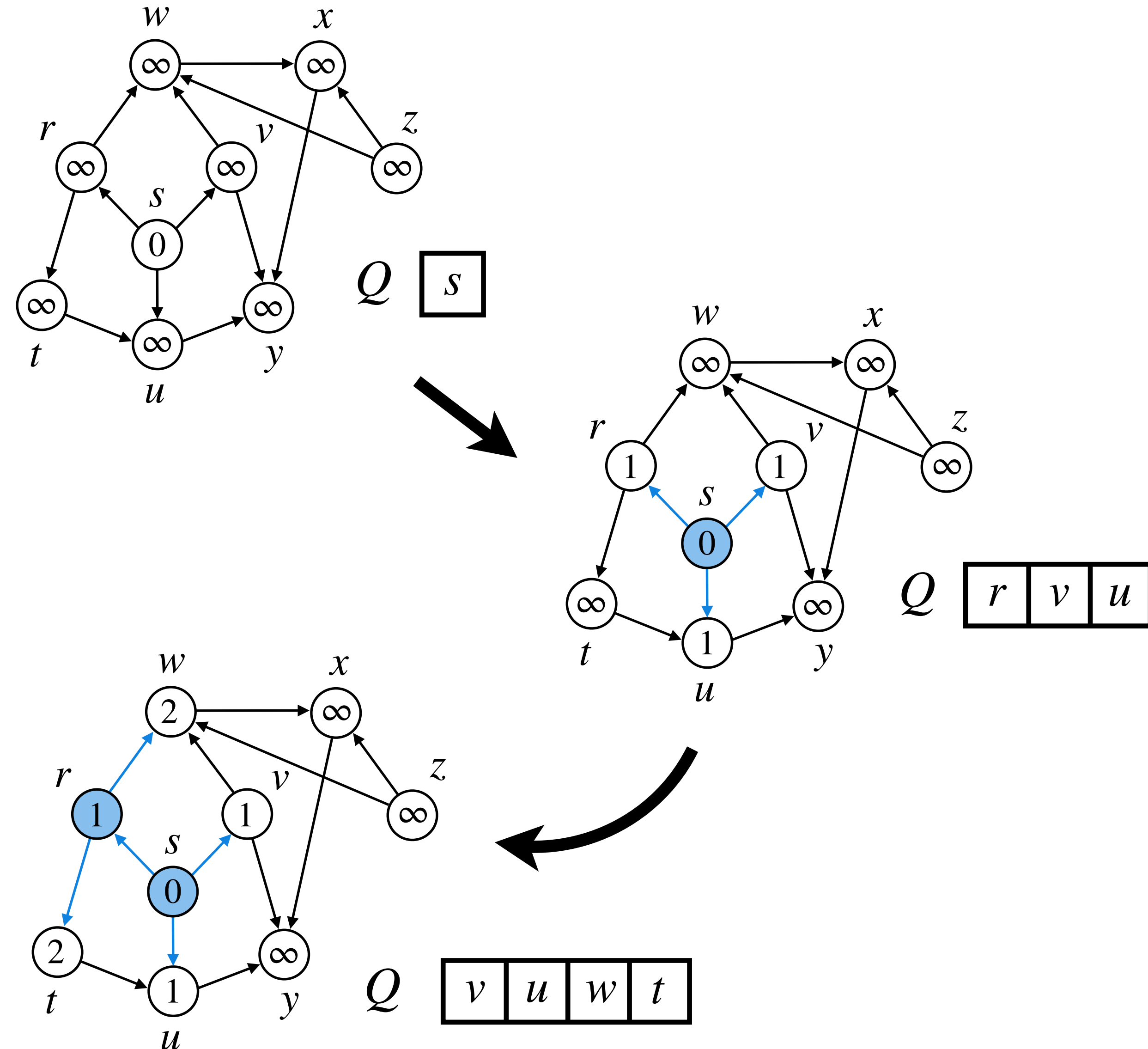
1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty$, $visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0$, $visited[s] = true$
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$
9. if $visited[v] == false$
10. $d[v] = d[u] + 1$, $visited[v] = true$
11. Enqueue(Q, v)



Breadth-First Search: Demonstration

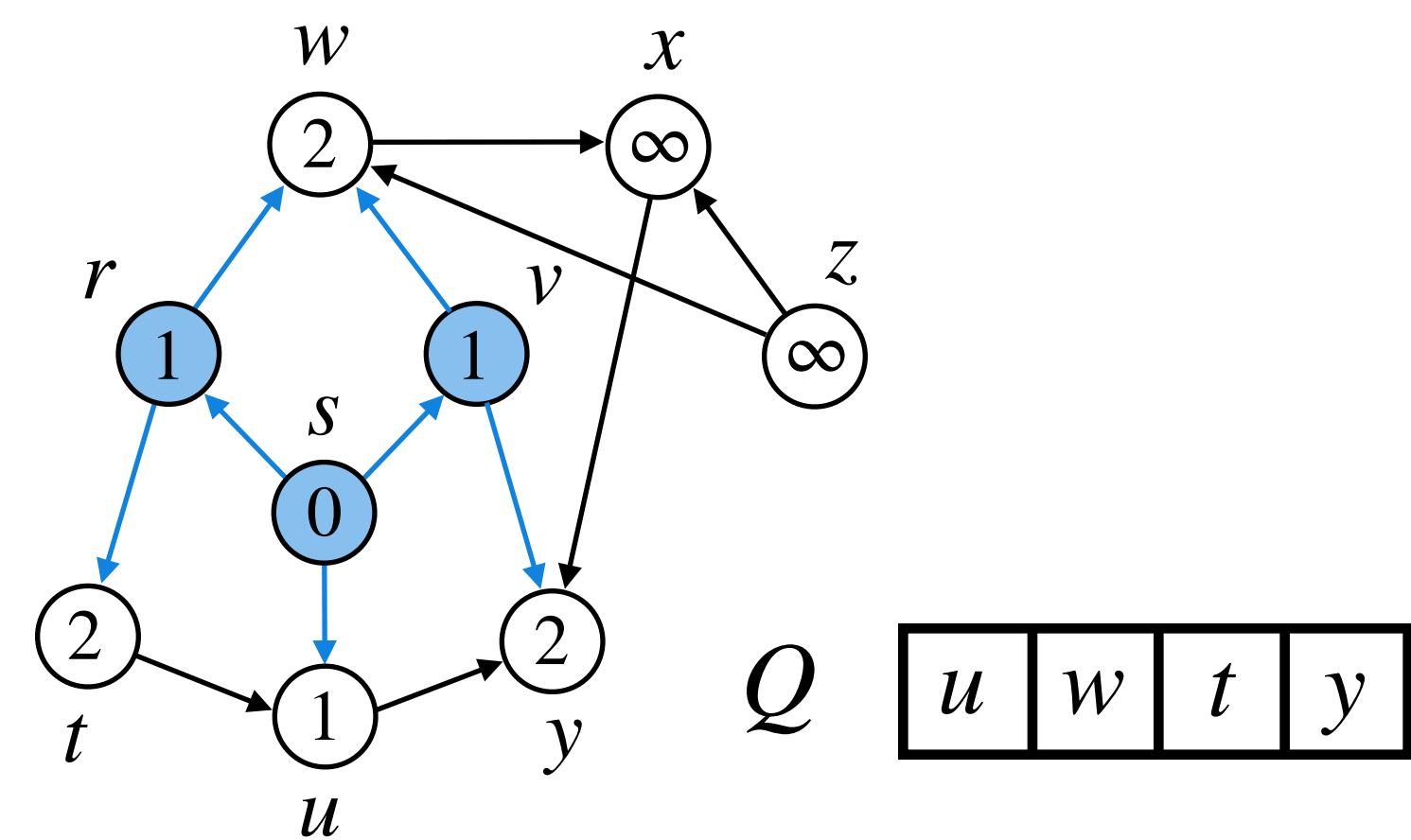
BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty$, $visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0$, $visited[s] = true$
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$
9. if $visited[v] == false$
10. $d[v] = d[u] + 1$, $visited[v] = true$
11. Enqueue(Q, v)

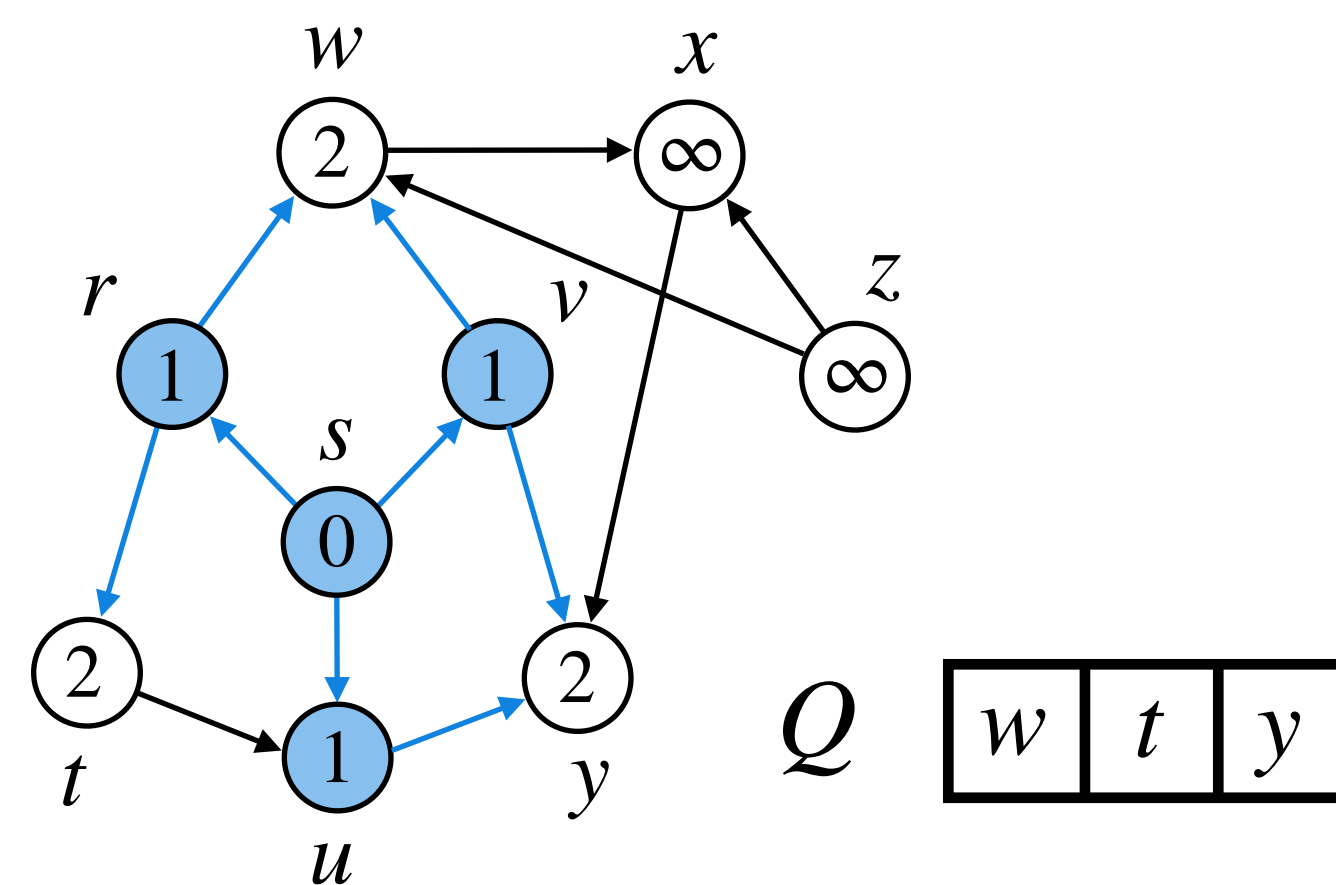
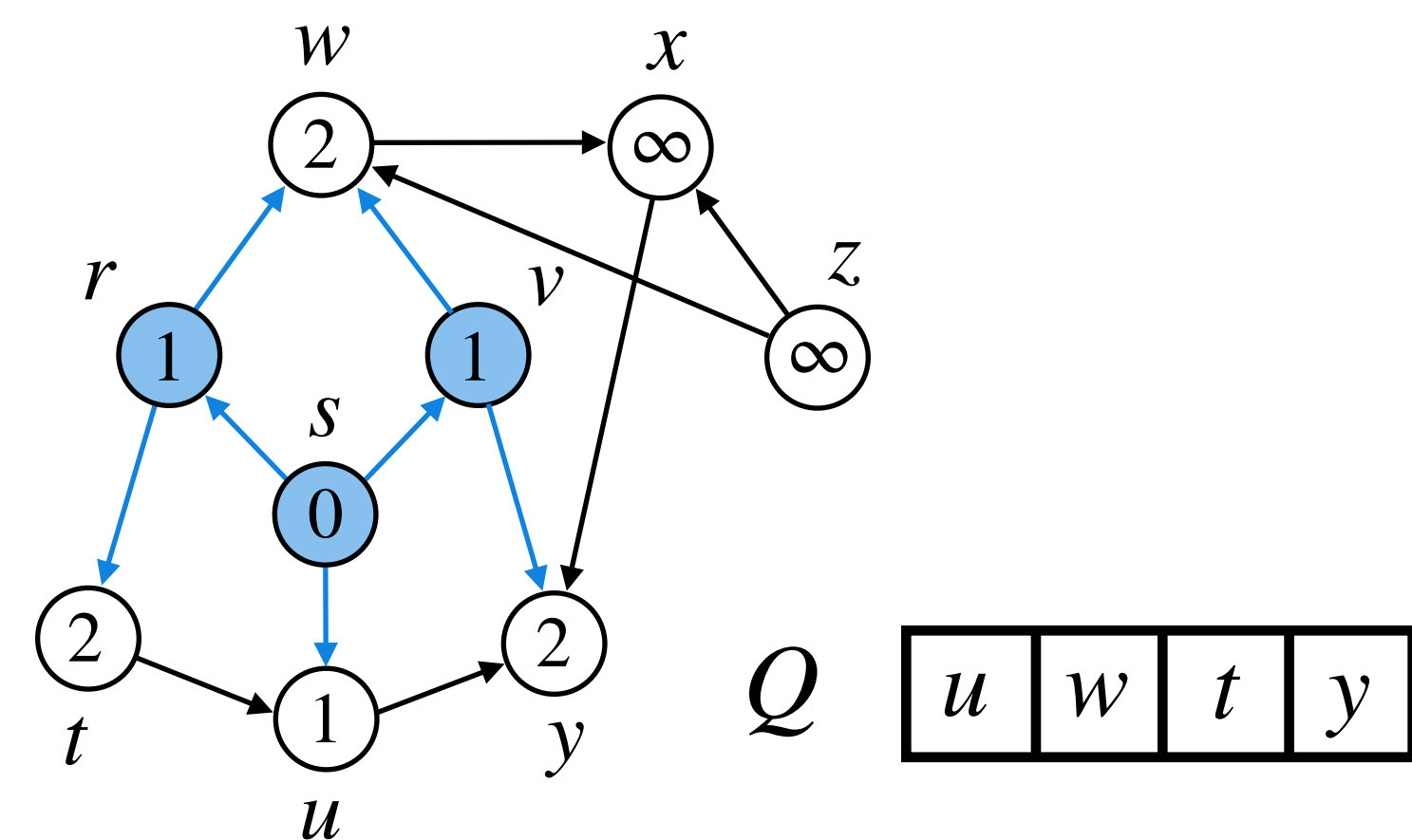


Breadth-First Search: Demonstration

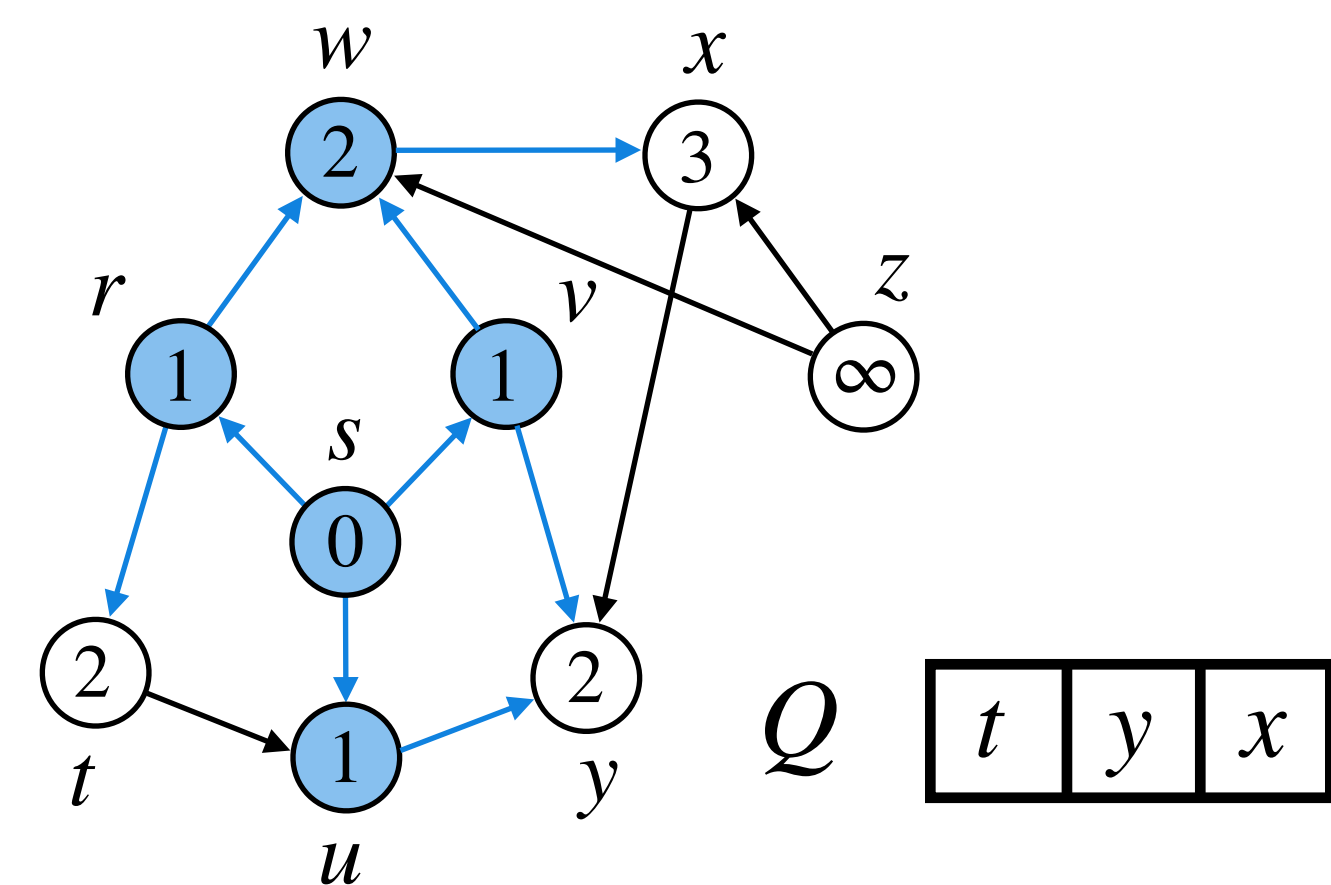
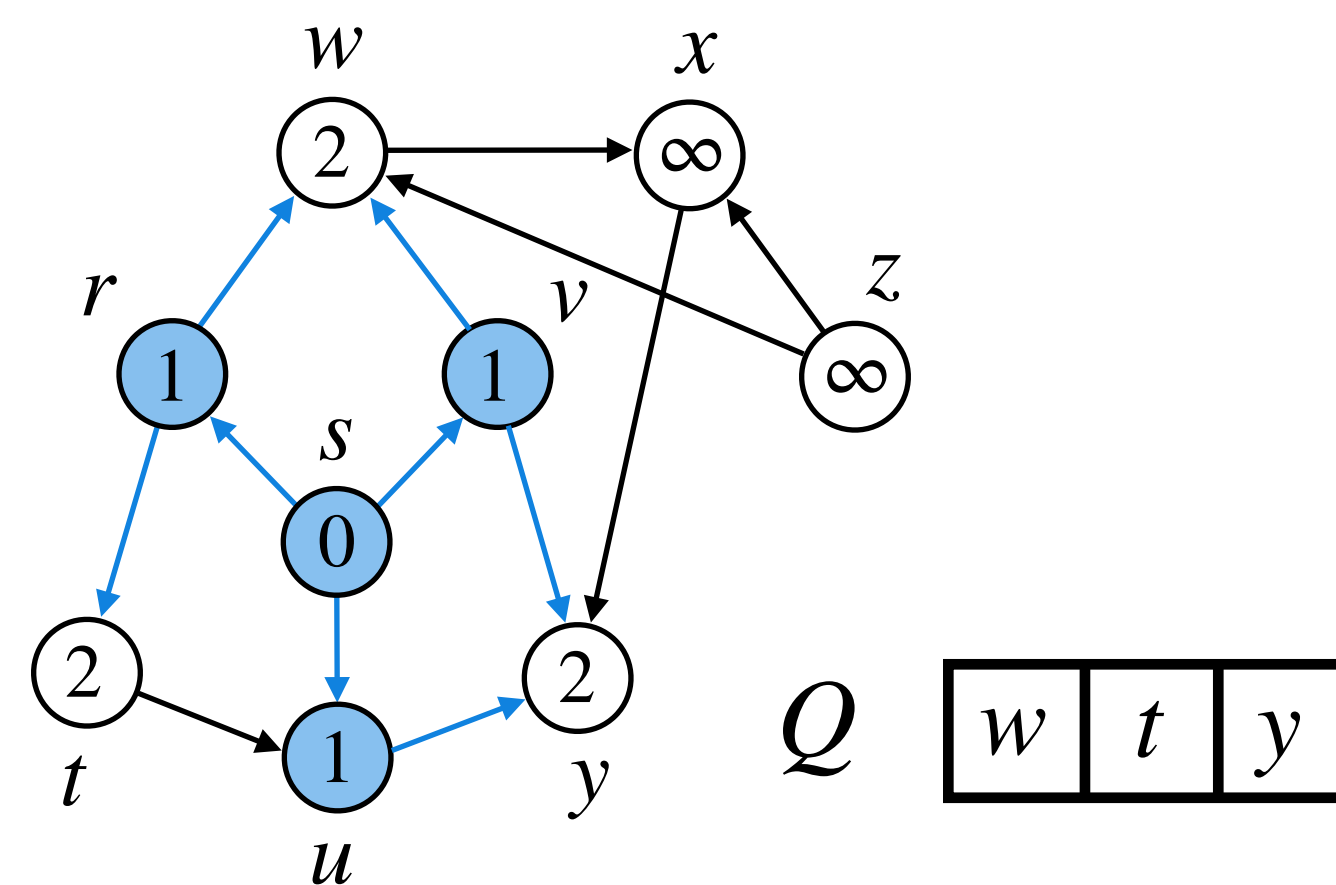
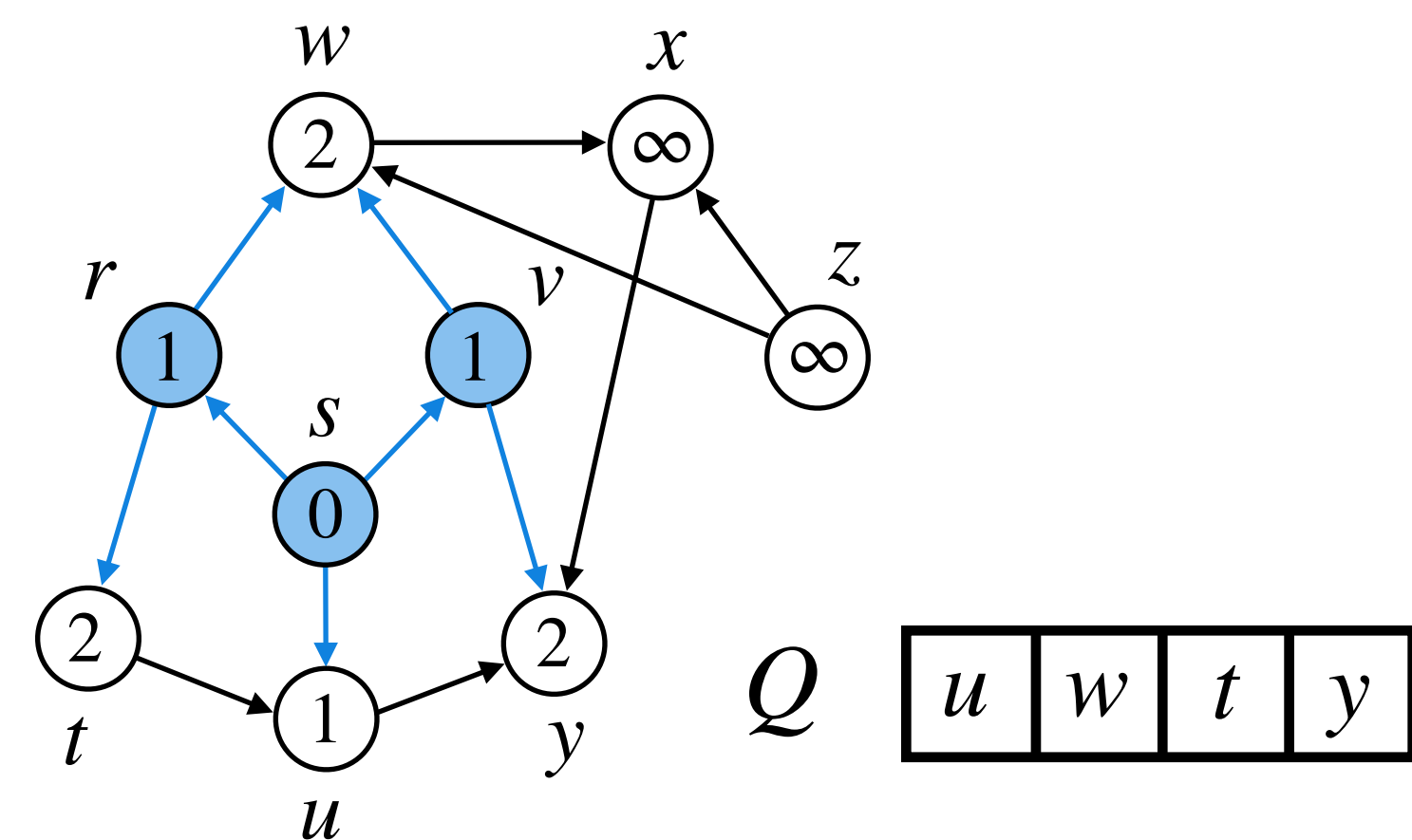
Breadth-First Search: Demonstration



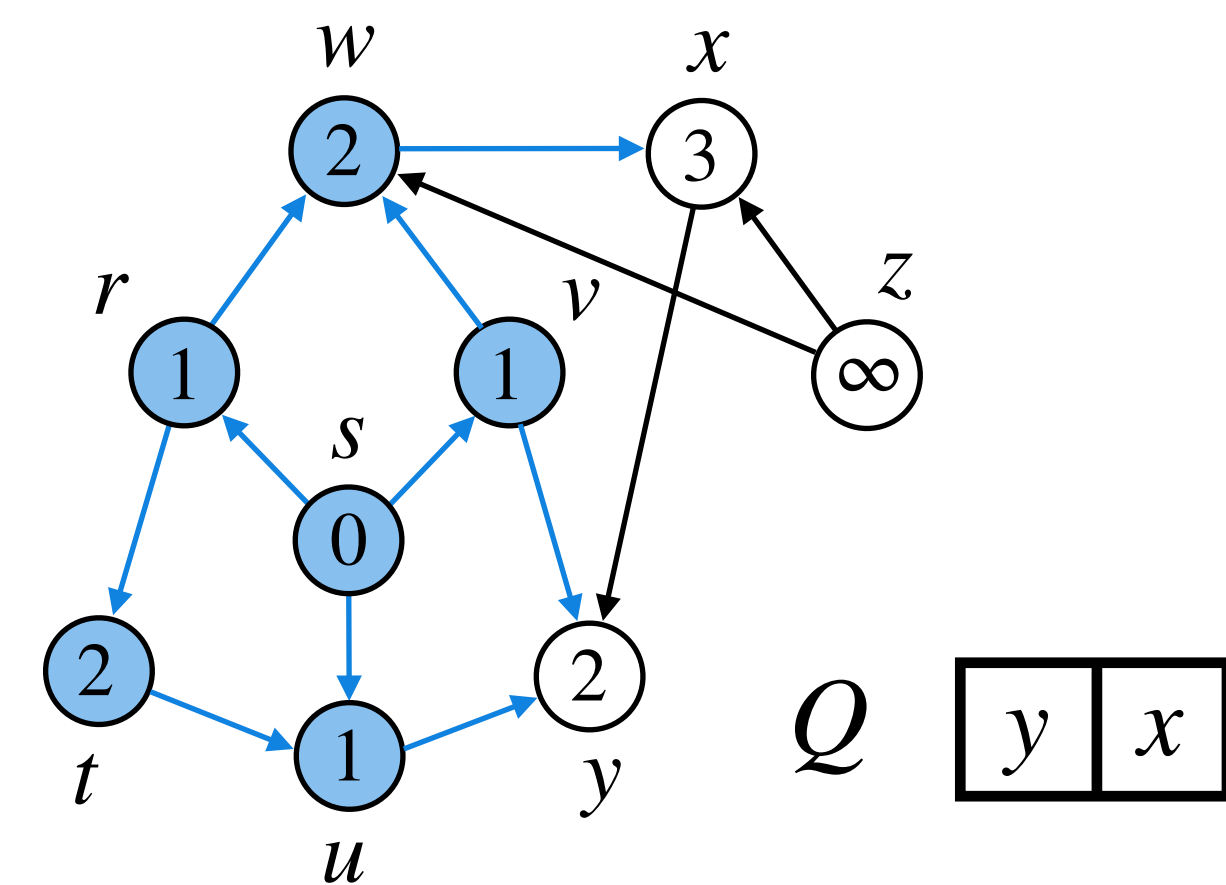
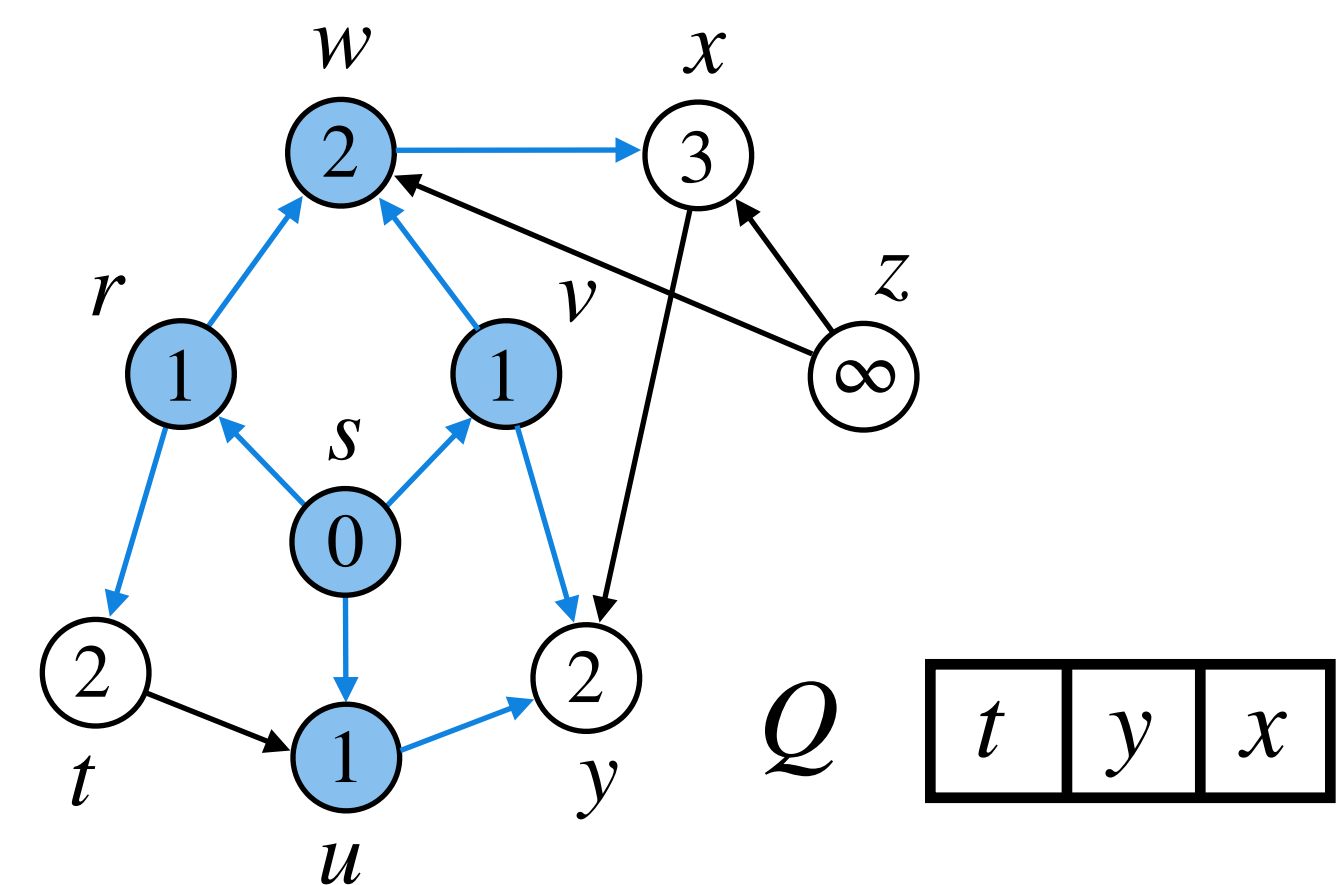
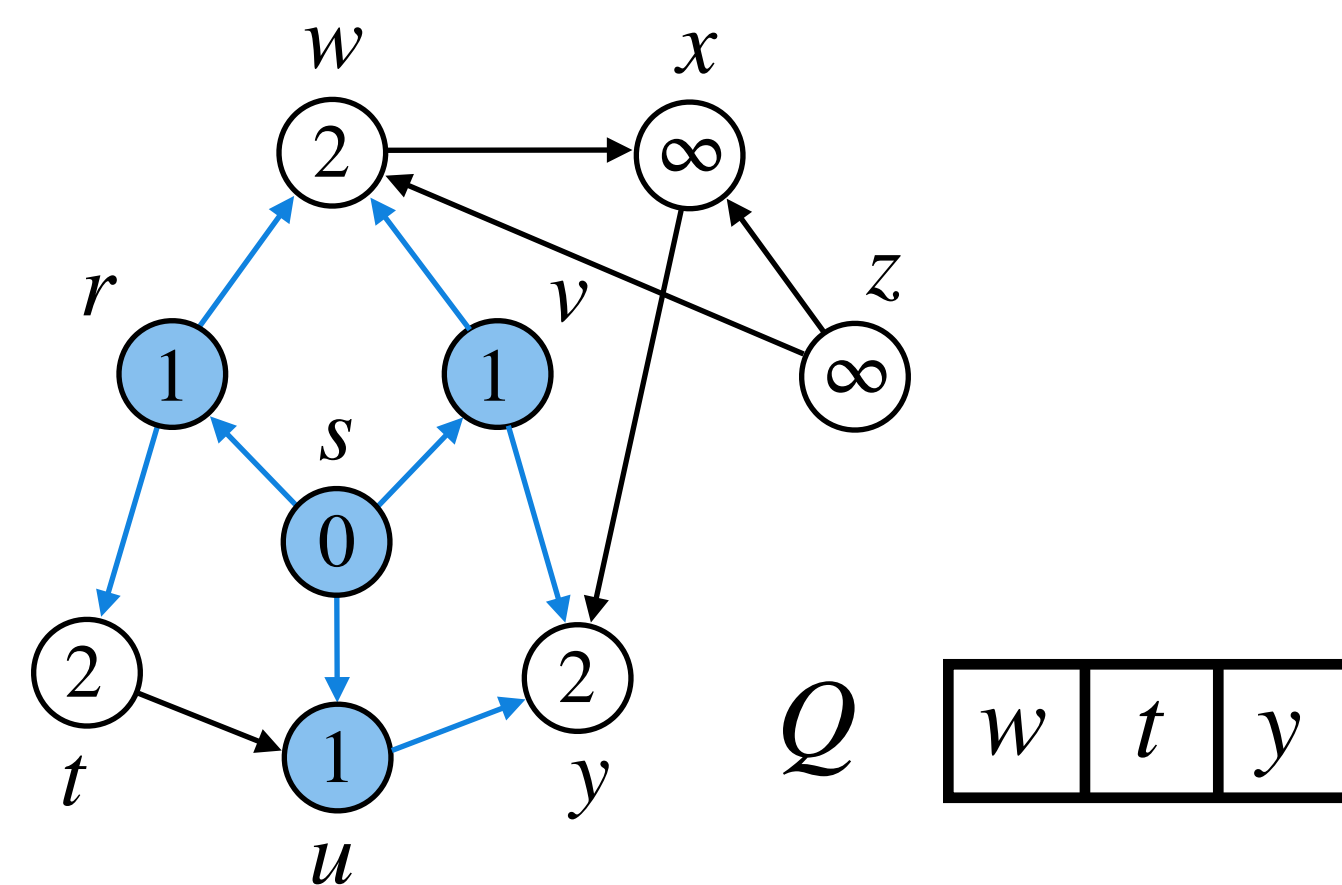
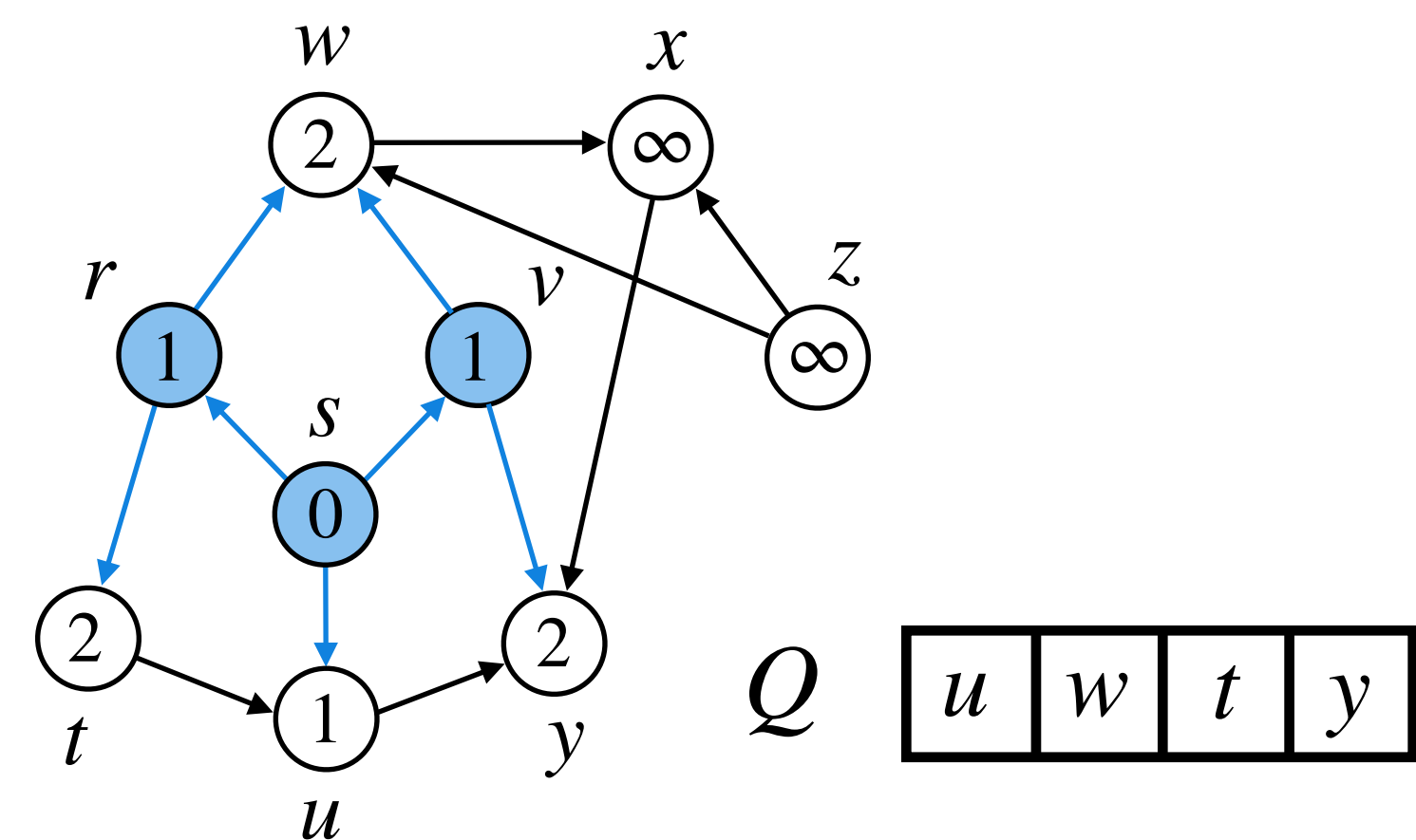
Breadth-First Search: Demonstration



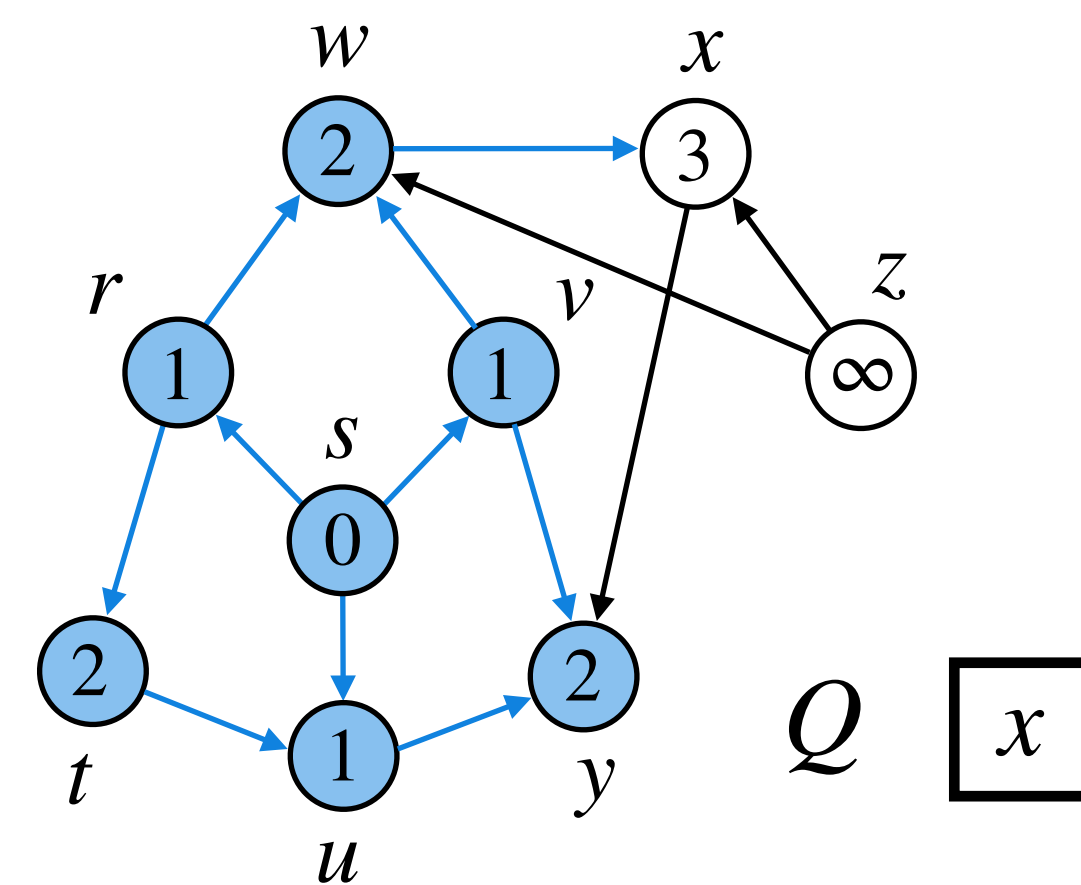
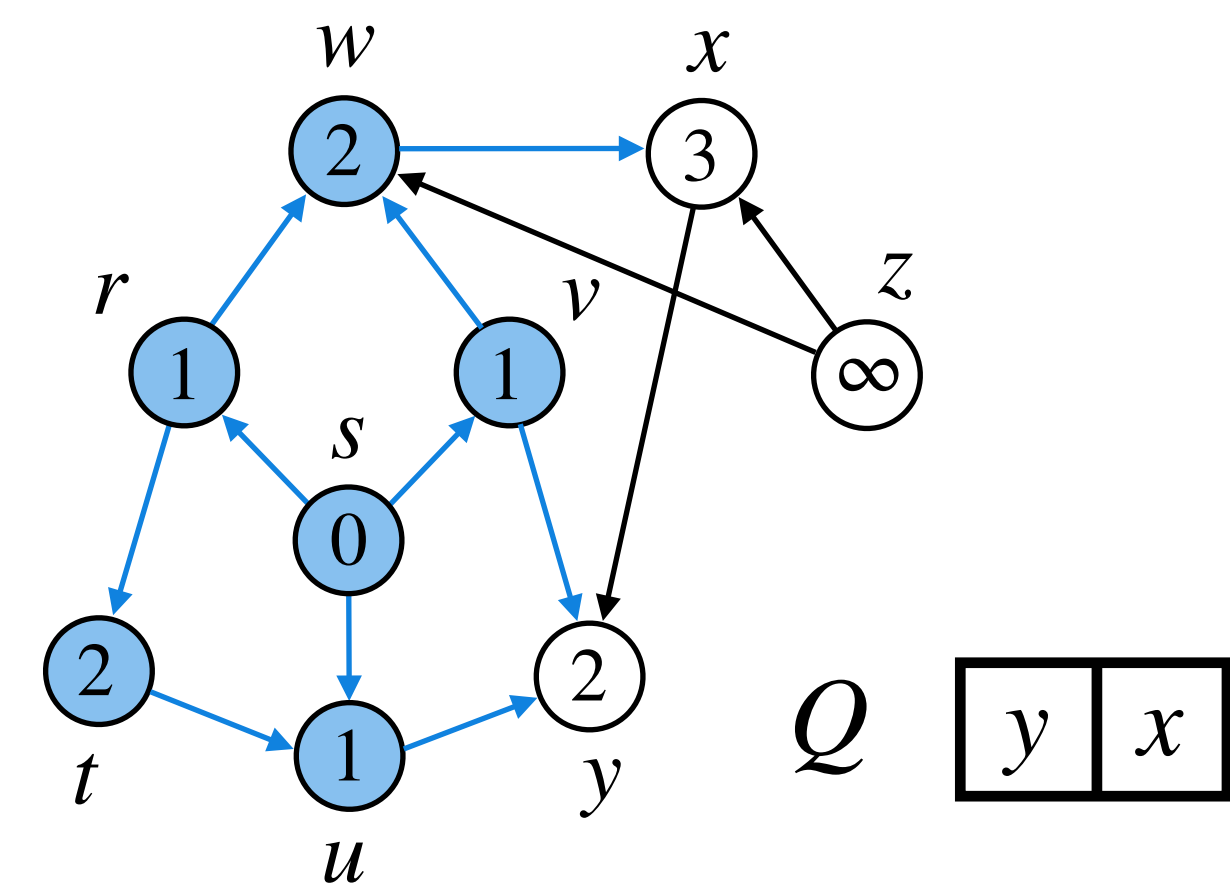
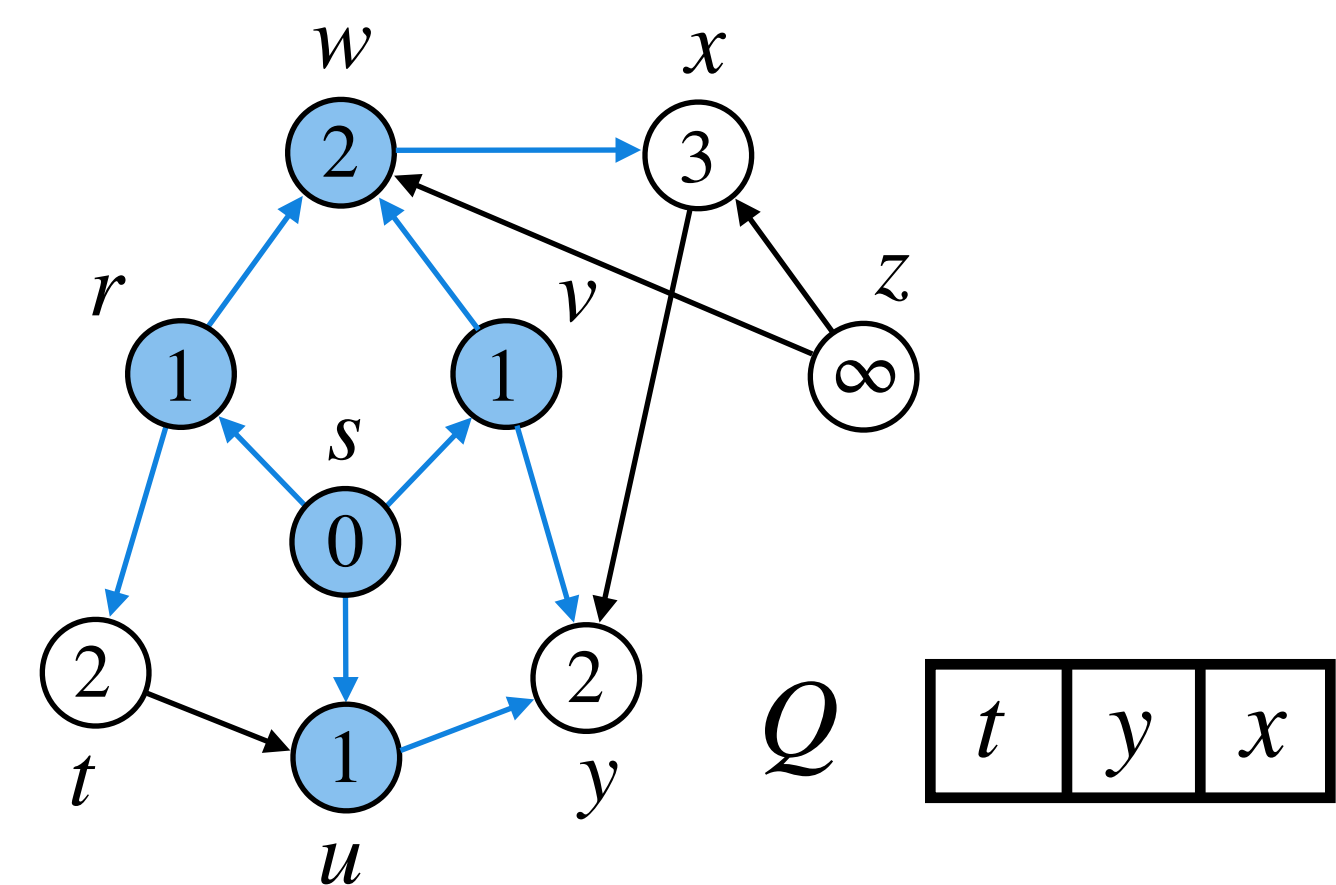
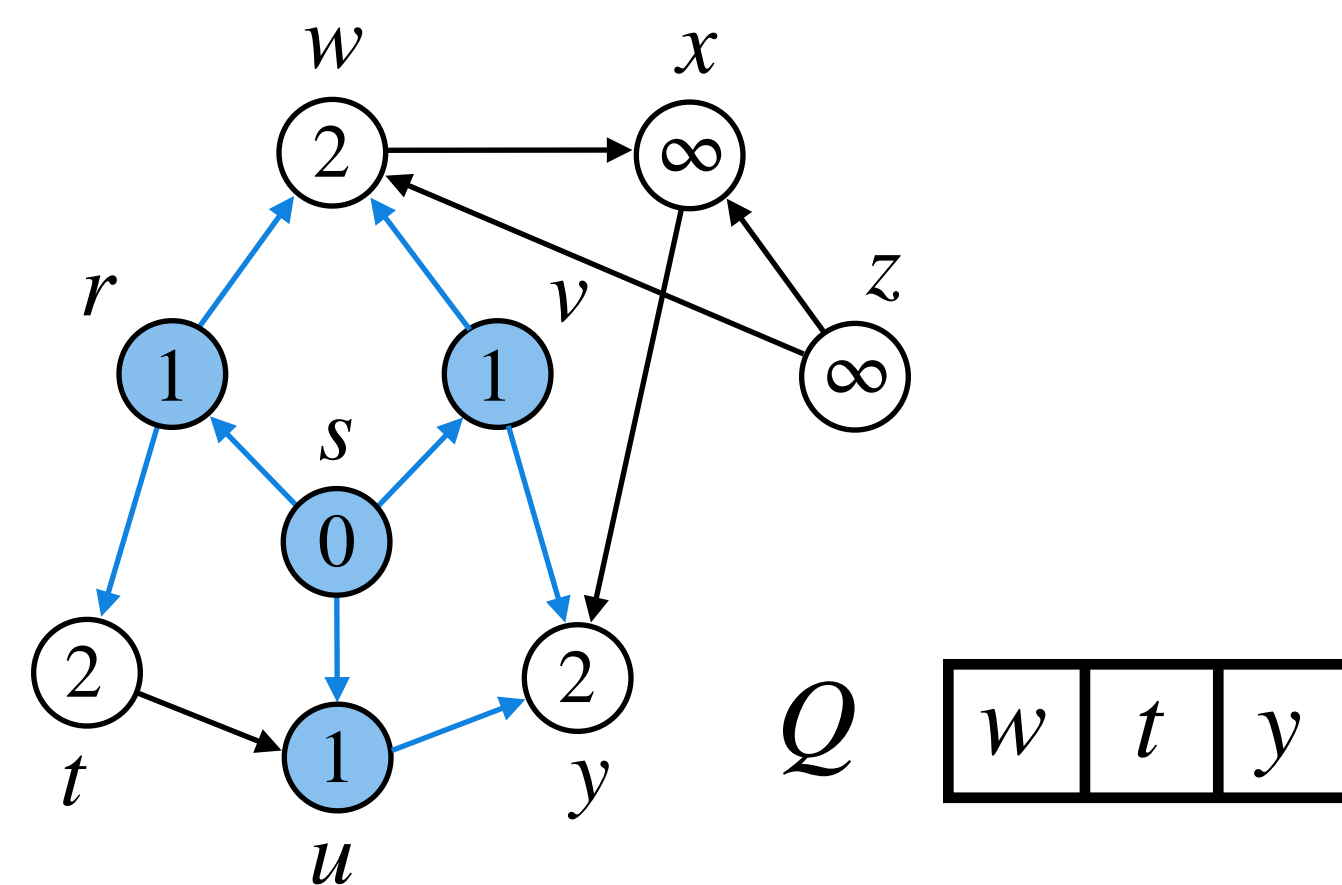
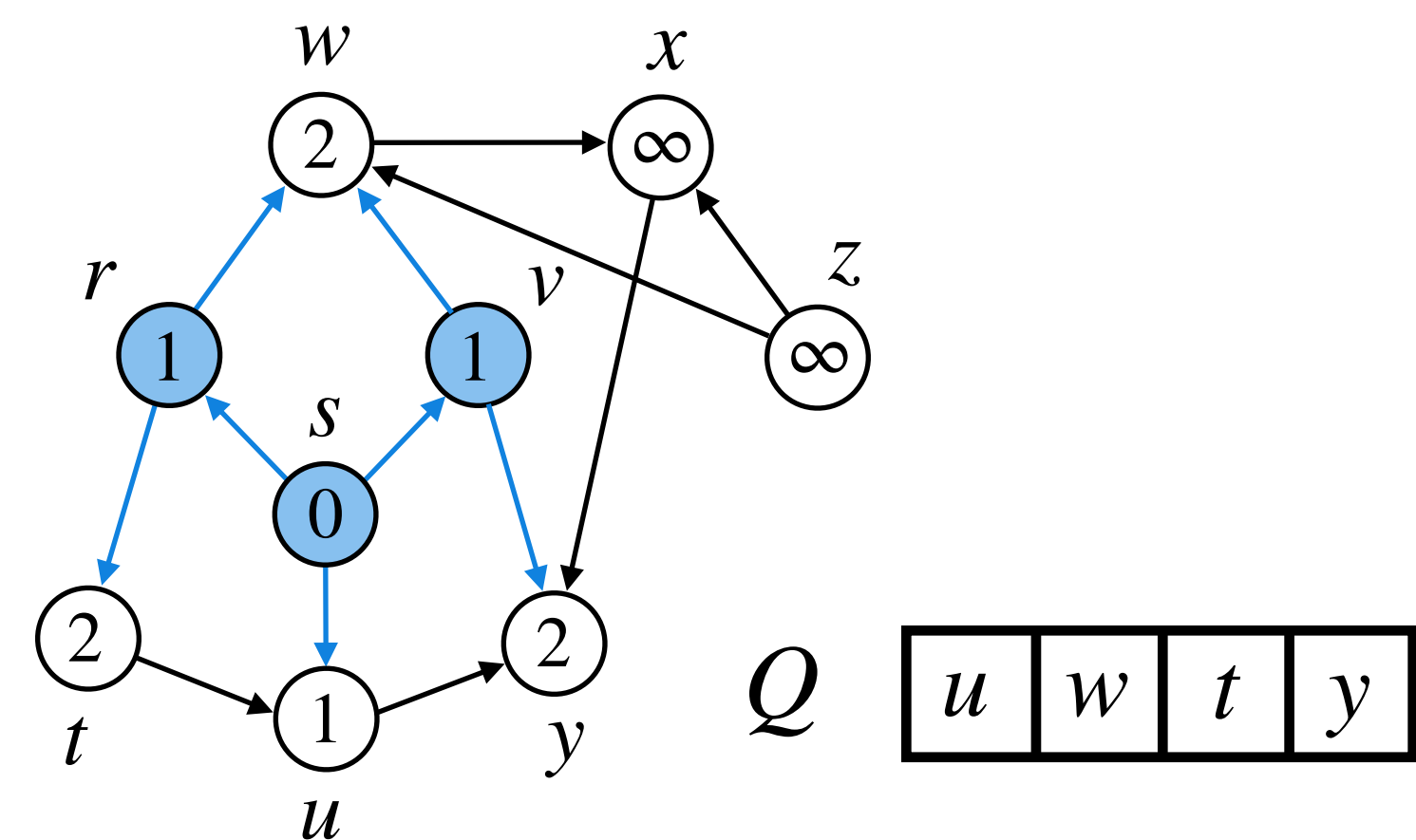
Breadth-First Search: Demonstration



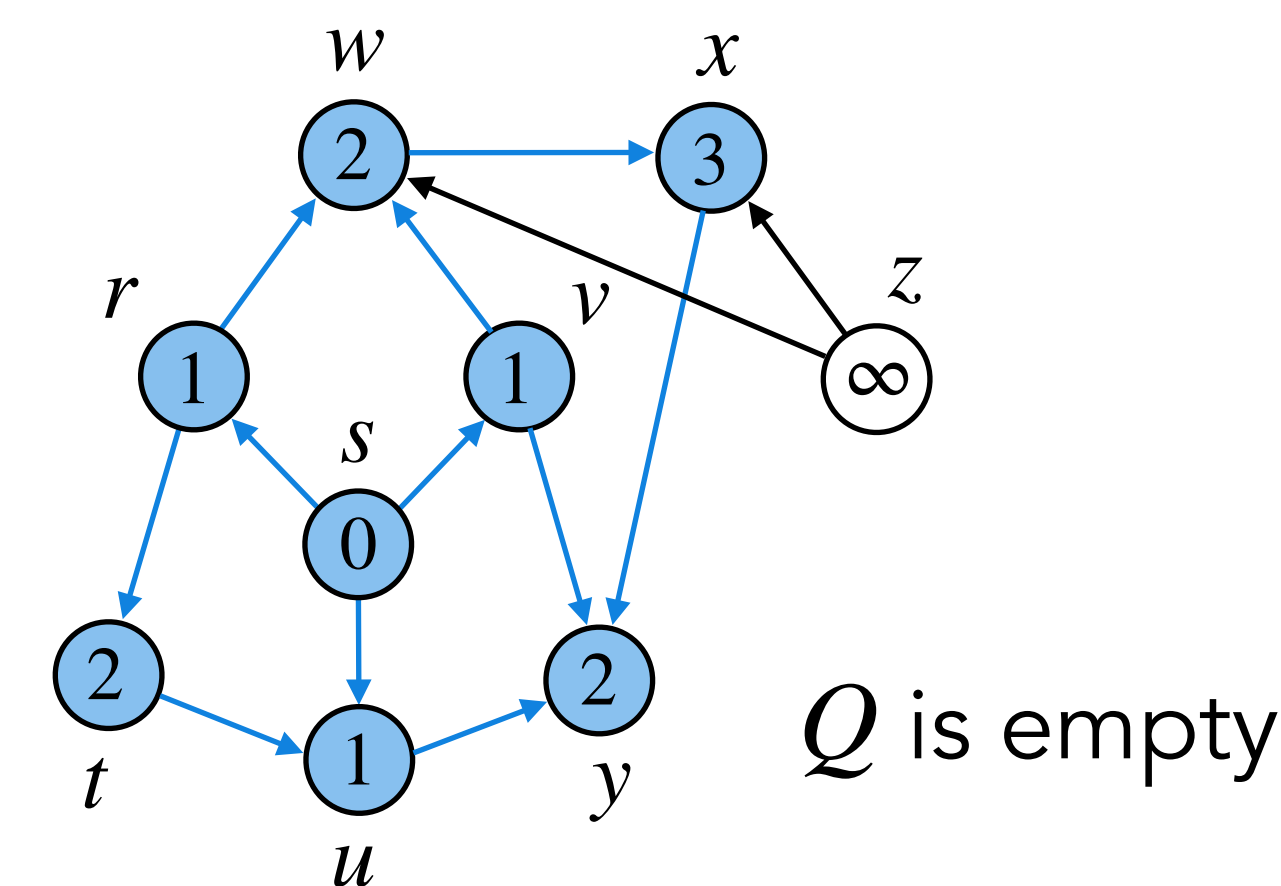
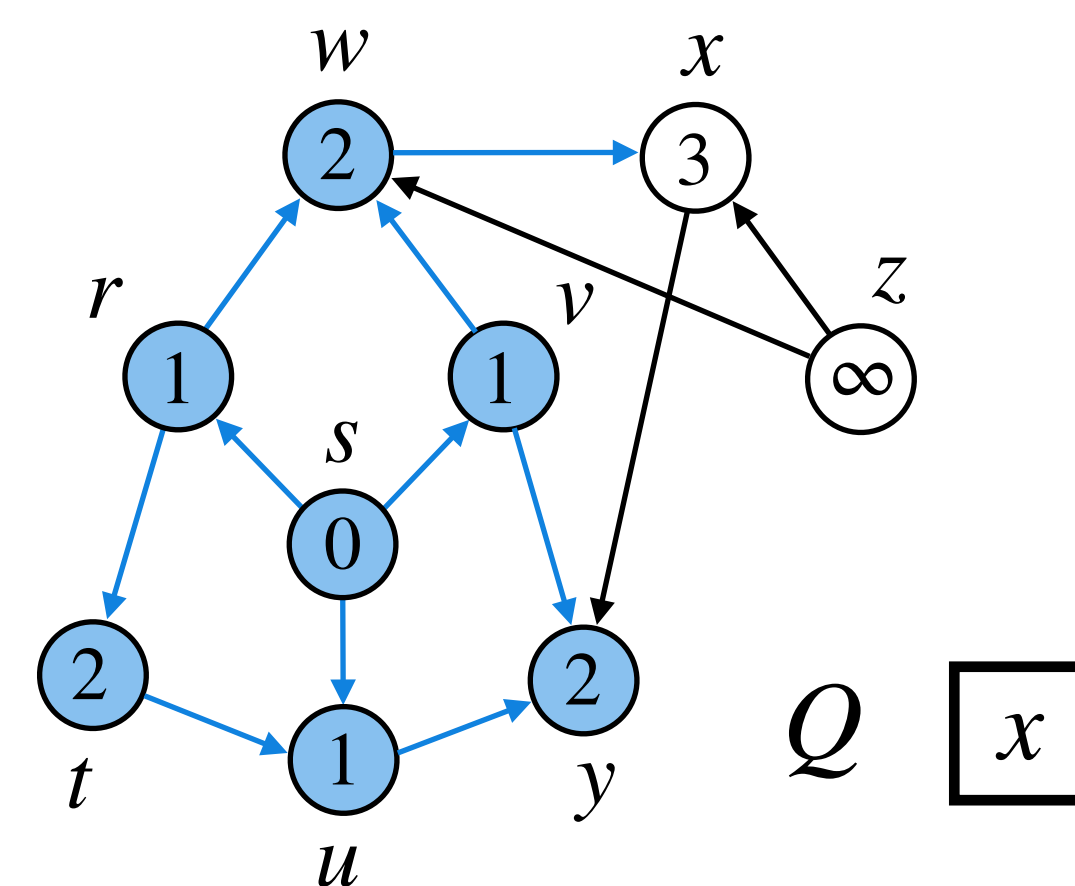
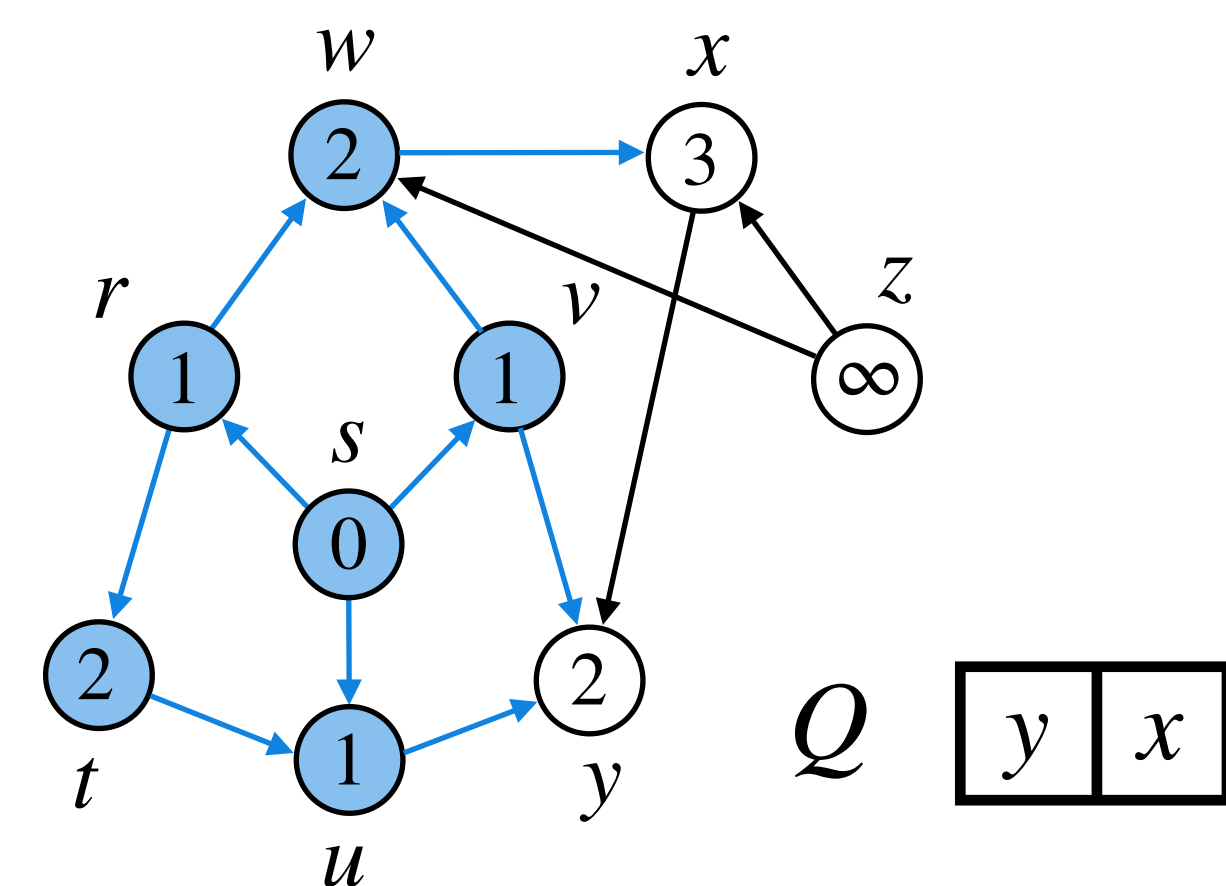
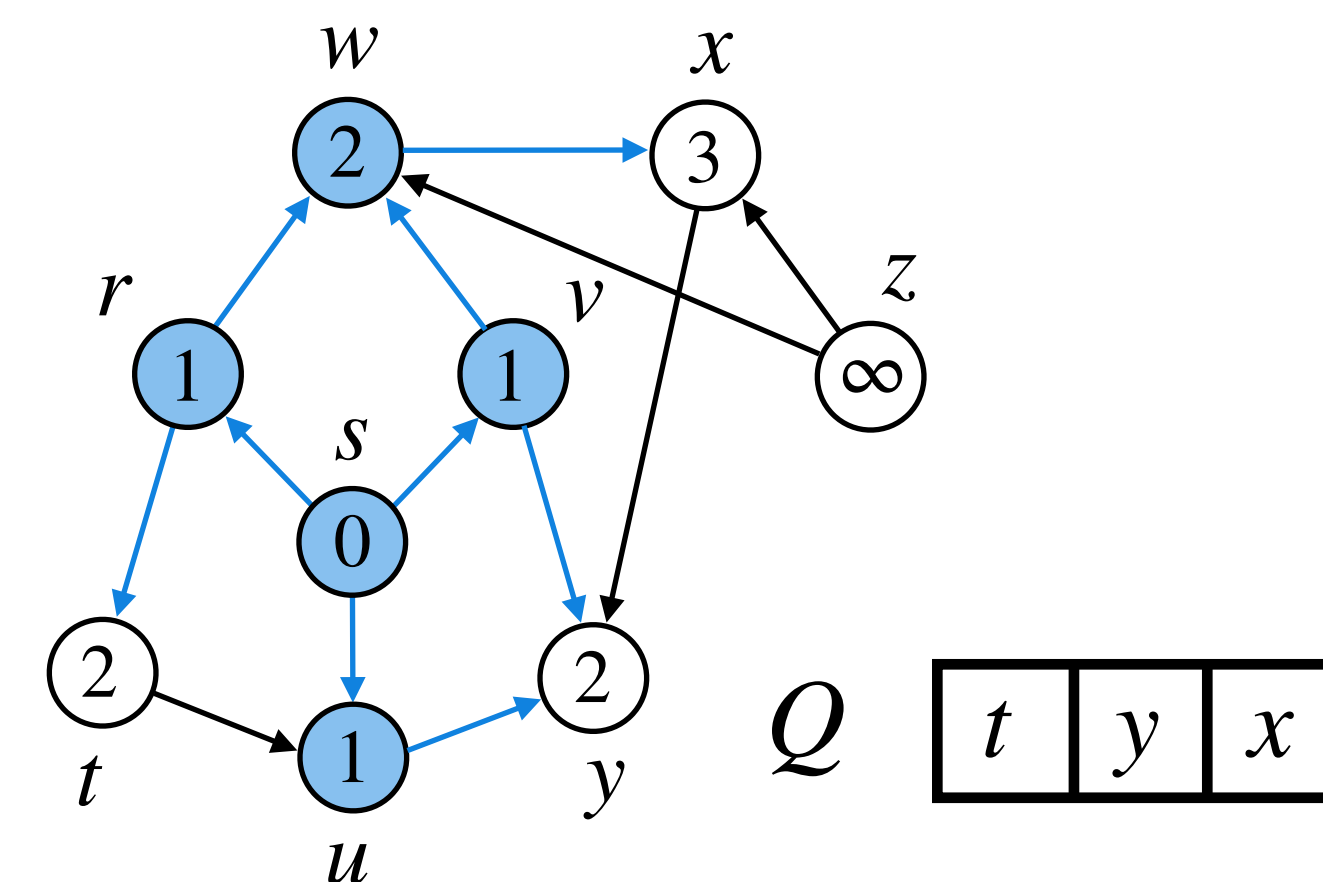
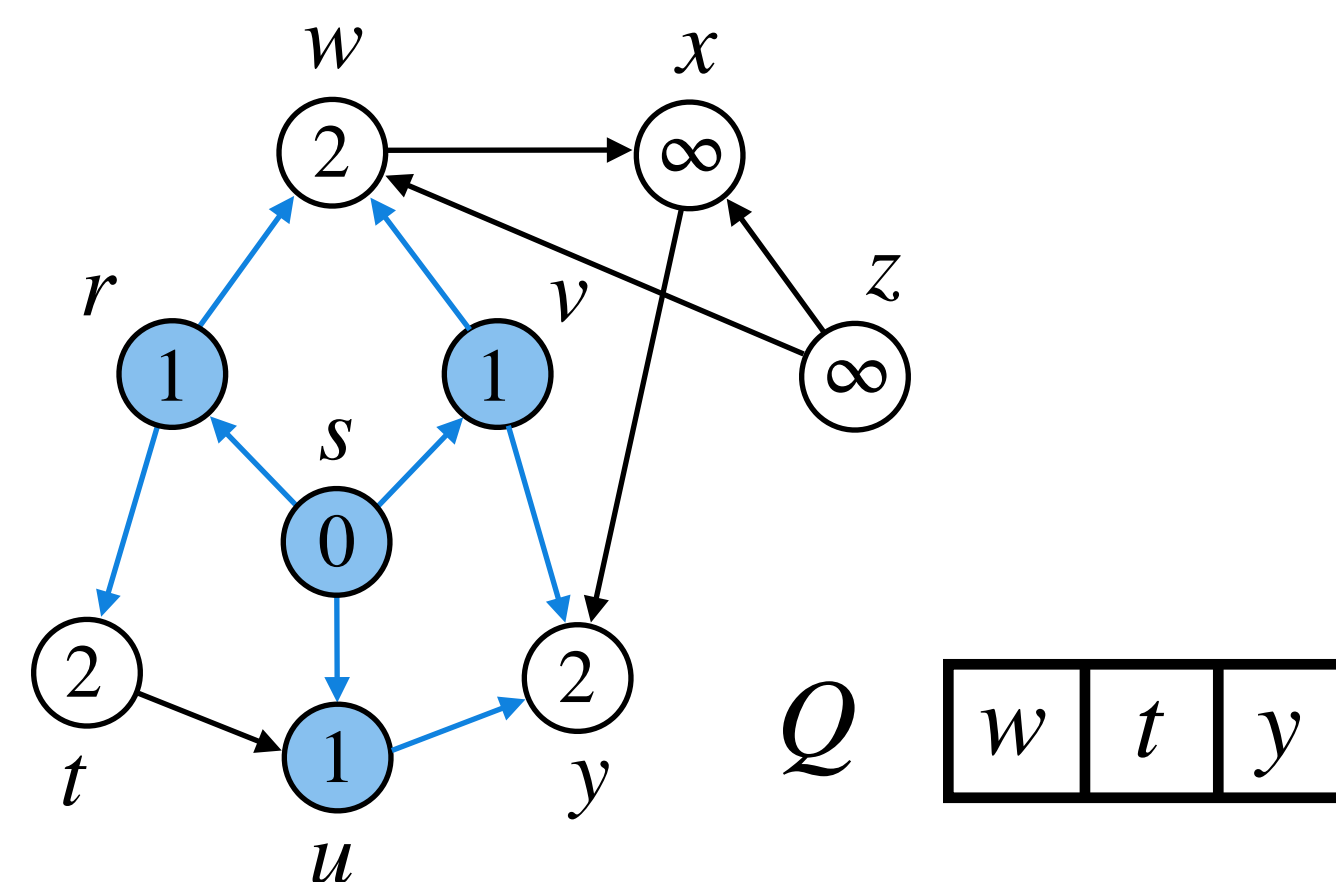
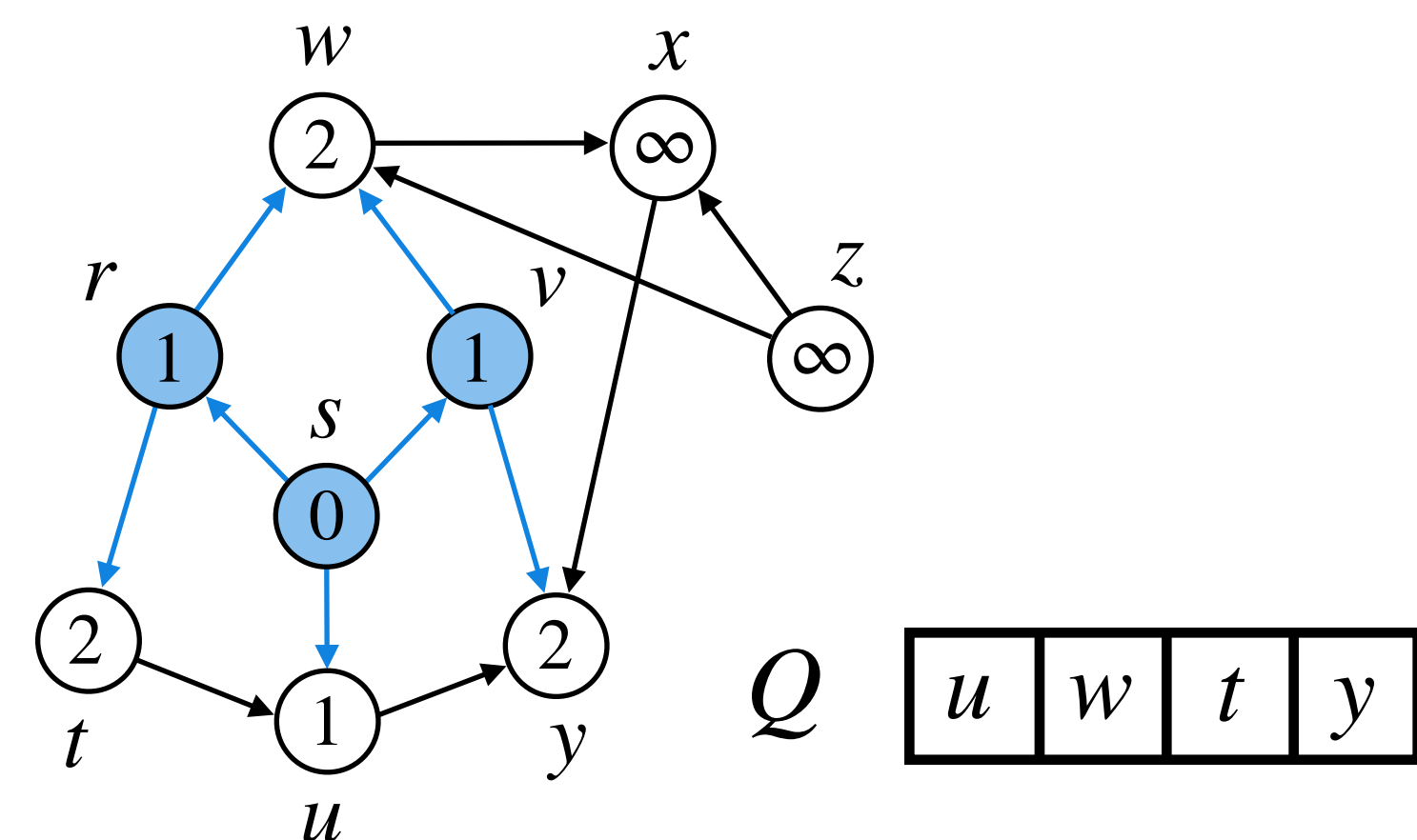
Breadth-First Search: Demonstration



Breadth-First Search: Demonstration



Breadth-First Search: Demonstration



Breadth-First Search: Analysis

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$
9. if $visited[v] == false$
10. $d[v] = d[u] + 1, visited[v] = true$
11. Enqueue(Q, v)

Breadth-First Search: Analysis

BFS(G, s):

1. **for every** $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$
5. Enqueue(Q, s)
6. **while** $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. **for every** v in $Adj[u]$
9. **if** $visited[v] == false$
10. $d[v] = d[u] + 1, visited[v] = true$
11. Enqueue(Q, v)

Suppose G has n vertices and m edges.

Breadth-First Search: Analysis

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$
9. if $visited[v] == false$
10. $d[v] = d[u] + 1, visited[v] = true$
11. Enqueue(Q, v)

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$



Breadth-First Search: Analysis

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$
9. if $visited[v] == false$
10. $d[v] = d[u] + 1, visited[v] = true$
11. Enqueue(Q, v)

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$

Cost of this loop is $O(n + m)$

Breadth-First Search: Analysis

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$
9. if $visited[v] == false$
10. $d[v] = d[u] + 1, visited[v] = true$
11. Enqueue(Q, v)

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$

Cost of this loop is $O(n + m)$

(Every vertex is enqueued at most once, and

Breadth-First Search: Analysis

BFS(G, s):

1. **for every** $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$
5. **Enqueue**(Q, s)
6. **while** $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. **for every** v in $Adj[u]$
9. **if** $visited[v] == false$
10. $d[v] = d[u] + 1, visited[v] = true$
11. **Enqueue**(Q, v)

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$

Cost of this loop is $O(n + m)$

(Every vertex is enqueued at most once, and when dequeued its adjacency list is traversed.)

Breadth-First Search: Analysis

BFS(G, s):

1. for every $u \in V(G) - \{s\}$
2. $d[u] = \infty, visited[u] = false$
3. $Q = \emptyset$
4. $d[s] = 0, visited[s] = true$
5. Enqueue(Q, s)
6. while $Q \neq \emptyset$
7. $u = \text{Dequeue}(Q)$
8. for every v in $Adj[u]$
9. if $visited[v] == false$
10. $d[v] = d[u] + 1, visited[v] = true$
11. Enqueue(Q, v)

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$

Cost of this loop is $O(n + m)$

(Every vertex is enqueued at most once, and when dequeued its adjacency list is traversed.)

Time complexity: $= O(n + m)$

Breadth-First Search: Correctness and More

Breadth-First Search: Correctness and More

Let $\delta(s, v)$ denote the length of a **shortest path** from s to v .

Breadth-First Search: Correctness and More

Let $\delta(s, v)$ denote the length of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

Breadth-First Search: Correctness and More

Let $\delta(s, v)$ denote the length of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

Theorem: Let $G = (V, E)$ be a directed or undirected graph,

Breadth-First Search: Correctness and More

Let $\delta(s, v)$ denote the length of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

Theorem: Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on

Breadth-First Search: Correctness and More

Let $\delta(s, v)$ denote the length of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

Theorem: Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$.

Breadth-First Search: Correctness and More

Let $\delta(s, v)$ denote the length of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

Theorem: Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then, when the BFS is finished, $d[v] = \delta(s, v)$, for all $v \in V$.

Breadth-First Search: Correctness and More

Let $\delta(s, v)$ denote the length of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

Theorem: Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then, when the BFS is finished, $d[v] = \delta(s, v)$, for all $v \in V$.

Proof: DIY.

Breadth-First Search: Correctness and More

Let $\delta(s, v)$ denote the length of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

Theorem: Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then, when the BFS is finished, $d[v] = \delta(s, v)$, for all $v \in V$.

Proof: DIY.

H.W: Modify the **BFS** procedure so that we can also produce the shortest path.

Weighted Graphs

Weighted Graphs

Defn: A **weighted graph** $G = (V, E)$ is graph with a weight function $w : E \rightarrow \mathbb{R}$ mapping edges

Weighted Graphs

Defn: A **weighted graph** $G = (V, E)$ is graph with a weight function $w : E \rightarrow \mathbb{R}$ mapping edges to **real-valued weights**.

Weighted Graphs

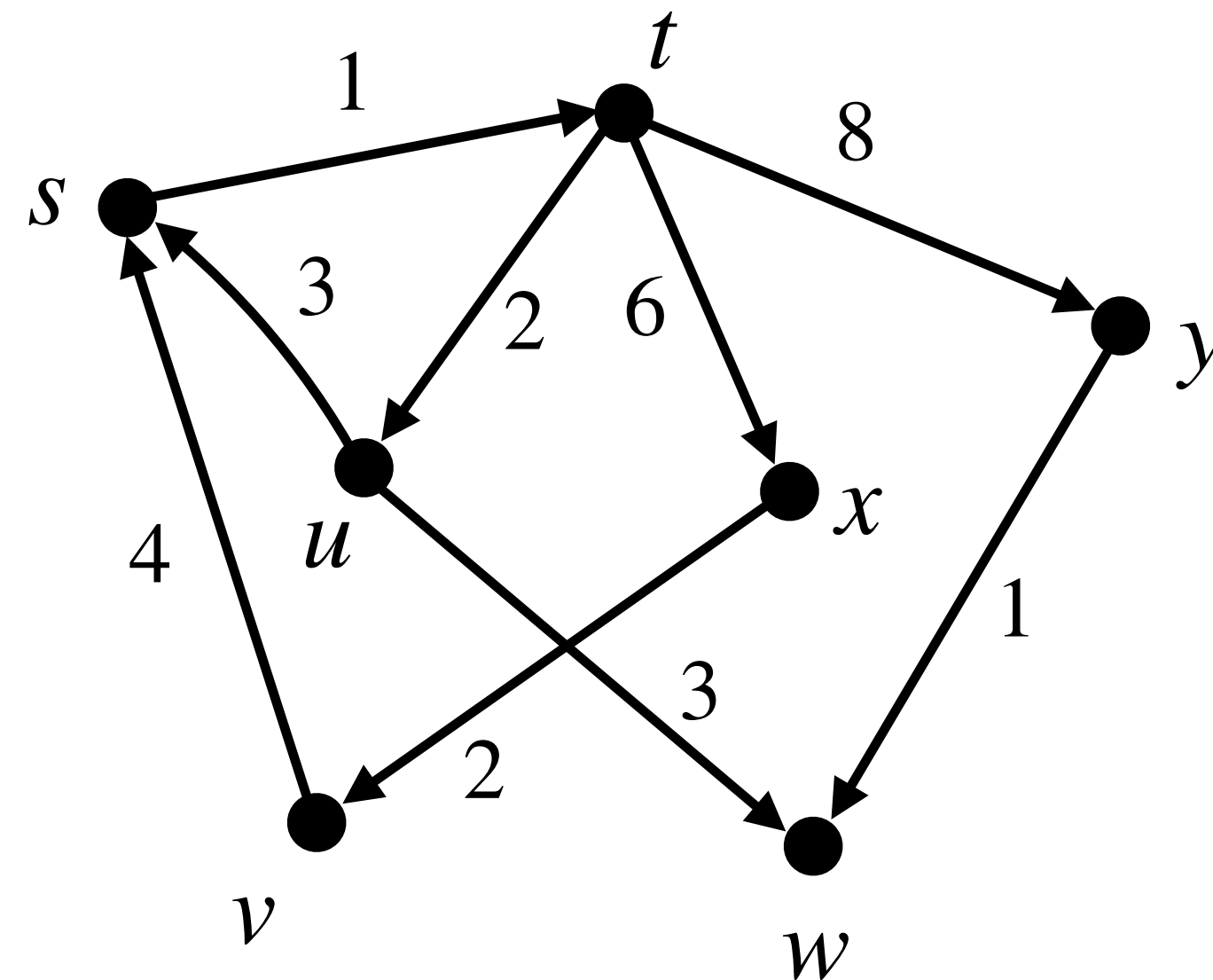
Defn: A **weighted graph** $G = (V, E)$ is graph with a weight function $w : E \rightarrow \mathbb{R}$ mapping edges to **real-valued weights**.

Example:

Weighted Graphs

Defn: A **weighted graph** $G = (V, E)$ is graph with a weight function $w : E \rightarrow \mathbb{R}$ mapping edges to **real-valued weights**.

Example:



Weighted Graphs

Weighted Graphs

Defn: **Weight of a path** is the **sum** of the weights of the edges in that path.

Weighted Graphs

Defn: *Weight of a path* is the *sum* of the weights of the edges in that path. In a *weighted graph*,

Weighted Graphs

Defn: **Weight of a path** is the **sum** of the weights of the edges in that path. In a **weighted graph**, a **shortest-path** between two vertices is the path with the **least weight**.

Weighted Graphs

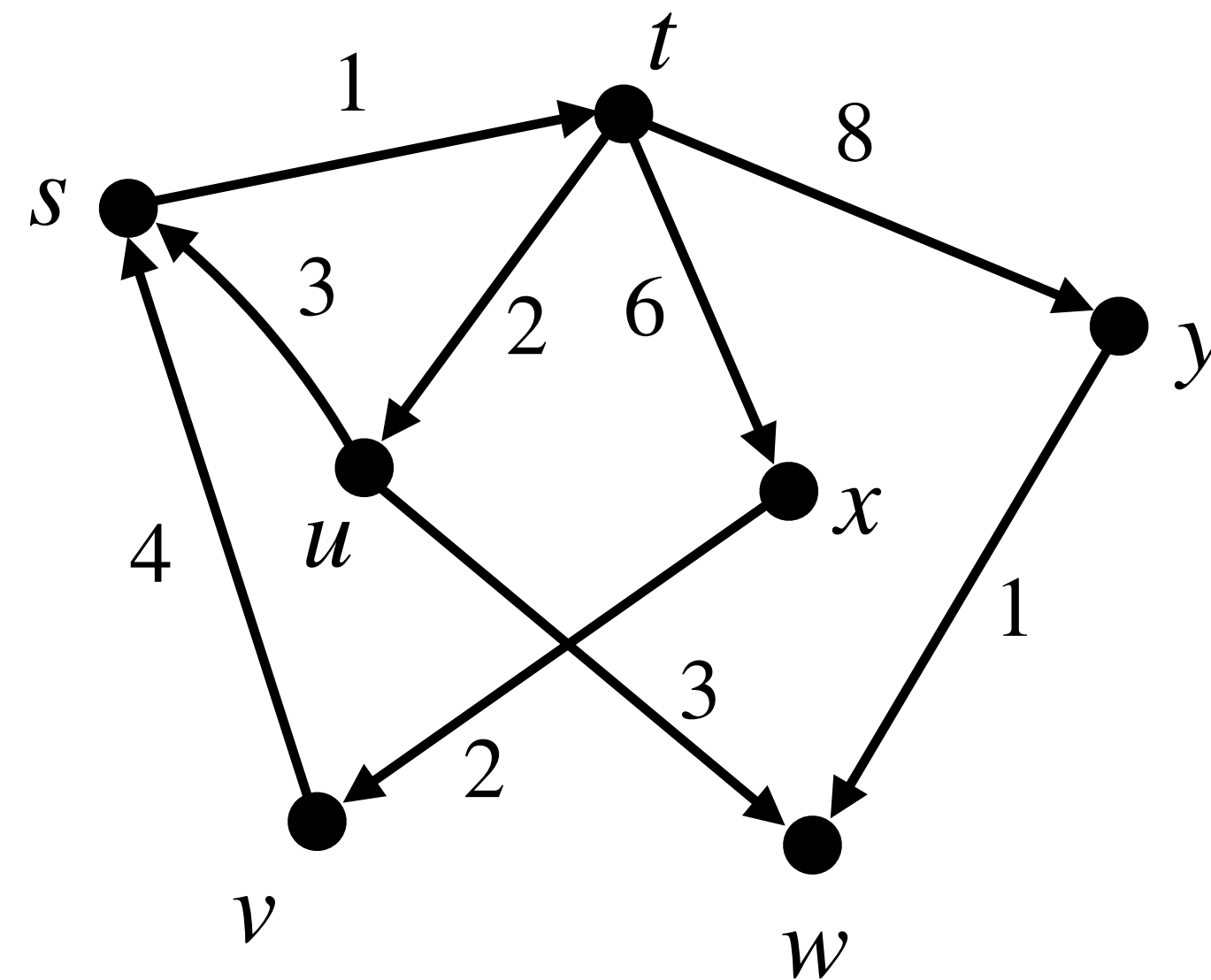
Defn: **Weight of a path** is the **sum** of the weights of the edges in that path. In a **weighted graph**, a **shortest-path** between two vertices is the path with the **least weight**.

Example: In the below graph shortest path from *s* to *w* is of weight 6.

Weighted Graphs

Defn: **Weight of a path** is the **sum** of the weights of the edges in that path. In a **weighted graph**, a **shortest-path** between two vertices is the path with the **least weight**.

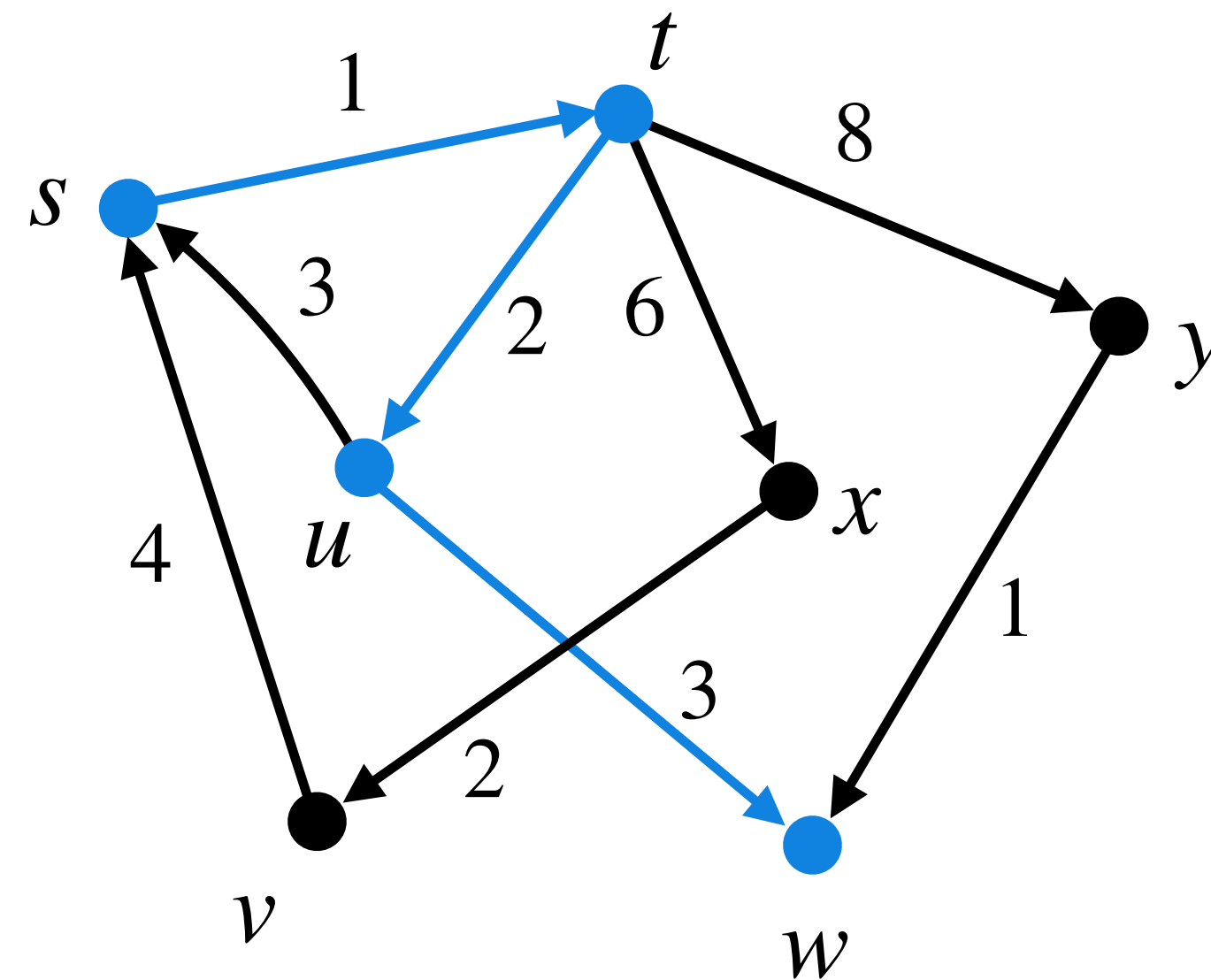
Example: In the below graph shortest path from ***s*** to ***w*** is of weight 6.



Weighted Graphs

Defn: **Weight of a path** is the **sum** of the weights of the edges in that path. In a **weighted graph**, a **shortest-path** between two vertices is the path with the **least weight**.

Example: In the below graph shortest path from ***s*** to ***w*** is of weight 6.



Single-Source Shortest Path Problem

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Input: A weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$ and a vertex s .

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Input: A weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$ and a vertex s .

Output: Compute distance from s to every other vertex. That is, an array $d[1 : |V|]$,

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Input: A weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$ and a vertex s .

Output: Compute distance from s to every other vertex. That is, an array $d[1 : |V|]$,
such that $d[i] = \delta(s, i)$.

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Input: A weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$ and a vertex s .

Output: Compute distance from s to every other vertex. That is, an array $d[1 : |V|]$,
such that $d[i] = \delta(s, i)$.

- **Dijkstra's** algorithm solves *SSSP* in graphs with **non-negative weights**.

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Input: A weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$ and a vertex s .

Output: Compute distance from s to every other vertex. That is, an array $d[1 : |V|]$,
such that $d[i] = \delta(s, i)$.

- **Dijkstra's** algorithm solves *SSSP* in graphs with **non-negative weights**.
- **Bellman-Ford's** algorithm solves *SSSP* in graphs with **real weights**.

Dijkstra's Algorithm: Sketch

Dijkstra's Algorithm: Sketch

Maintain a set of explored vertices S for which algorithm has found $d[u] = \delta(s, u)$:

Dijkstra's Algorithm: Sketch

Maintain a set of explored vertices S for which algorithm has found $d[u] = \delta(s, u)$:

Step 1: Initialise $S = \{s\}$, $d[s] = 0$.

Dijkstra's Algorithm: Sketch

Maintain a set of explored vertices S for which algorithm has found $d[u] = \delta(s, u)$:

Step 1: Initialise $S = \{s\}$, $d[s] = 0$.

Step 2: Choose an unexplored vertex v from $V(G) \setminus S$ which minimizes:

Dijkstra's Algorithm: Sketch

Maintain a set of explored vertices S for which algorithm has found $d[u] = \delta(s, u)$:

Step 1: Initialise $S = \{s\}$, $d[s] = 0$.

Step 2: Choose an unexplored vertex v from $V(G) \setminus S$ which minimizes:

$$\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$$

Dijkstra's Algorithm: Sketch

Maintain a set of explored vertices S for which algorithm has found $d[u] = \delta(s, u)$:

Step 1: Initialise $S = \{s\}$, $d[s] = 0$.

Step 2: Choose an unexplored vertex v from $V(G) \setminus S$ which minimizes:

$$\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$$

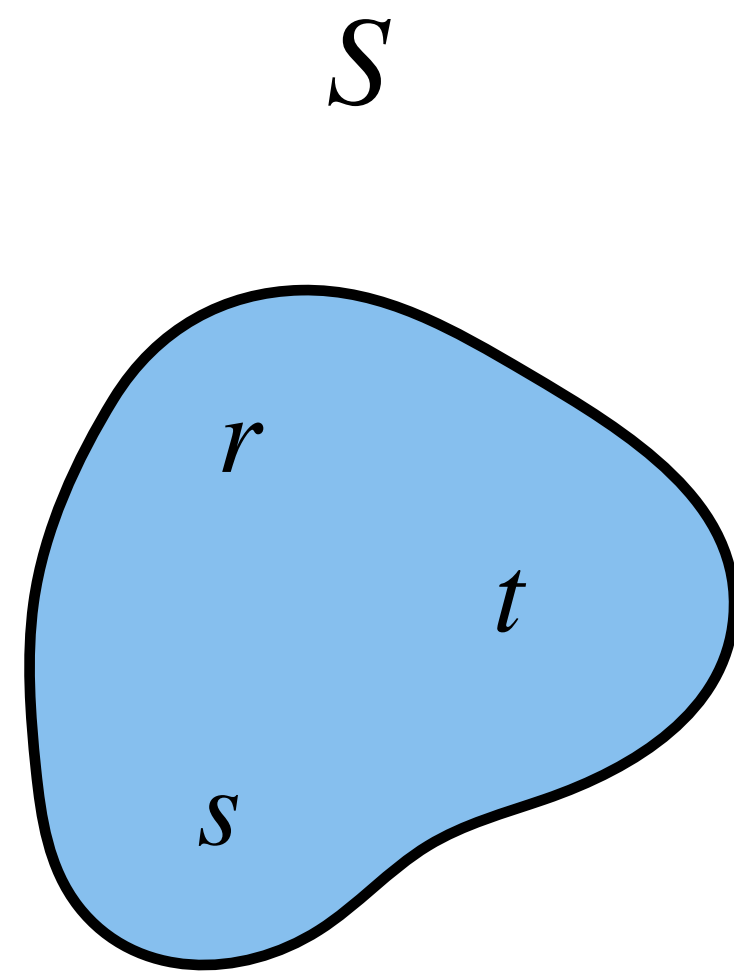
Add v to S and set $d[v] = \pi[v]$.

Dijkstra's Algorithm: Sketch

Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$

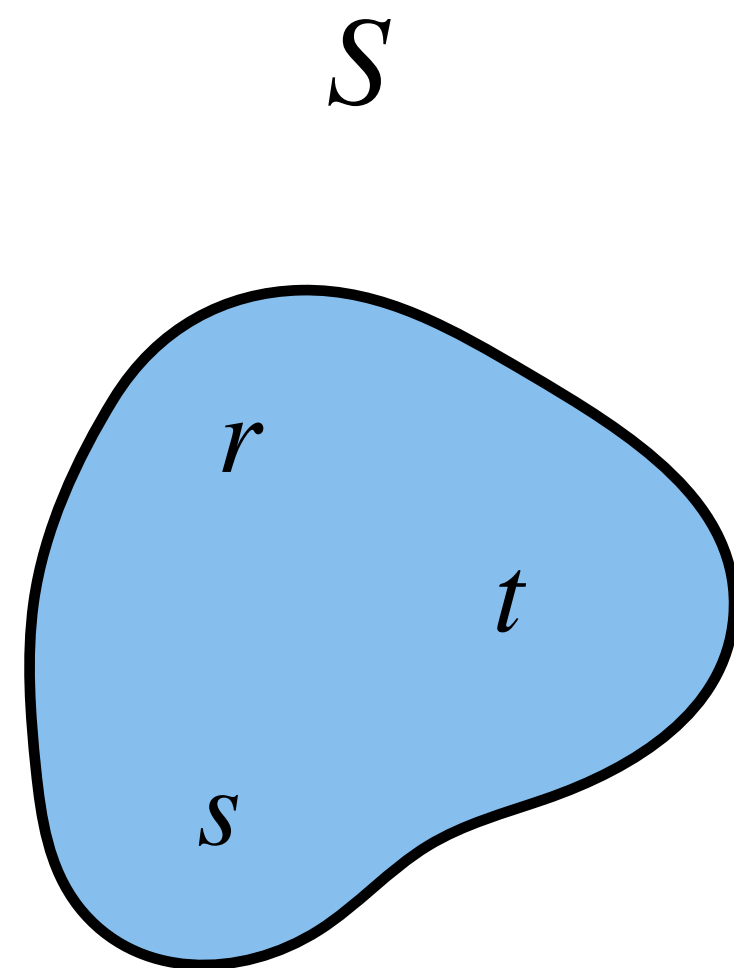
Dijkstra's Algorithm: Sketch

Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$



Dijkstra's Algorithm: Sketch

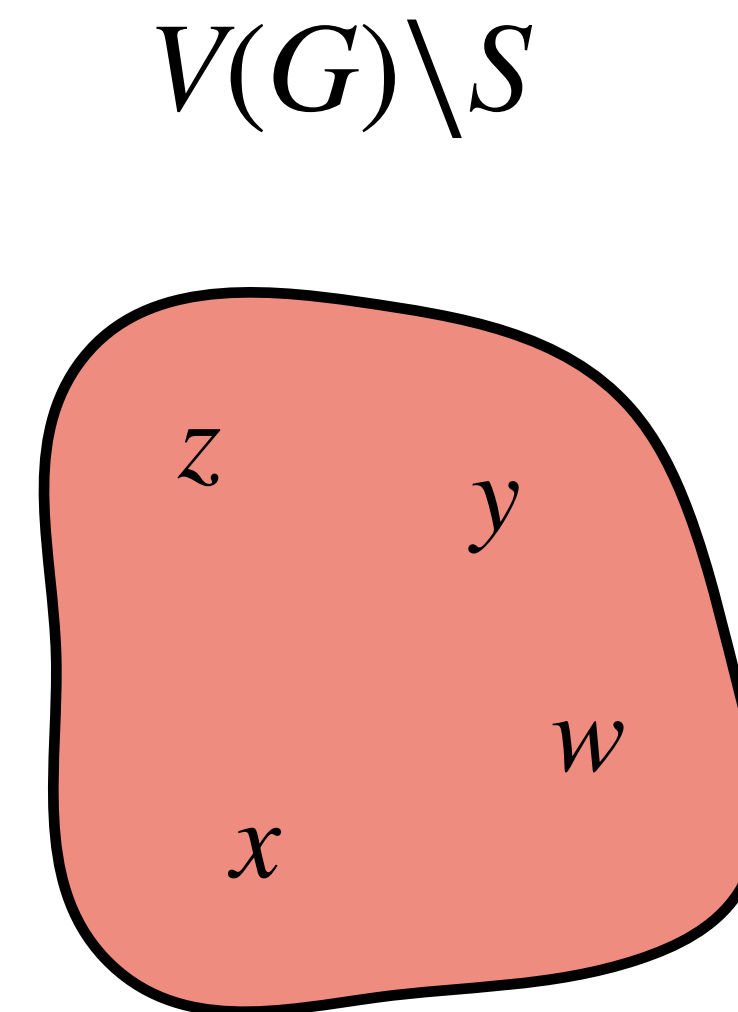
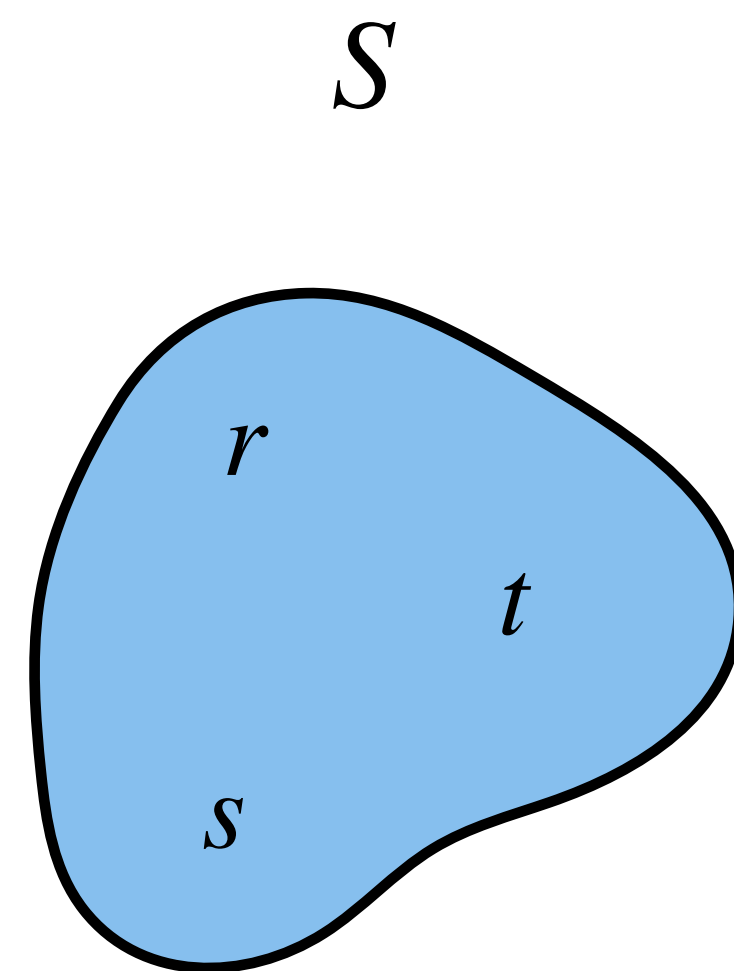
Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$



$$d[s] = 0, d[r] = 3, d[t] = 2$$

Dijkstra's Algorithm: Sketch

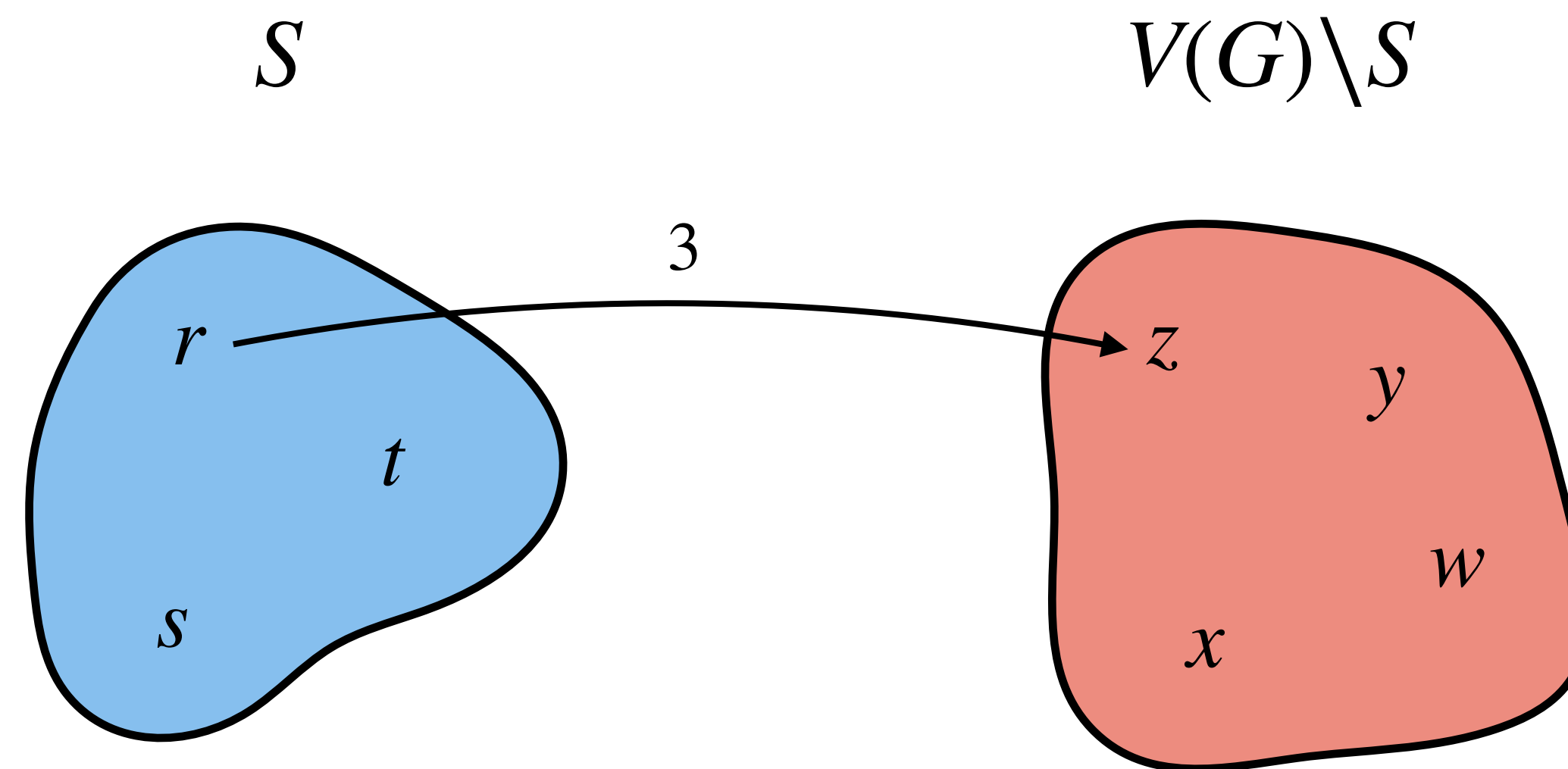
Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$



$$d[s] = 0, d[r] = 3, d[t] = 2$$

Dijkstra's Algorithm: Sketch

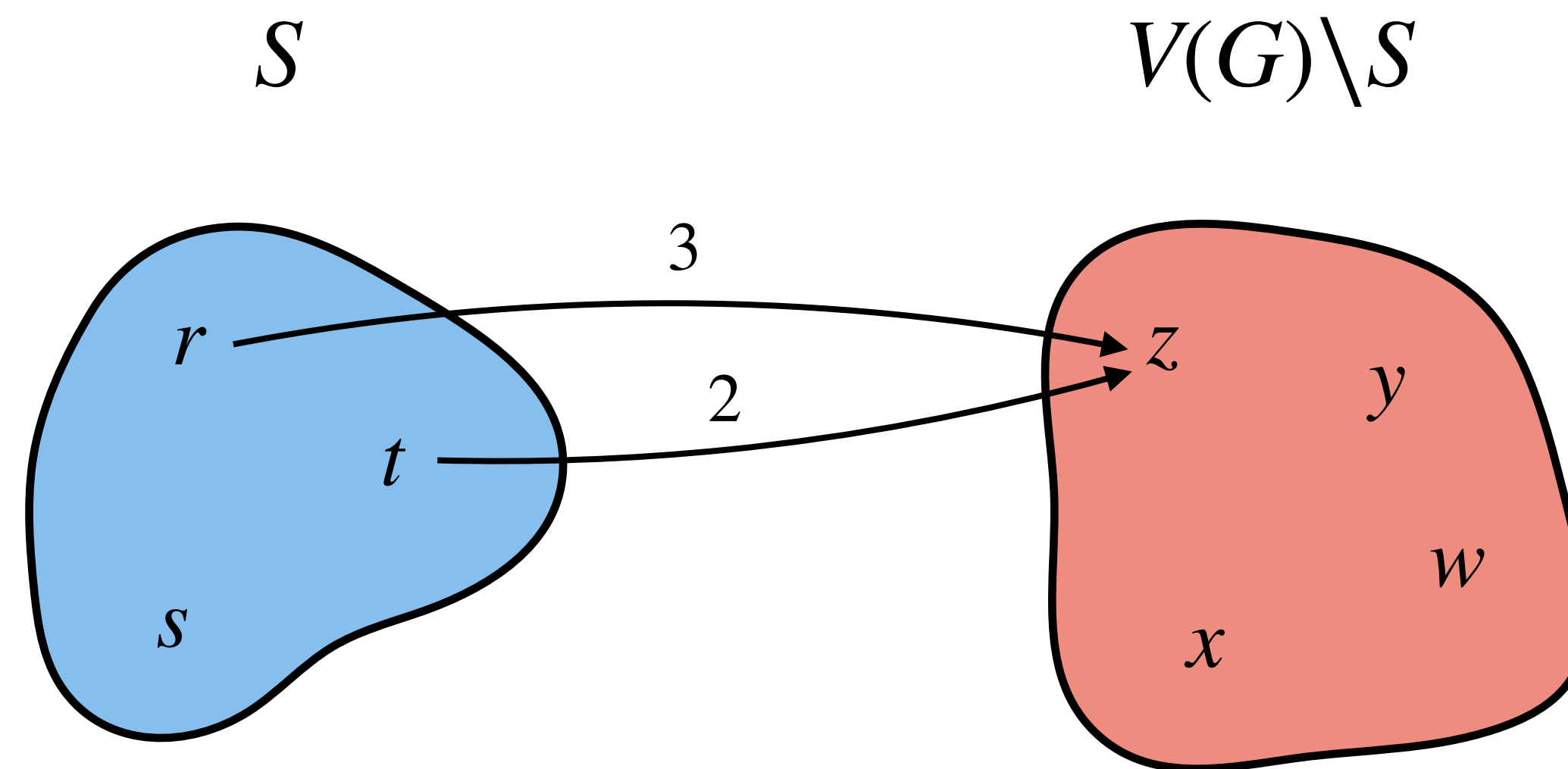
Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$



$$d[s] = 0, d[r] = 3, d[t] = 2$$

Dijkstra's Algorithm: Sketch

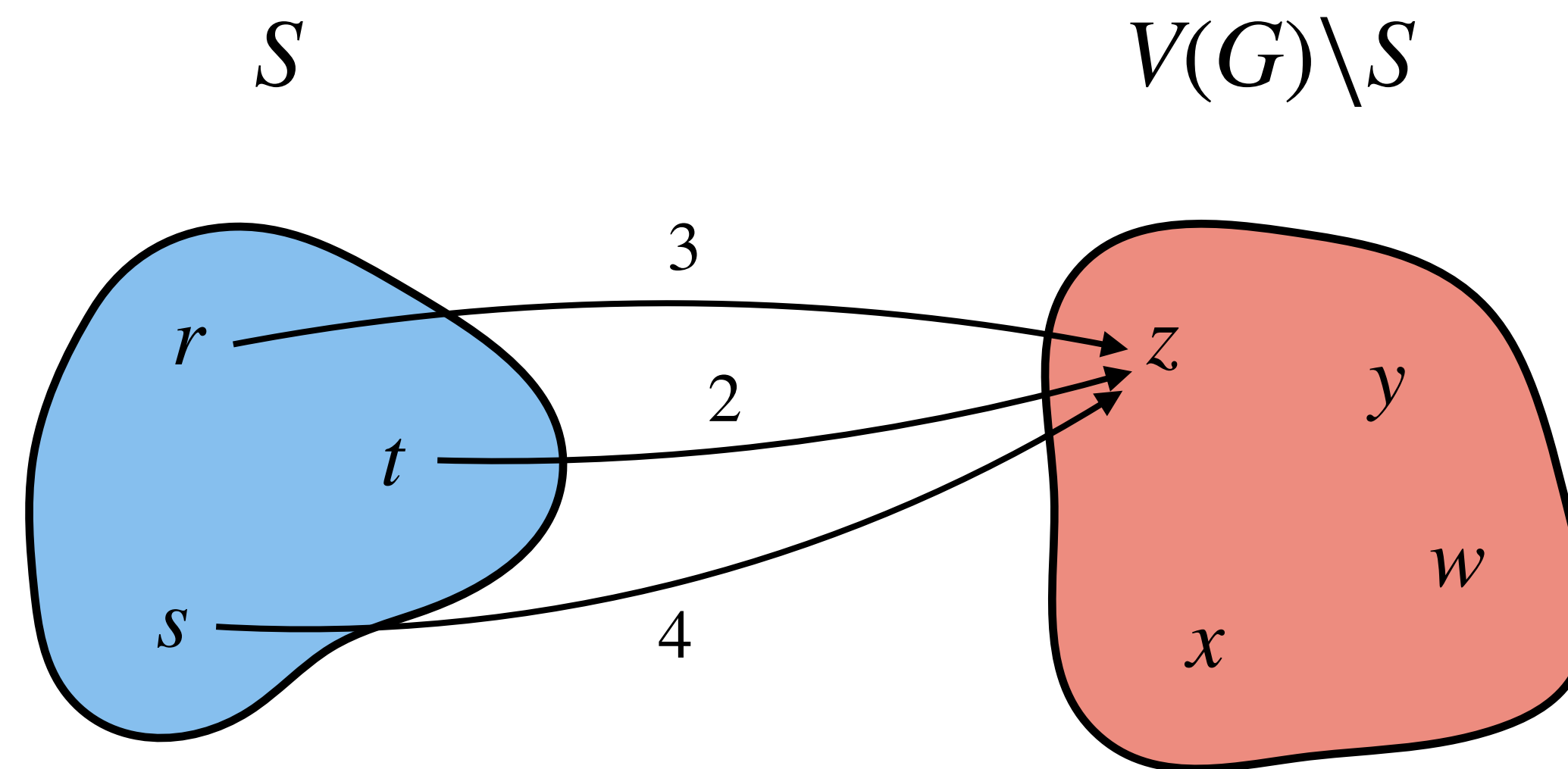
Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$



$$d[s] = 0, d[r] = 3, d[t] = 2$$

Dijkstra's Algorithm: Sketch

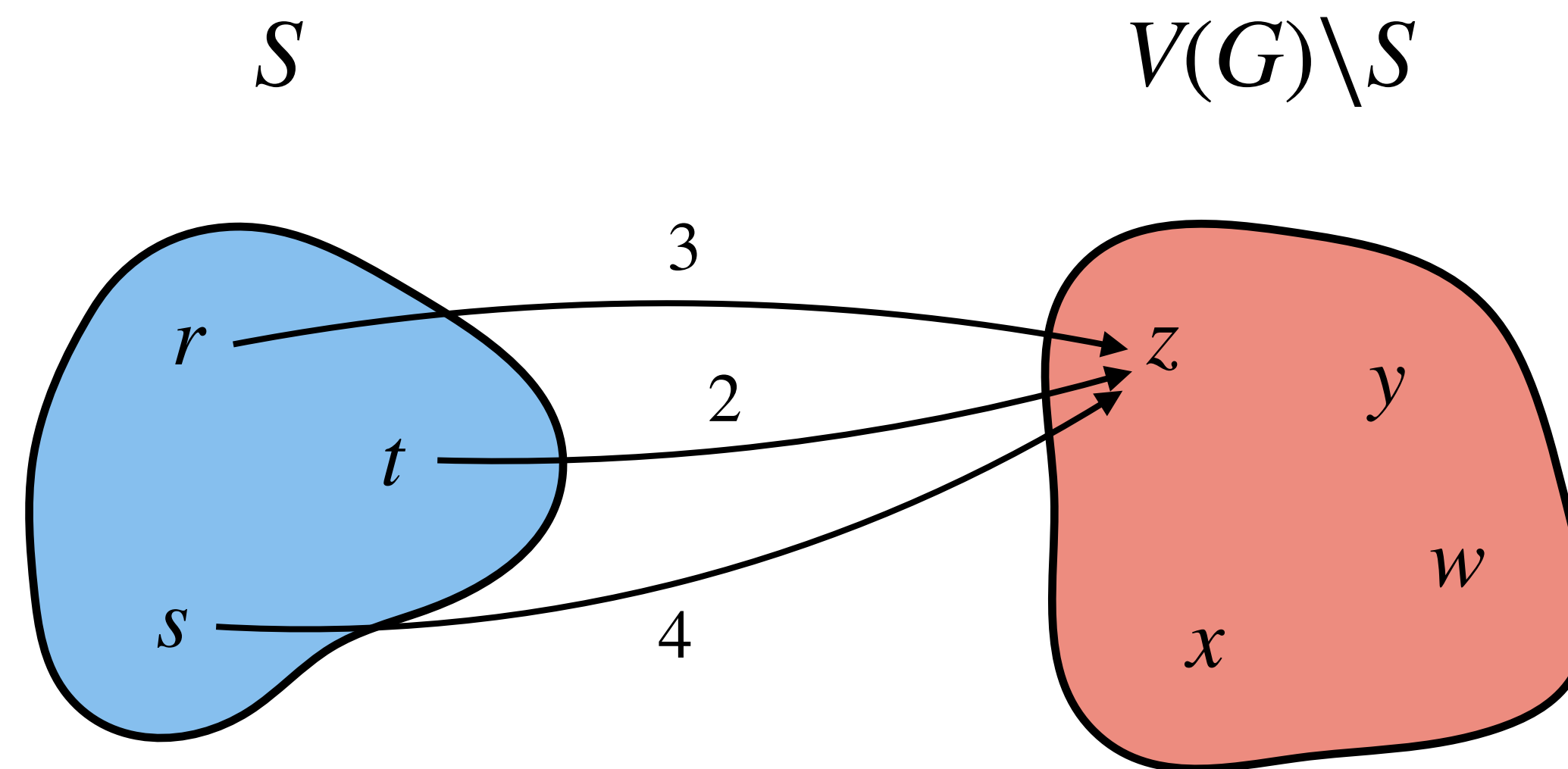
Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$



$$d[s] = 0, d[r] = 3, d[t] = 2$$

Dijkstra's Algorithm: Sketch

Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$

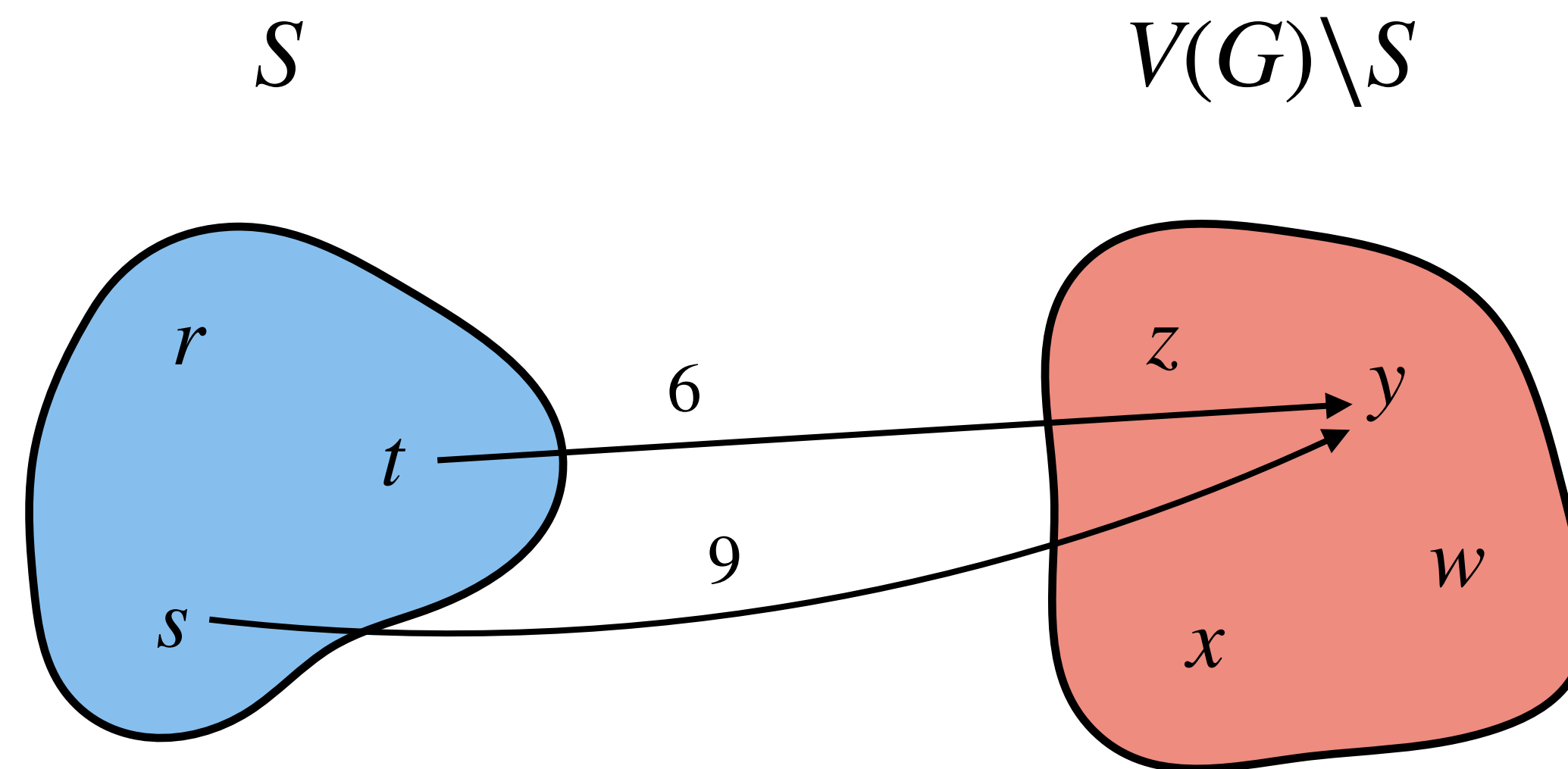


$$d[s] = 0, d[r] = 3, d[t] = 2$$

$$\pi[z] = 4$$

Dijkstra's Algorithm: Sketch

Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$

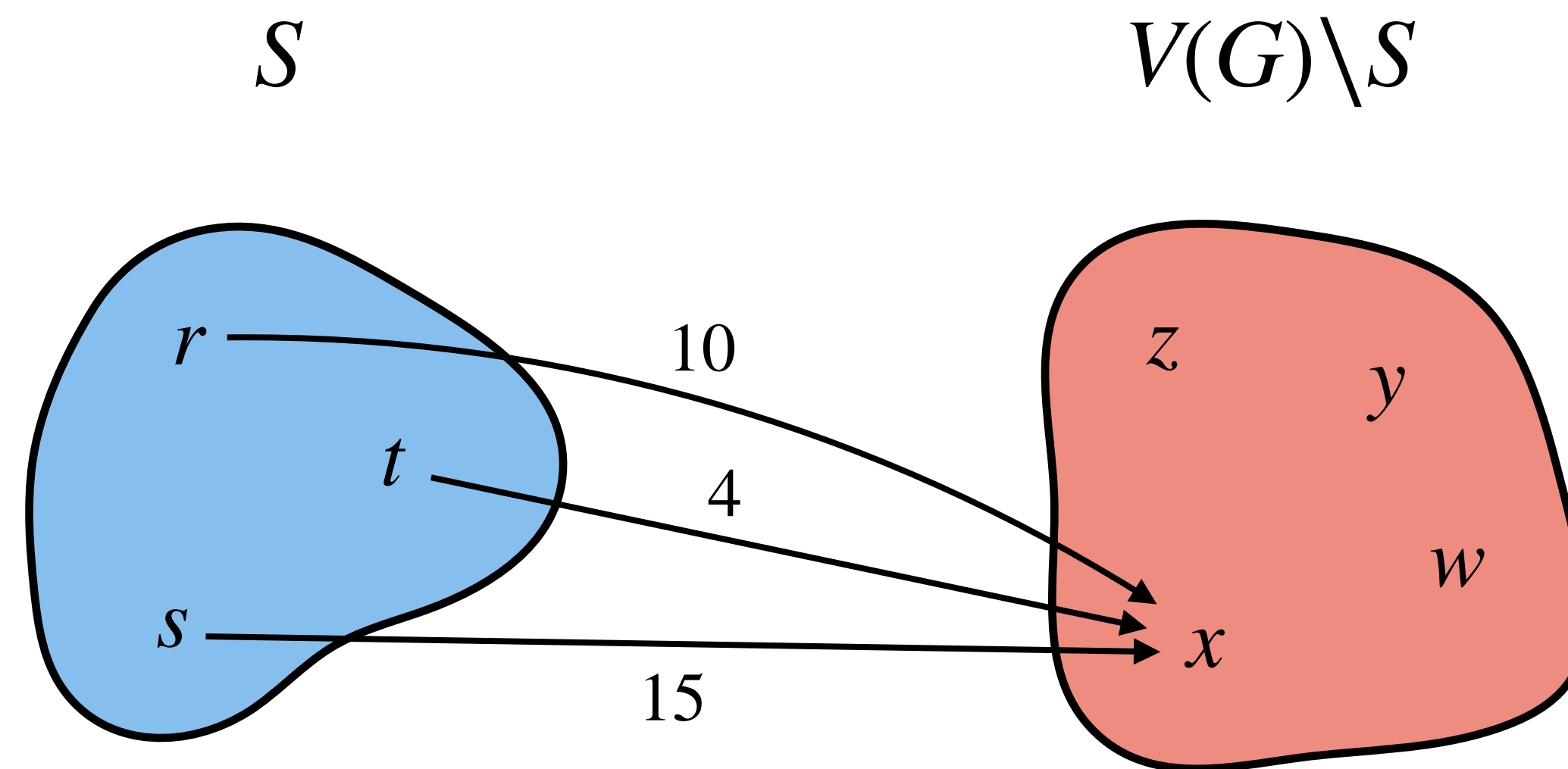


$$d[s] = 0, d[r] = 3, d[t] = 2$$

$$\pi[z] = 4, \pi[y] = 8,$$

Dijkstra's Algorithm: Sketch

Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$

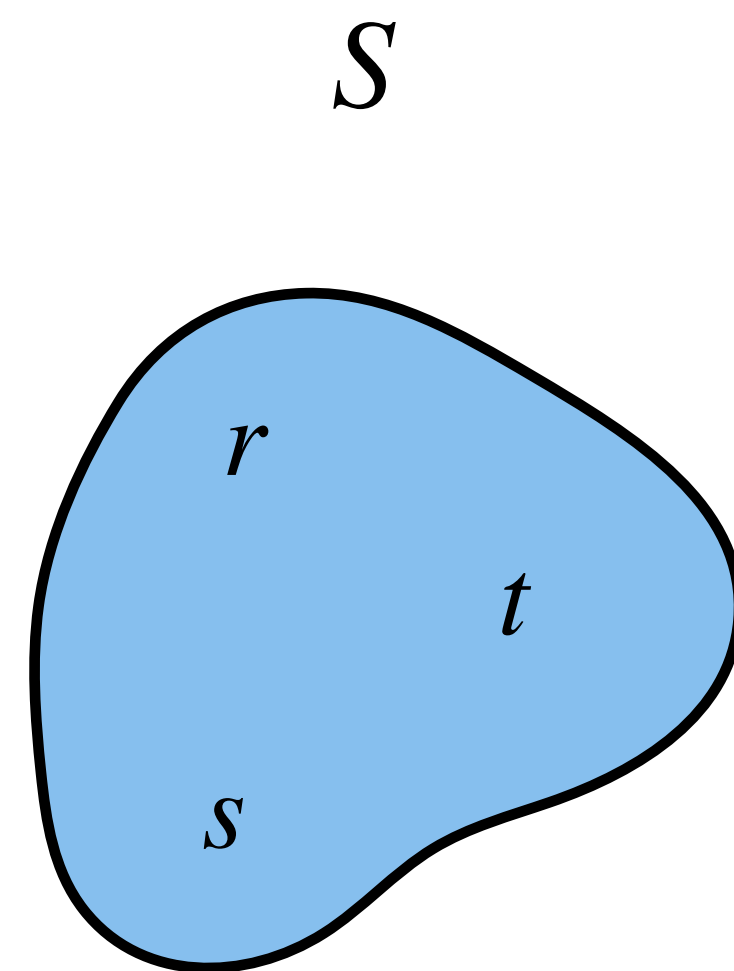


$$d[s] = 0, d[r] = 3, d[t] = 2$$

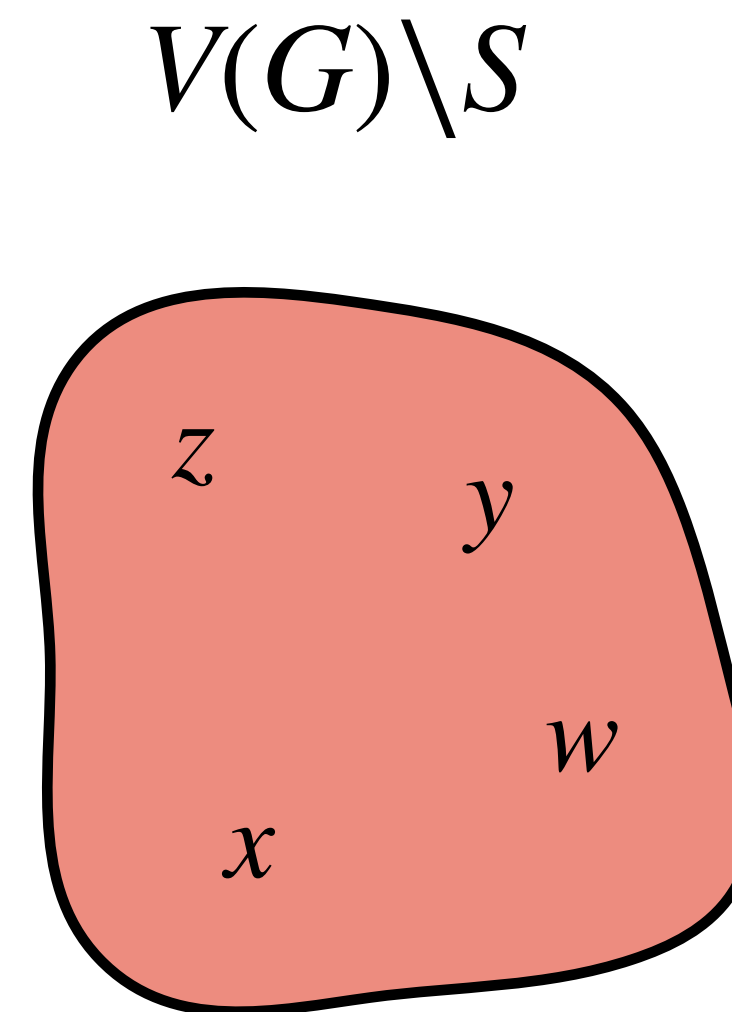
$$\pi[z] = 4, \pi[y] = 8, \pi[x] = 6,$$

Dijkstra's Algorithm: Sketch

Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$



$d[s] = 0, d[r] = 3, d[t] = 2$

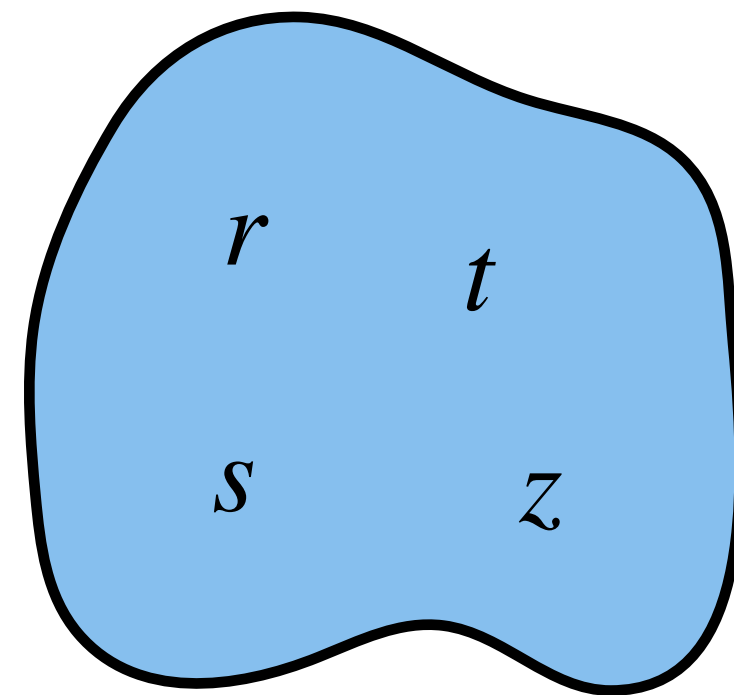


$\pi[z] = 4, \pi[y] = 8, \pi[x] = 6, \pi[w] = \text{Invalid}$

Dijkstra's Algorithm: Sketch

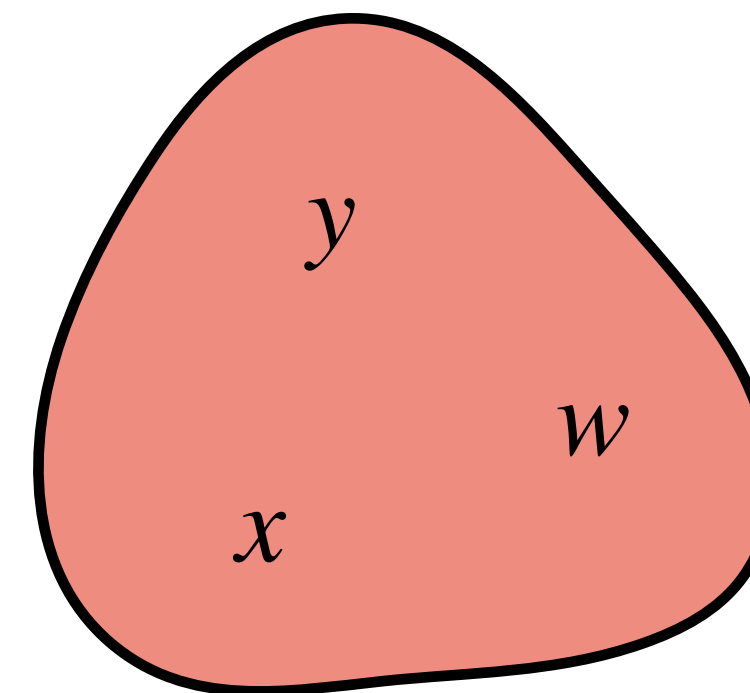
Computing $\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$

S



$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

$V(G) \setminus S$



$\pi[y] = 8, \pi[x] = 6, \pi[w] = \text{Invalid}$

Dijkstra's Algorithm: Sketch

Maintain a set of explored vertices S for which algorithm has found $d[u] = \delta(s, u)$:

Step 1: Initialise $S = \{s\}$, $d[s] = 0$.

Step 2: Choose an unexplored vertex v from $V(G) \setminus S$ which minimizes:

$$\pi[v] = \min_{(u,v) \in E, u \in S} d[u] + w(u, v)$$

Add v to S and set $d[v] = \pi[v]$.

Step 3: Go to **Step 2** if it can be performed.

Dijkstra's Algorithm: Correctness

Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof:

Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step:

Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

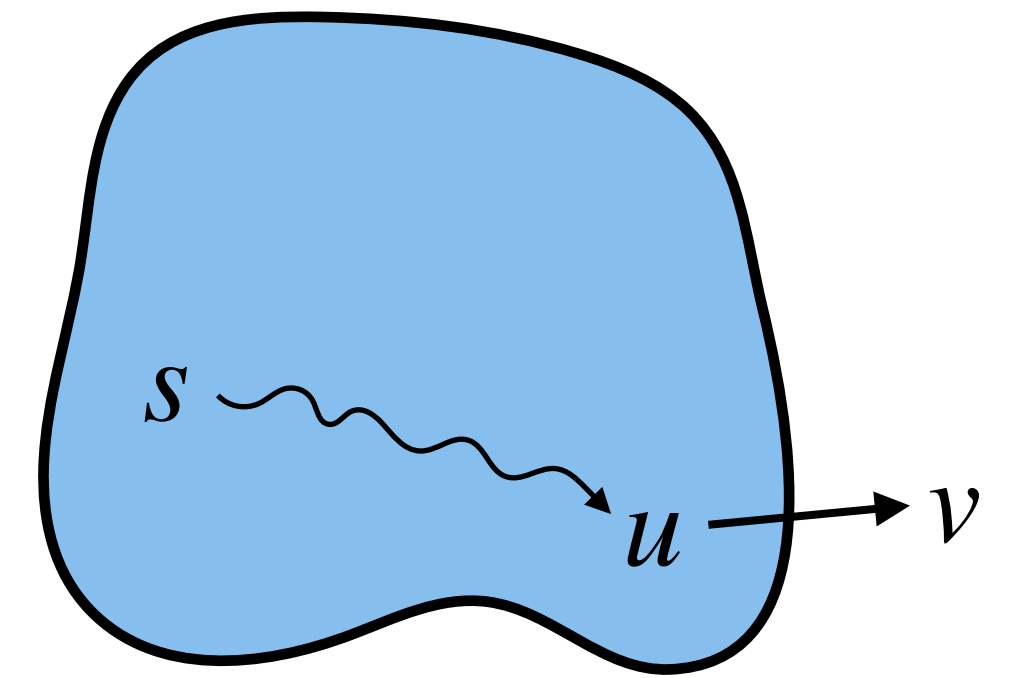
Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .



Dijkstra's Algorithm: Correctness

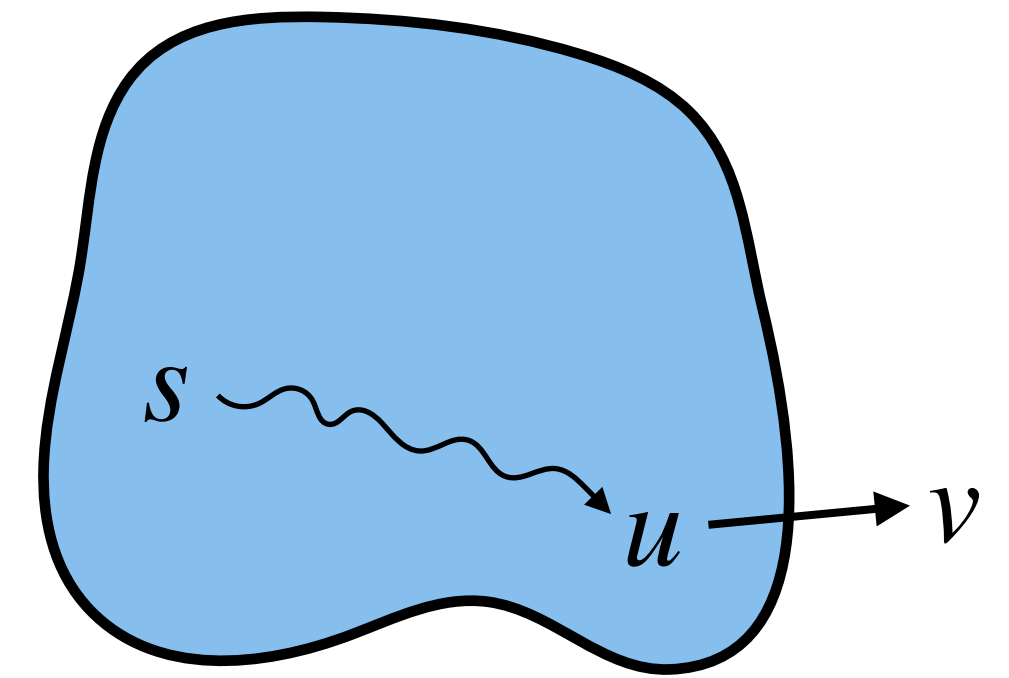
Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

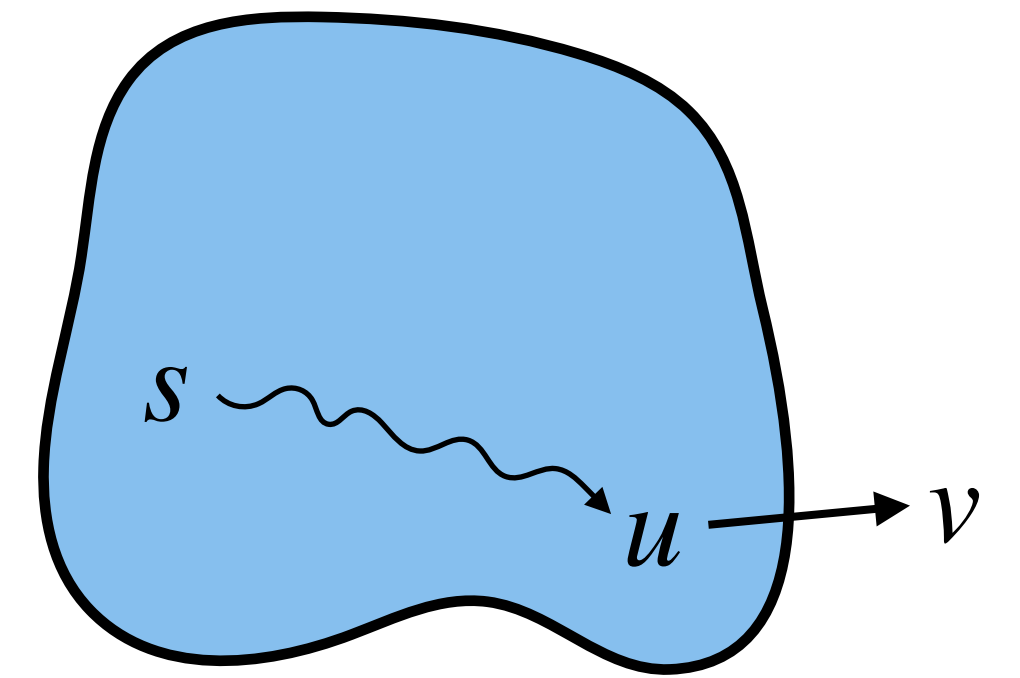
Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v)$$



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

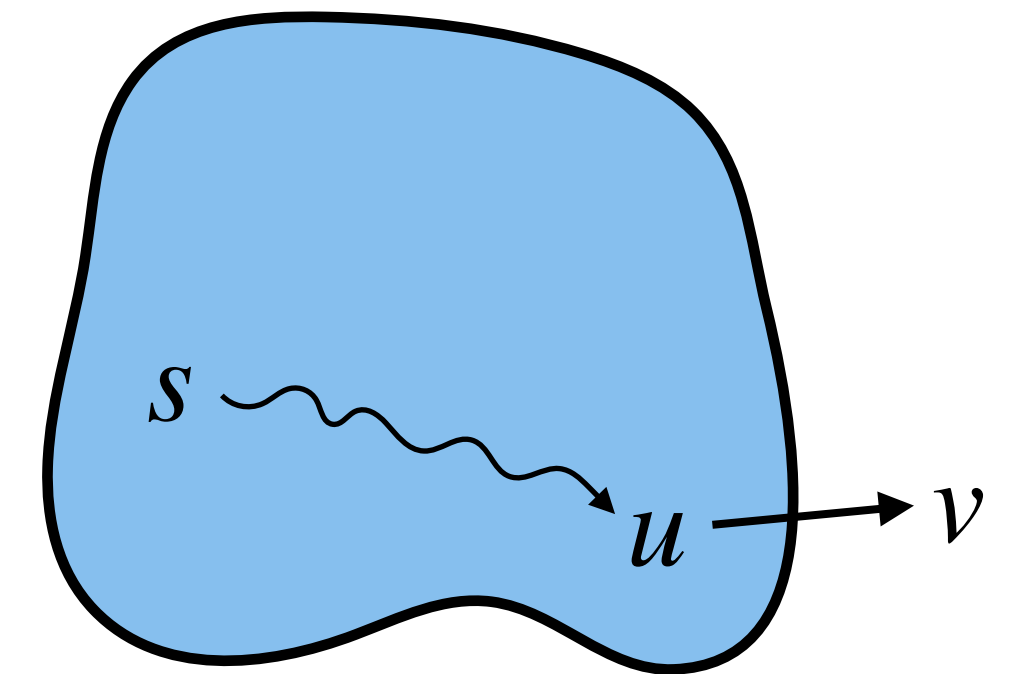
Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

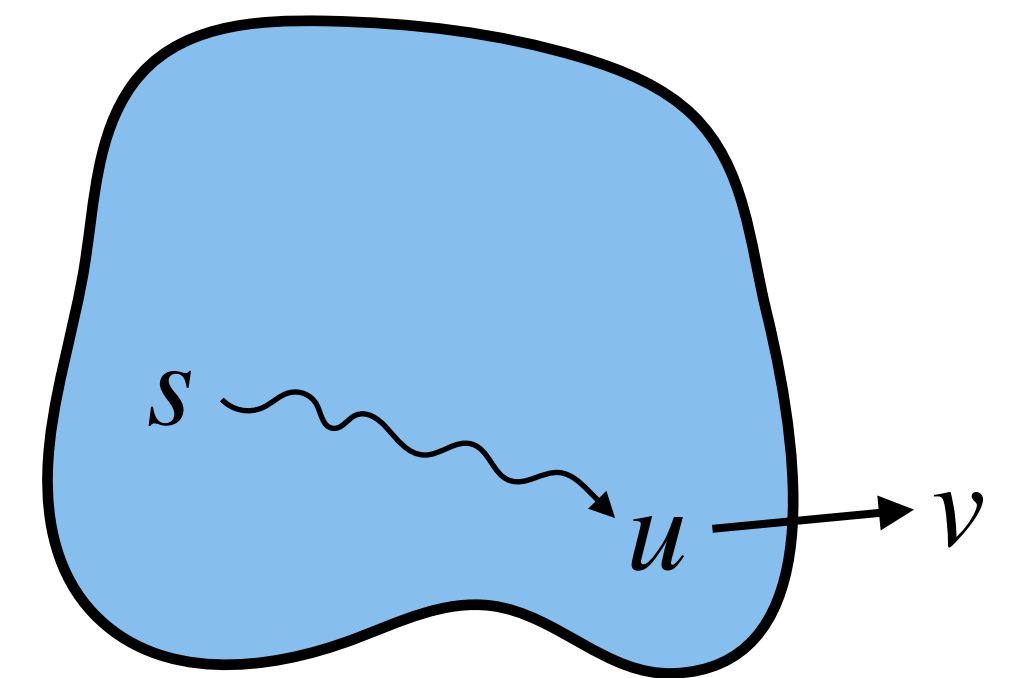
Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v .



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

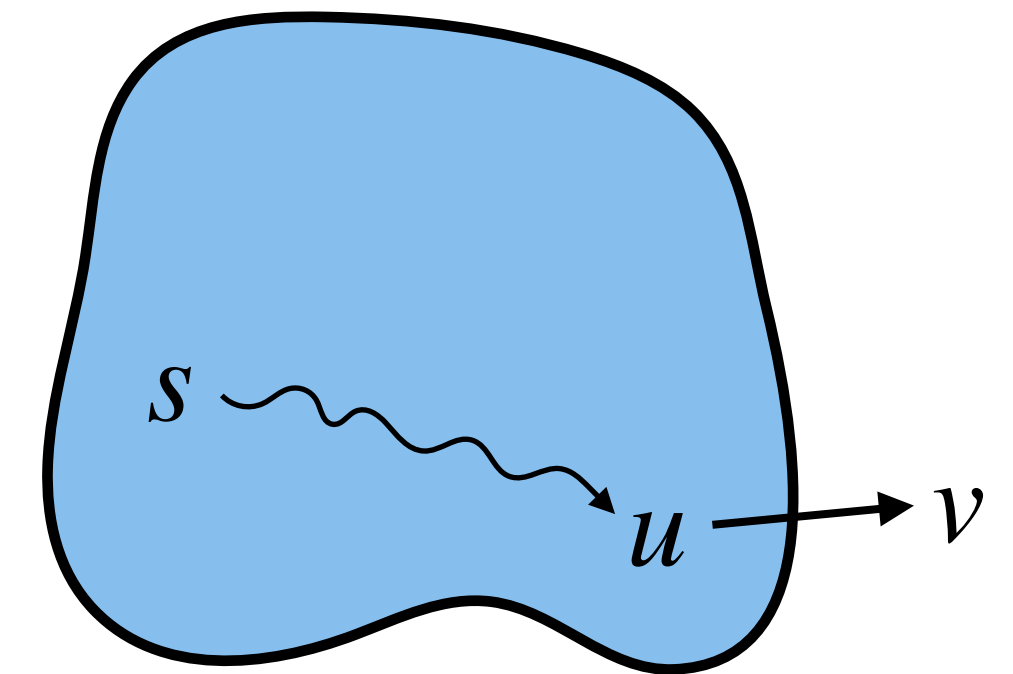
Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

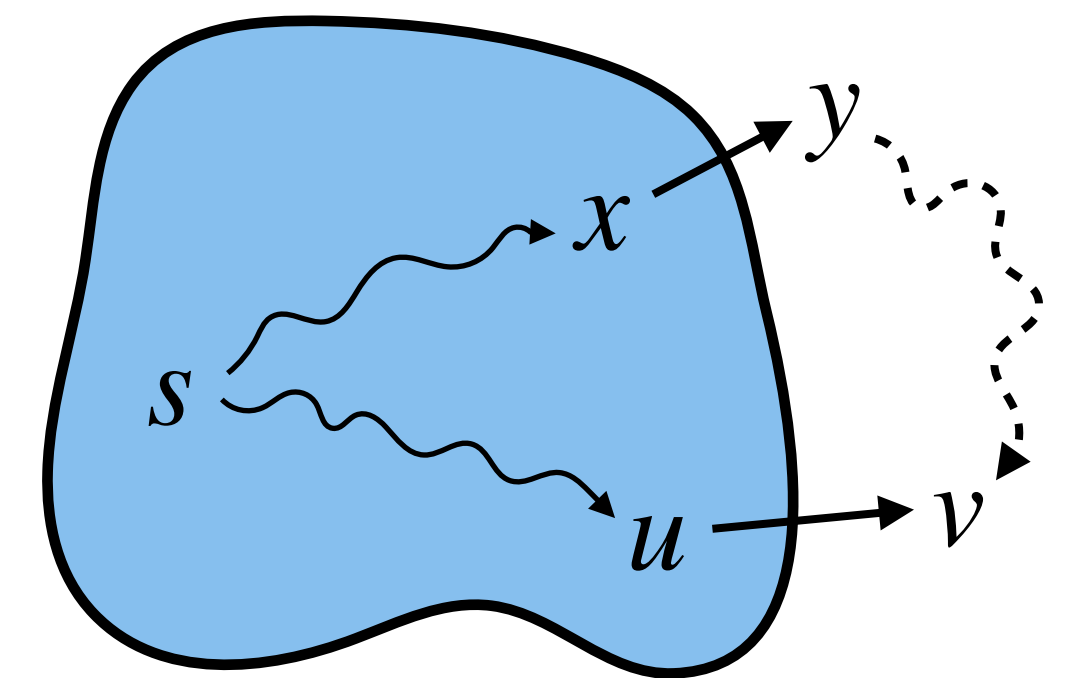
Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

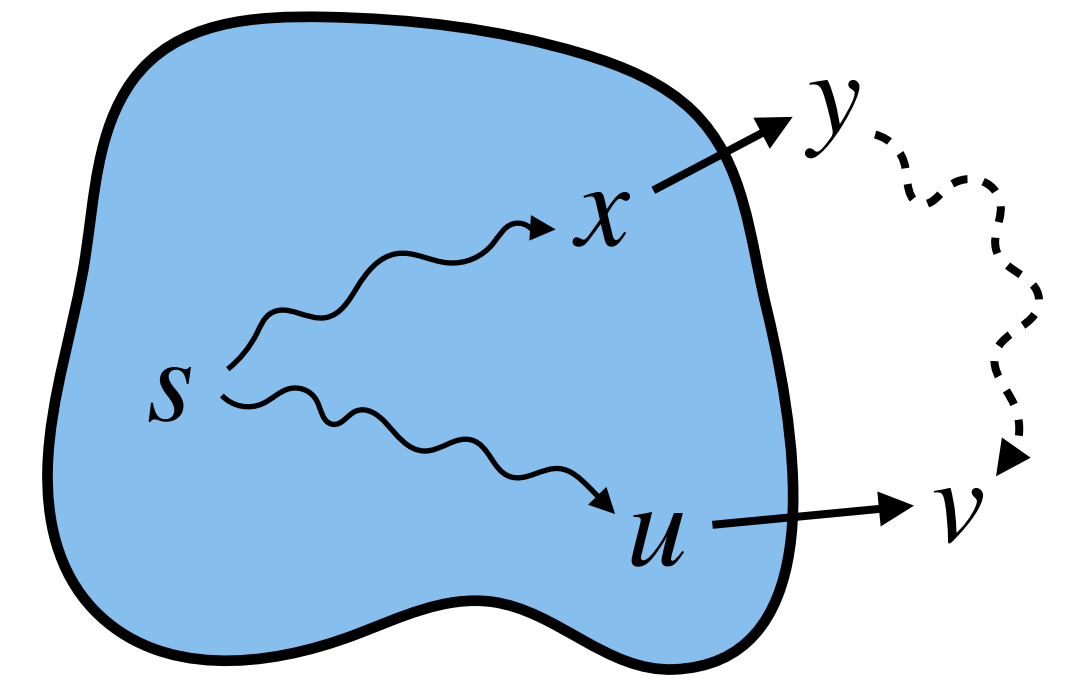
Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

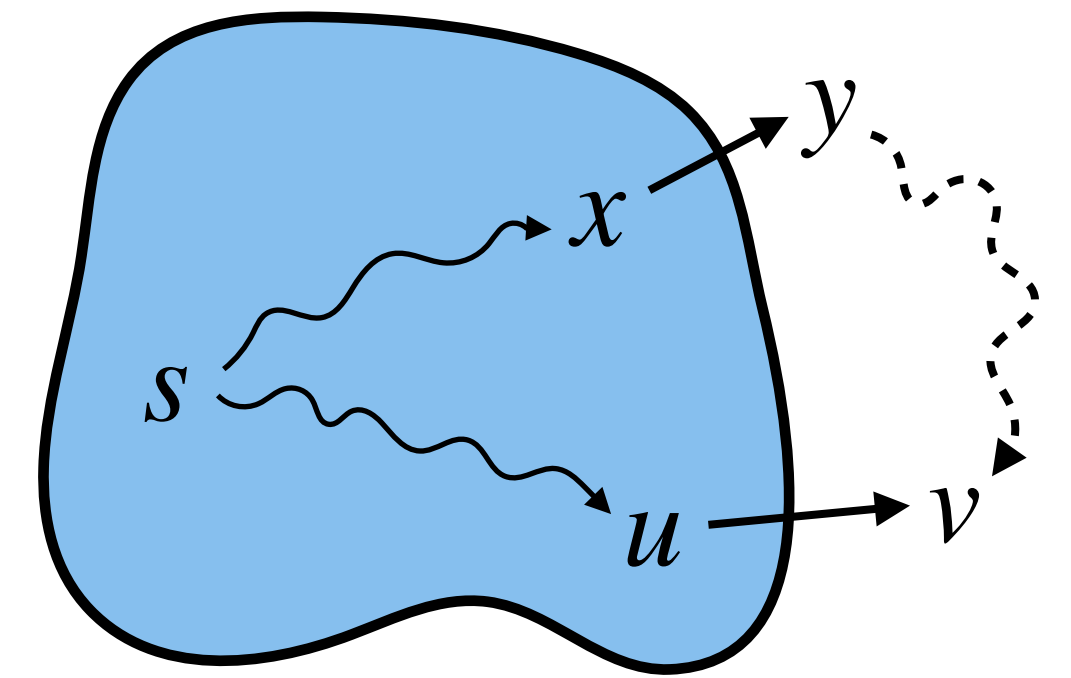
Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.

$$w(P)$$



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

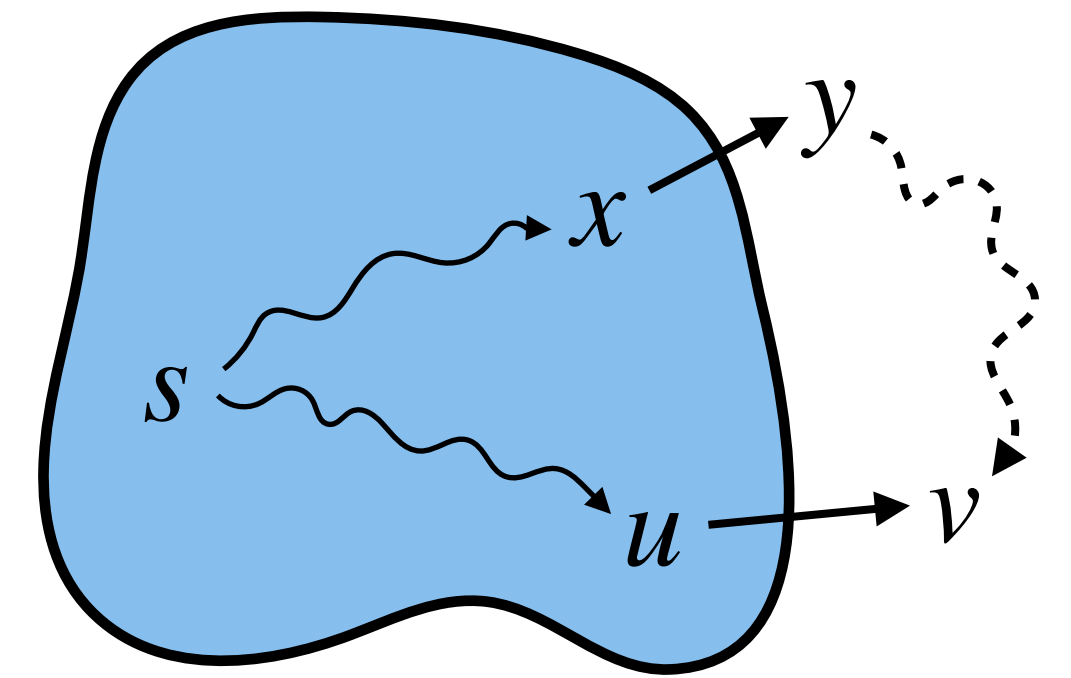
Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.

$$w(P) = \pi(v)$$



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

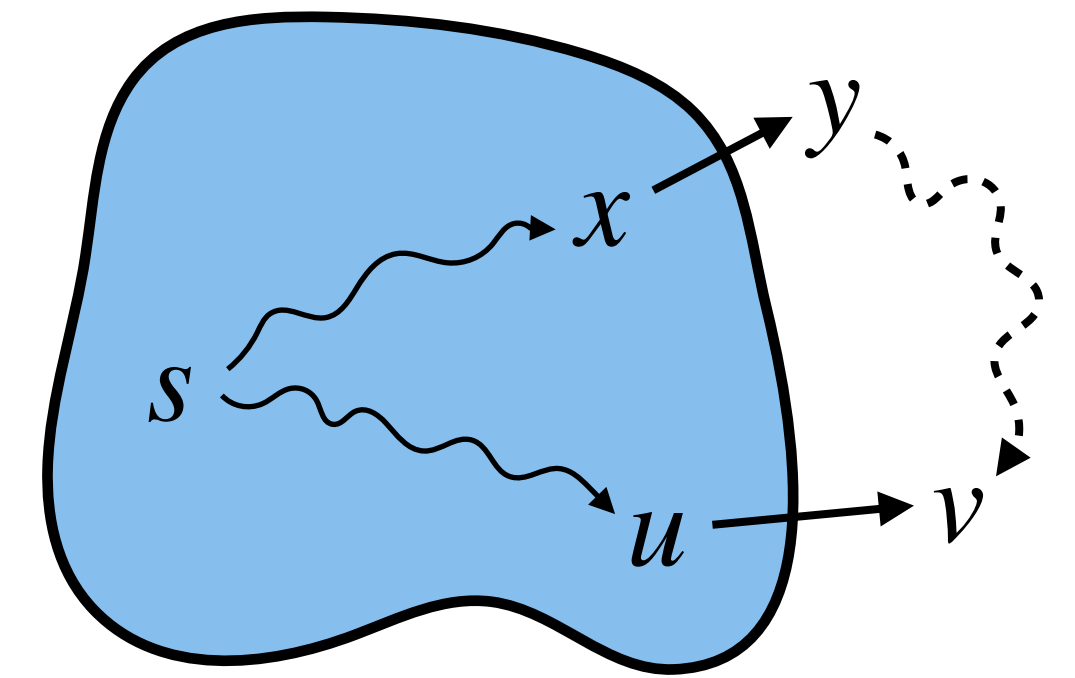
Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.

$$w(P) = \pi(v)$$



Shown above



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

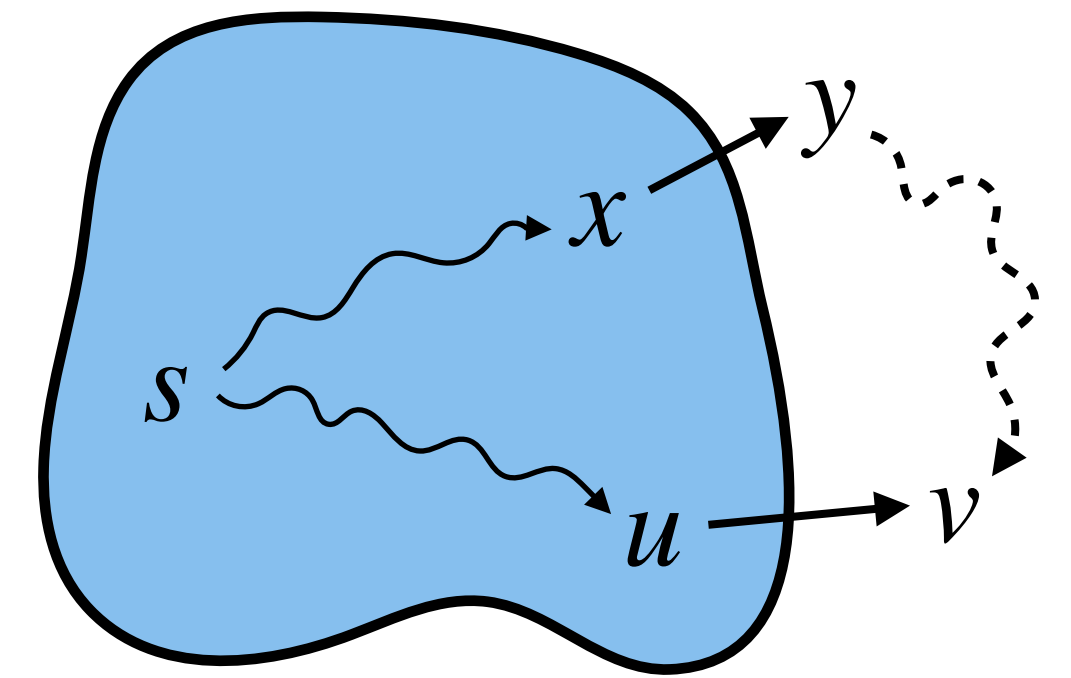
Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.

$$w(P) = \pi(v) \leq \pi(y)$$



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

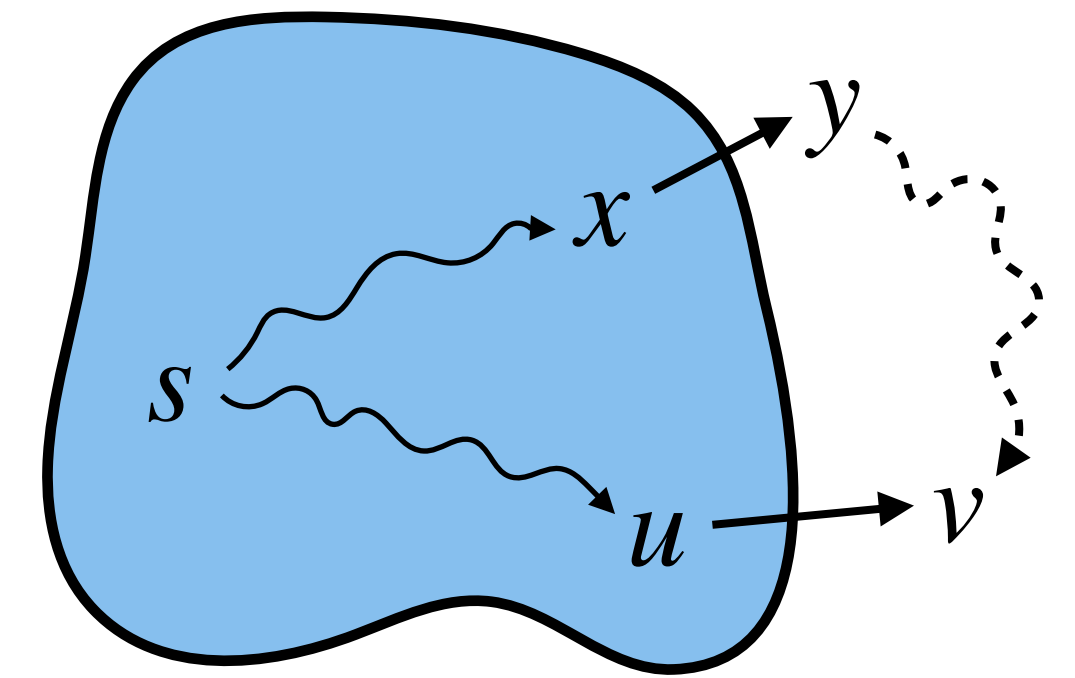
Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.

$$w(P) = \pi(v) \leq \pi(y)$$



Dijkstra chose v over y



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

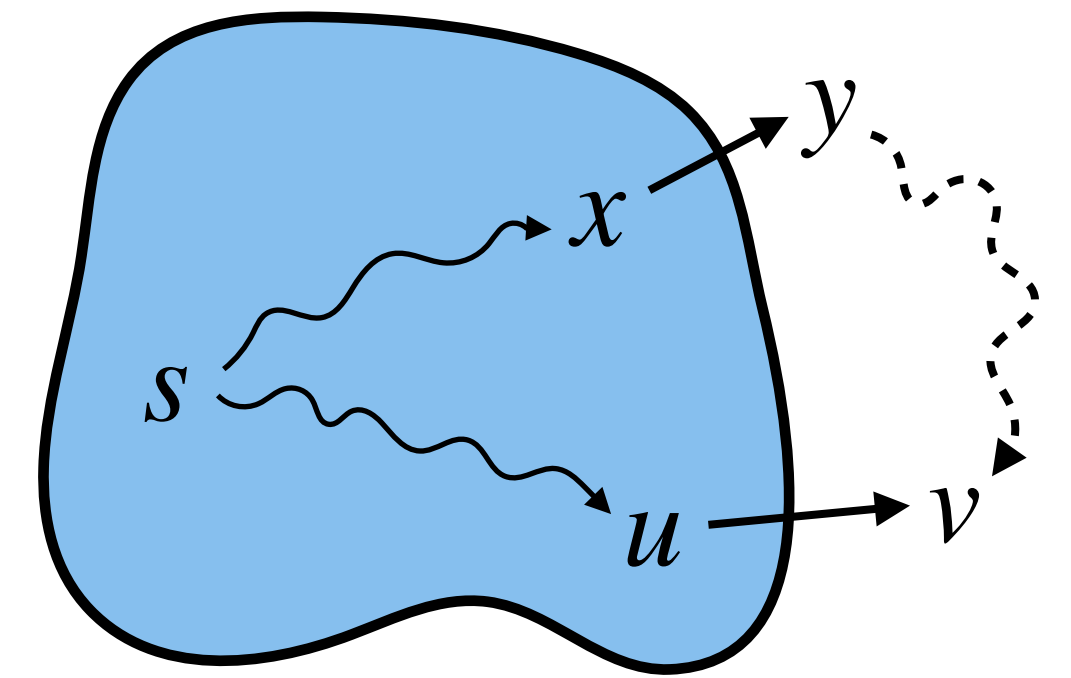
Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.

$$w(P) = \pi(v) \leq \pi(y) \leq d[x] + w(x, y)$$



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

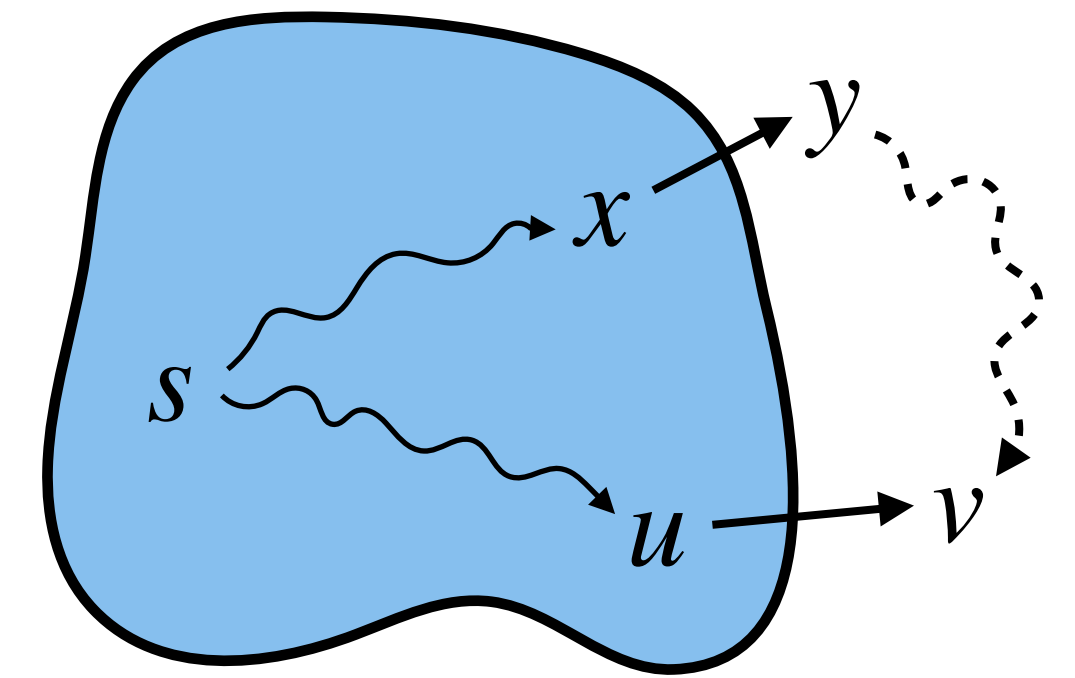
$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.

$$w(P) = \pi(v) \leq \pi(y) \leq d[x] + w(x, y)$$

↑
Definition of π



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

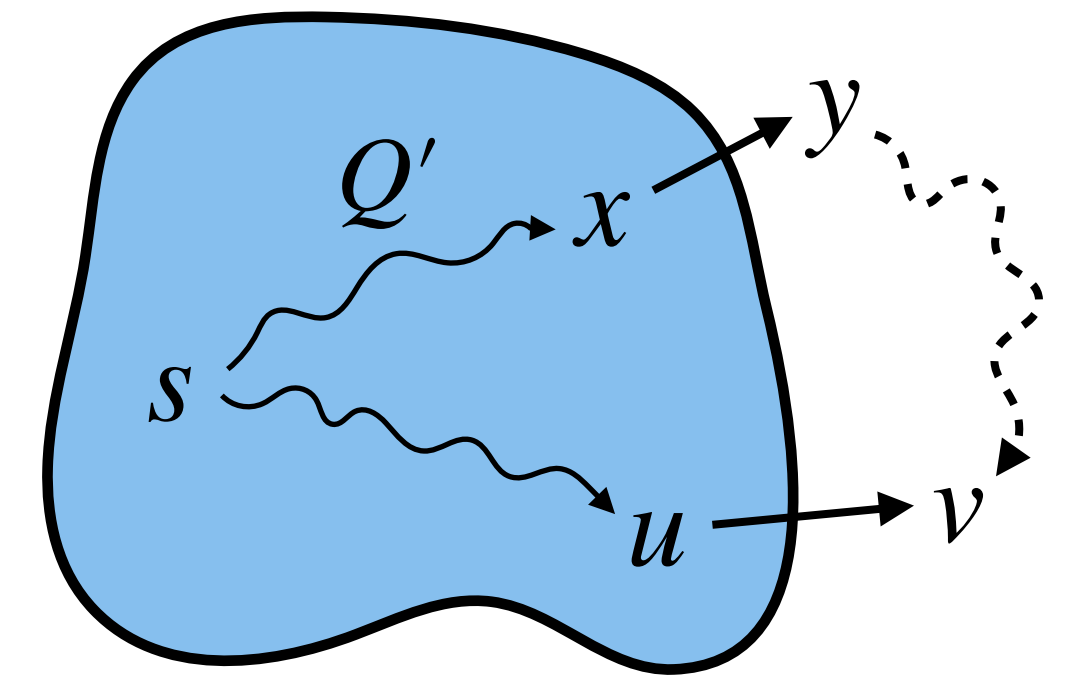
Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.

$$w(P) = \pi(v) \leq \pi(y) \leq d[x] + w(x, y) \leq w(Q') + w(x, y)$$



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

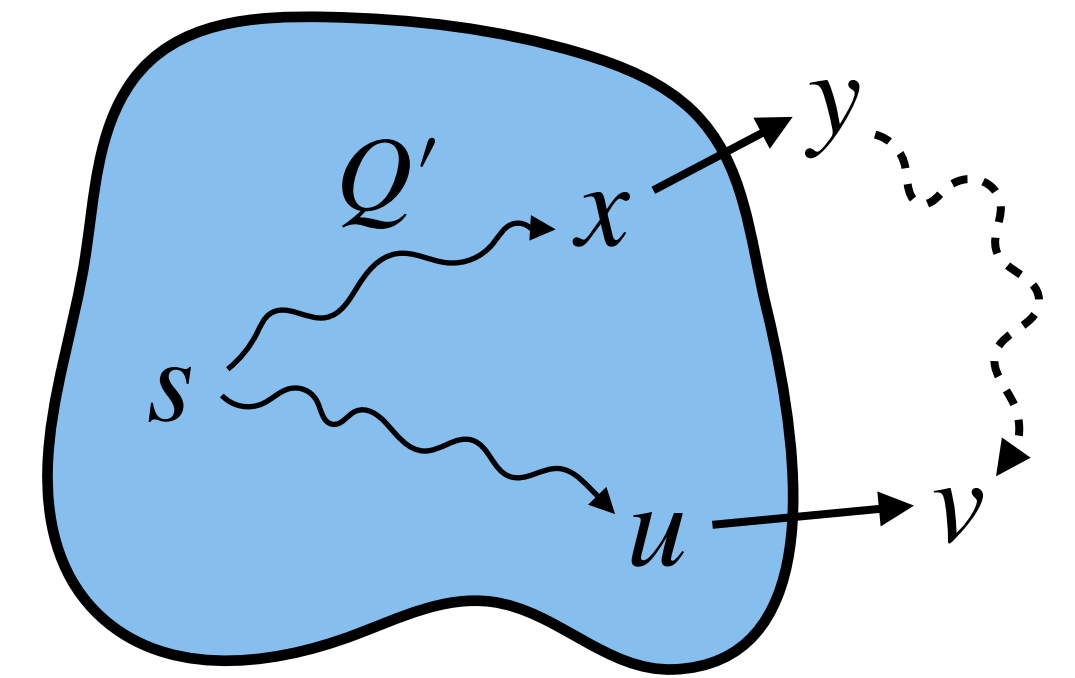
$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.

$$w(P) = \pi(v) \leq \pi(y) \leq d[x] + w(x, y) \leq w(Q') + w(x, y)$$

↑
Because x is in S



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

Let v be the next vertex to be added to S through edge (u, v) .

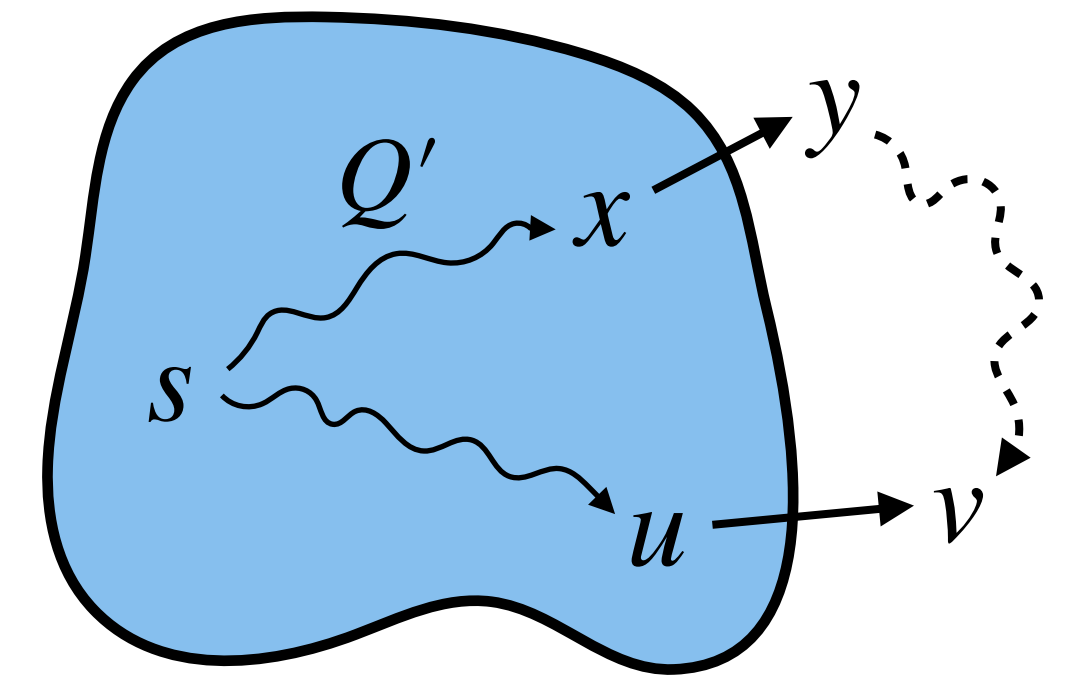
Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.

$$w(P) = \pi(v) \leq \pi(y) \leq d[x] + w(x, y) \leq w(Q') + w(x, y) \leq w(Q)$$



Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

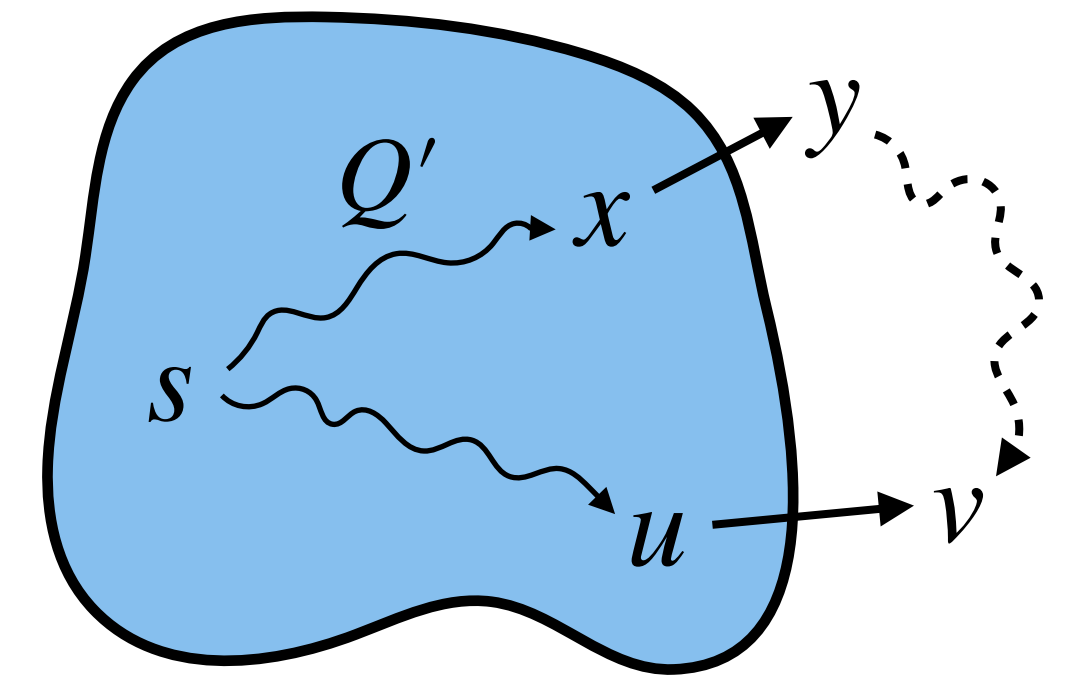
Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.



$$w(P) = \pi(v) \leq \pi(y) \leq d[x] + w(x, y) \leq w(Q') + w(x, y) \leq w(Q)$$

Because y to v subpath in Q will have non-negative weights

Dijkstra's Algorithm: Correctness

Theorem: For every $u \in S$, $d[u] = \delta(s, u)$.

Proof: Basis Step: When $|S| = 1$, statement is trivially true as $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$.

Inductive Step: Assume the statement is true for some S such that $|S| \geq 1$.

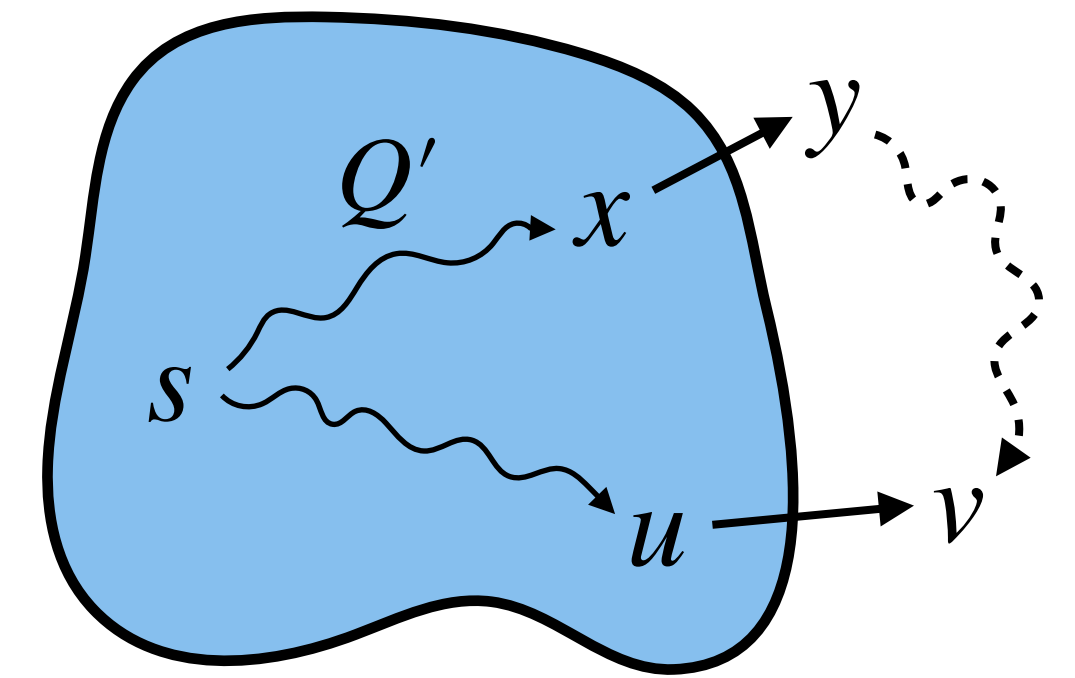
Let v be the next vertex to be added to S through edge (u, v) .

Path P from s to v = Shortest $s \rightsquigarrow u$ path followed by edge (u, v)

$$w(P) = d[u] + w(u, v) = \pi[v]$$

Take any other path Q from s to v . We claim that $w(Q) \geq w(P)$.

Let (x, y) be the first edge on Q going from S to $V \setminus S$.



$$w(P) = \pi(v) \leq \pi(y) \leq d[x] + w(x, y) \leq w(Q') + w(x, y) \leq w(Q)$$

Because y to v subpath in Q will have non-negative weights